POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

Different Approaches for Lift Coefficient Prediction on an Airfoil: Classical CFD and Deep Learning Techniques

Project for the course of Advanced Methods for Scientific Computing

Giulio Enzo Donninelli, Alessia Rigoni, Vittorio Sironi and Alessandro Verrengia

Advisor:
Prof. Luca Dedè

Academic year:
2025-2026

Abstract: This work presents an integrated computational pipeline for rapid aerodynamic analysis, bridging high-fidelity simulations with a data-driven surrogate model. We extended the `LifeX` C++ finite element library with a Reynolds-Averaged Navier-Stokes (RANS) solver using the Spalart-Allmaras turbulence model. This solver generated a reference dataset on the CINECA Leonardo HPC cluster, which was used to train a Convolutional Neural Network (CNN) built from scratch in C++ . The CNN predicts lift coefficients from a Signed Distance Function (SDF) geometry representation and is trained in parallel using MPI. The model achieves a validation Mean Squared Error of 0.0104 and a 35× speedup over direct simulation, providing a robust framework for efficient aerodynamic design.

1. Introduction

Aerodynamic design balances a trade-off between the demand for physical fidelity and the need for rapid design iteration. High-fidelity Computational Fluid Dynamics (CFD) simulations, such as those based on the Reynolds-Averaged Navier-Stokes (RANS) equations, offer unparalleled accuracy in predicting complex flow phenomena. However, their significant computational cost makes them impractical for design-space exploration or real-time analysis. Conversely, data-driven surrogate models, especially deep learning architectures like Convolutional Neural Networks (CNNs), can provide near-instantaneous predictions but are entirely dependent on the quality and availability of the high-fidelity data used for their training.

This project develops a pipeline that integrates both methods. Inspired by the framework of Cuozzo et al. (2026), we first build a high-performance CFD solver to generate reference-quality aerodynamic data. We then use this data to train a custom CNN surrogate model. By establishing this workflow for clean symmetric airfoils, we made a base idea for future extensions to more complex iced geometries.

The document is structured to follow this approach, beginning with the development of the high-fidelity simulation framework. Section 2 (**Navier-Stokes Simulation**) details the extension of the `LifeX` C++ finite element library to handle external aerodynamic flows. We describe the integration of the one-equation Spalart-Allmaras RANS model, the adaptation of the solver to two-dimensional domains, the implementation of specialized boundary conditions and the development of utilities for computing aerodynamic forces. This section also provides a comprehensive overview of the High-Performance Computing (HPC) workflow established on the CINECA Leonardo cluster.

Building upon the dataset generated by these simulations, Section 3 (**Convolutional Neural Network**), presents the design, implementation and training of our surrogate model. A key contribution of this work is

the development of a CNN entirely from scratch in C++ , designed to predict the lift coefficient from an airfoil's geometric representation along with the angle of attack and Reynolds number. We elaborate on the mathematical foundations of the network architecture, the physics-informed loss function that regularizes training and the implementation of distributed-memory parallelism using MPI, enabling efficient execution on HPC hardware.

2. Navier-Stokes Simulation

The main goal of this section of the project is to extend the `LifeX` library with new features, making it a more general-purpose and high-performance simulation framework. Specifically, the work focuses on integrating one-equation **RANS turbulence model** (Spalart-Allmaras) into the existing incompressible **Navier-Stokes solver** (NS), enabling the simulation of turbulent flows in external aerodynamic configurations.

2.1. `LifeX` Implementation

As detailed by Africa et al. (2024), `LifeX` is an open-source C++ library built on top of `deal.II`, originally designed for life-science applications such as cardiac and cardiovascular fluid dynamics. While the library provides a robust and well-structured finite element infrastructure, its original focus on biomedical applications necessitated several non-trivial adaptations to extend it to external aerodynamic flows.

This work addressed four primary challenges:

- adapting the solver and utility functions, originally designed for three-dimensional problems, to two-dimensional domains (Section 2.1.1);
- integrating the one-equation Spalart-Allmaras (SA) RANS model to account for turbulent effects (Section 2.1.2);
- implementing specialized conditions for aerodynamic simulations, including wall functions and far-field boundaries (Section 2.1.3)
- developing a residual-based post-processing utility to compute lift and drag forces (Section 2.1.4).

The following sections detail the implementation of each of these components.

2.1.1. 2D Implementation

While the `LifeX` `FluidDynamics` solver is templated on the spatial dimension `dim` through the `deal.II`convention, its original test cases and utilities were primarily designed for three-dimensional domains. Extending the framework to 2D involved adapting three key components: the finite element function space, the calculation of vorticity and the computation of the wall distance field.

Function space and triangulation

The SA solver uses Q_1 **finite elements** (`FE_Q` of degree 1) on the same triangulation as the NS solver:

$$\tilde{\nu}_h \in V_h = \left\{ v \in H^1(\Omega) \;\middle|\; v\big|_{\partial\Omega_D} = g_D \right\} \tag{1}$$

On the implementation side, the SA Degree of Freedom (DoF) handler (`dof_handler_sa_`) is independent from the NS one (`dof_handler`), in a way that the two systems have disjoint degrees of freedom without altering the block structure of the Navier-Stokes system. The shared triangulation object is the key architectural choice that allows both DoF handlers to be iterated in lockstep during matrix assembly.

Vorticity in 2D

The modified vorticity $\tilde{S}$ entering the SA production term requires the vorticity magnitude $S = |\boldsymbol{\omega}|$. In three dimensions, this is computed as $S = |\nabla \times \mathbf{u}|$.

In a 2D context, it simplifies to the absolute value of the scalar curl:

$$S = \left| \frac{\partial u_y}{\partial x} - \frac{\partial u_x}{\partial y} \right| \tag{2}$$

which is extracted from the velocity gradient tensor $\nabla \mathbf{u}^{n+1}$ interpolated at quadrature points via `fe_values_ns_.get_function_gradients`.

Wall distance in 2D

The wall distance $d(\mathbf{x})$ is computed once during `setup()` via a parallel algorithm. Each MPI rank collects the barycenters of boundary faces tagged as `prm_wall_tag_` (airfoil surface), serializes them into a flat `double` array of size $N_{\text{face,loc}} \times 2$, and participates in a global `MPI_Allgatherv` to build the complete set $\{\mathbf{c}_w\}_{w=1}^{N_{\text{face,tot}}}$.

The distance at every locally owned SA node i is then:

$$d_i = \max\Big(\min_w \|\mathbf{x}_i - \mathbf{c}_w\|_2,\ 10^{-12} \Big) \tag{3}$$

where the lower clamp prevents division by zero for nodes sitting exactly on the wall boundary.

The result is stored in `wall_distance_ghosted_`, a Trilinos ghosted vector indexed on `dof_handler_sa_`, following the standard pattern:

1. write into an owned vector and call `compress(VectorOperation::insert)`.
2. copy owned $\rightarrow$ ghosted to propagate ghost entries across MPI ranks.

The overall complexity per rank is $O(N_{\text{face,tot}} \cdot N_{\text{nodes,loc}})$, which is acceptable for the 2D meshes used in the airfoil example.

2.1.2. Turbulence Model

To enable the simulation of turbolent flows, the framework was extended with the one-equation SA RANS model, introduced by Spalart and Allmaras (1992). This model introduces a transport equation for a scalar variable known as the *modified kinematic viscosity*, $\tilde{\nu}$:

$$\boxed{\frac{\partial \tilde{\nu}}{\partial t} + \mathbf{u} \cdot \nabla \tilde{\nu} = \underbrace{c_{b1} \tilde{S} \tilde{\nu}}_{\text{production}} - \underbrace{c_{w1} f_w \left(\frac{\tilde{\nu}}{d} \right)^2}_{\text{destruction}} + \underbrace{\frac{1}{\sigma} \Big[\nabla \cdot \big((\nu + \tilde{\nu}) \nabla \tilde{\nu} \big) + c_{b2} |\nabla \tilde{\nu}|^2 \Big]}_{\text{diffusion + cross-diffusion}}} \tag{4}$$

The model constants (Table 1) follow the standard set by **NASA Turbulence Modeling Resource (TMR)** without the f_{t2} trip term.

Table 1: Spalart-Allmaras model constants based on the NASA TMR standard.

Constant	Value	Role
c_{b1}	0.1355	production coefficient
c_{b2}	0.622	cross-diffusion coefficient
σ	2/3	turbulent Prandtl number
κ	0.41	von Kármán constant
c_{v1}	7.1	viscous damping
c_{w1}	≈ 3.2391	destruction (derived from log-law consistency)
c_{w2}	0.3	regularization in f_w
c_{w3}	2.0	regularization in f_w

Closure functions

The model relies on a set of non-linear closure functions to relate $\tilde{\nu}$ to the turbulent eddy viscosity. Defining the viscosity ratio $\chi = \tilde{\nu}/\nu$, these functions are:

$$f_{v1}(\chi) = \frac{\chi^3}{\chi^3 + c_{v1}^3}, \qquad f_{v2}(\chi) = 1 - \frac{\chi}{1 + \chi f_{v1}(\chi)},$$

$$\tilde{S} = \max\left(S + \frac{\tilde{\nu}\, f_{v2}}{\kappa^2 d^2},\ 10^{-10} \right),$$

$$r = \min\left(\frac{\tilde{\nu}}{\tilde{S}\,\kappa^2 d^2},\ 10\right), \quad g = r + c_{w2}(r^6 - r), \quad f_w(g) = g\left(\frac{1 + c_{w3}^6}{g^6 + c_{w3}^6}\right)^{1/6},$$

The final turbulent eddy viscosity, ν_t, which modifies the momentum equations, is then computed as:

$$\nu_t = \tilde{\nu}\, f_{v1}(\chi).$$

Time discretization and operator splitting.

To solve the coupled Navier-Stokes and Spalart-Allmaras system, this is advanced at each time step using a **first-order Lie splitting** scheme (splitting error $O(\Delta t)$). This approach is consistent with the first-order BDF1 time integration used for both solvers.

Given the state $(\mathbf{u}^n, p^n, \tilde{\nu}^n)$ at time t^n, the solution at t^{n+1} is obtained in two steps:

1. the **NS step** solve the momentum and continuity equations with the with effective viscosity frozen at $\tilde{\nu}^n$:

$$\rho\frac{\mathbf{u}^{n+1} - \mathbf{u}^n}{\Delta t} + \rho(\mathbf{u}^* \cdot \nabla)\mathbf{u}^{n+1} - \nabla \cdot \left(2\mu_{\text{eff}}^n\, \nabla^S \mathbf{u}^{n+1}\right) + \nabla p^{n+1} = \mathbf{0},$$

   where $\mu_{\text{eff}}^n = \mu + \nu_t(\tilde{\nu}^n)$ is *explicit* with respect to the SA variable;

2. the **SA step** solve the transport equation using the velocity field $\mathbf{u}^{n+1}$ just computed and a semi-implicit linearization:

$$\frac{\tilde{\nu}^{n+1} - \tilde{\nu}^n}{\Delta t} + \mathbf{u}^{n+1} \cdot \nabla\tilde{\nu}^{n+1} = c_{b1}\tilde{S}^n\tilde{\nu}^{n+1} - c_{w1}f_w^n\frac{\tilde{\nu}^n}{d^2}\tilde{\nu}^{n+1} + \frac{\nu + \tilde{\nu}^n}{\sigma}\Delta\tilde{\nu}^{n+1} + \frac{c_{b2}}{\sigma}|\nabla\tilde{\nu}^n|^2.$$

The linearization strategy is the following:

- the **production** is treated implicitly using a Picard iteration. $\tilde{S}^n$ is frozen and $\tilde{\nu}^{n+1}$ appears linearly in the matrix (stabilizing for $\tilde{S}^n > 0$);
- the **destruction**is linearized around $\tilde{\nu}^n$. The coefficient $c_{w1}f_w^n\tilde{\nu}^n/d^2$ enters as a positive reaction term, ensuring coercivity;
- about the **Diffusion**, the coefficient $(\nu + \tilde{\nu}^n)/\sigma$ is explicit while $\Delta\tilde{\nu}^{n+1}$ is implicit;
- about the **Cross-diffusion**, $c_{b2}/\sigma\,|\nabla\tilde{\nu}^n|^2$ is fully explicit and moved to the right-hand side.

Weak formulation and SUPG stabilization

To handle the advection-dominated nature of the SA equation, the weak form is stabilized using the **Streamline-Upwind Petrov-Galerkin** (SUPG) method.

Multiplying by the SUPG test function $\psi_h = v_h + \tau_{\text{SUPG}}\,\mathbf{u} \cdot \nabla v_h$ and integrating over Ω, the discrete linear system components become:

$$A_{ij}^K = \int_K \left[\psi_i\frac{\phi_j}{\Delta t} + \psi_i(\mathbf{u} \cdot \nabla\phi_j) + D\,\nabla\phi_i \cdot \nabla\phi_j - \psi_i\, c_{b1}\tilde{S}^n\,\phi_j + \psi_i\, c_{w1}f_w^n\frac{\tilde{\nu}^n}{d^2}\,\phi_j\right] dK,$$

$$b_i^K = \int_K \psi_i\left[\frac{\tilde{\nu}^n}{\Delta t} + \frac{c_{b2}}{\sigma}|\nabla\tilde{\nu}^n|^2\right] dK,$$

where $D = (\nu + \tilde{\nu}^n)/\sigma$.

The stabilization parameterτ_{SUPG} is defined locally for each cell K as:

$$\tau_{\text{SUPG}} = \frac{1}{\sqrt{\left(\frac{2}{\Delta t}\right)^2 + \left(\frac{2|\mathbf{u}|}{h_K}\right)^2 + \left(\frac{4D}{h_K^2}\right)^2}},$$

where h_K is the minimum vertex distance of cell K.

Physical bound enforcement.

After each linear solve, a physical bound is enforced on the owned vector before updating the ghosted copy (the ordering `compress(VectorOperation::insert)` → ghosted copy is mandatory):

- negative values ($\tilde{\nu} < 0$) are clamped to 0.0, since turbulent viscosity cannot be negative (SA physical bound);
- `NaN` values ($\tilde{\nu} = \texttt{NaN}$) are reset to $\tilde{\nu}_\infty$ (freestream value) rather than zero, so that the affected cell remains active and self-corrects over subsequent time steps, avoiding a permanently "dead" region in the domain.

Adaptive time-stepping and bisection.

The bisection mechanism of `FluidDynamics` is fully supported.

`try_timestep_with_bisection` is declared `virtual` in the base class, and `FluidDynamicsSA` overrides it by calling `sa_solver.take_snapshot()` before each attempt and `sa_solver.restore_snapshot()` on failure, ensuring that the SA state $\tilde{\nu}^n$ is rolled back consistently at every bisection level.

2.1.3. Boundary Conditions

The implementation handles three distinct boundary condition types (the airfoil wall, the farfield inlet and the outlet boundaries) imposed on the SA variable each with a different physical motivation.

Wall-resolved (low-Re regime).

A homogeneous Dirichlet condition is applied on the airfoil surface: $\partial\Omega_{\text{wall}}$:

$$\tilde{\nu} = 0 \quad \text{on } \partial\Omega_{\text{wall}}. \tag{5}$$

This is the physically correct condition in the viscous sublayer, as the viscosity ν_t must vanish as the distance to the wall $d \to 0$.

Wall functions (high-Re regime).

When the first mesh node is at $y^+ > 5$ (unresolved sublayer), a non-homogeneous Dirichlet condition is derived from the **log-law**:

1. the friction velocity is estimated via Schlichting's flat-plate formula:
$$C_f = \frac{0.026}{Re^{0.143}} \qquad \text{and} \qquad u_\tau = U_\infty \sqrt{C_f/2}$$
2. the wall-normal distance of the first node is converted to wall units:
$$y^+ = \frac{d_{\text{node}}\, u_\tau}{\nu}.$$
3. a switch criterion at $y^+_{\text{sw}} = 11.06$ separates the sublayer from the log-layer:
$$\tilde{\nu}_{\text{wall}} = \begin{cases} 0 & \text{if } y^+ < y^+_{\text{sw}}, \\ \text{root of } \nu_t(\tilde{\nu}) = \nu\kappa y^+ & \text{otherwise.} \end{cases}$$
4. the inversion of $\nu_t(\tilde{\nu}) = \tilde{\nu}\, f_{v1}(\tilde{\nu}/\nu)$ is performed with a **fixed-point iteration**:
$$\tilde{\nu}^{(k+1)} = \frac{\nu\kappa y^+}{f_{v1}\left(\tilde{\nu}^{(k)}/\nu\right)}, \quad k = 0, \ldots, 4,$$
which converges in 5 iterations due to the monotonicity of f_{v1}.

Inlet and farfield.

A Dirichlet condition with freestream value is imposed:

$$\tilde{\nu} = \tilde{\nu}_\infty \in [3\nu,\, 5\nu] \quad \text{on } \partial\Omega_{\text{inlet}} \cup \partial\Omega_{\text{farfield}}. \tag{6}$$

Outlet.

No condition is explicitly imposed: the natural Neumann condition $\partial_n \tilde{\nu} = 0$ arises automatically from the boundary term produced by integration by parts of the diffusion operator in the weak form.

2.1.4. Aerodynamic Coefficients (Residual-Based Approach)

Lift and drag coefficients are computed at the end of each time step via a **residual-based boundary force method**, using the `FluidDynamics::compute_force(tag)` method available in `LifeX`, which integrates the full stress tensor over the boundary face with the prescribed tag (airfoil wall):

$$\mathbf{F} = \int_{\partial\Omega_{\text{wall}}} \boldsymbol{\sigma} \cdot \mathbf{n}\, d\Gamma, \qquad \boldsymbol{\sigma} = -p\mathbf{I} + 2\mu_{\text{eff}}\, \nabla^S \mathbf{u}. \tag{7}$$

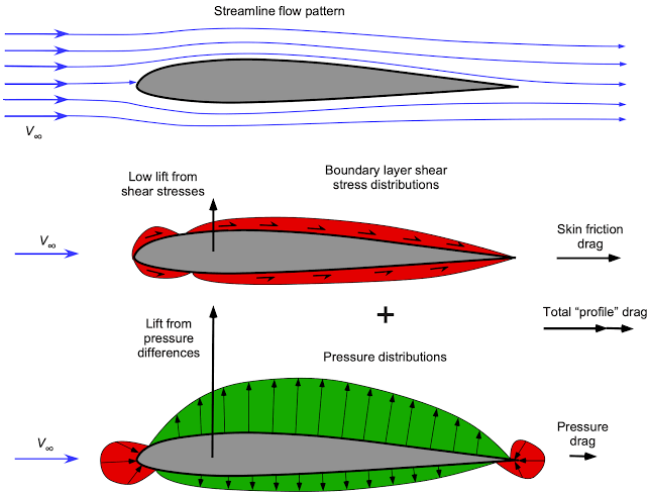


Figure 1: Aerodynamic forces on an airfoil. Source: Eagle Pubs

As illustrated in Figure 1, the Cartesian force components (F_x, F_y) are projected into the aerodynamic reference frame using the angle of attack α:

$$D = F_x \cos\alpha + F_y \sin\alpha, \qquad L = -F_x \sin\alpha + F_y \cos\alpha. \tag{8}$$

The forces are normalized by the dynamic pressure q_∞ and the reference surface S_{ref} (which corresponds to the chord length in 2D simulations):

$$q_\infty = \tfrac{1}{2}\rho U_\infty^2, \qquad C_L = \frac{L}{q_\infty S_{\text{ref}}}, \qquad C_D = \frac{D}{q_\infty S_{\text{ref}}}. \tag{9}$$

Implementation details

This procedure is encapsuled in a header-only utility class `AerodynamicCoefficientsEvaluator` that exposes a single method:

```
compute_coefficients(tag, density, U_inf, S_ref, alpha)
```

Listing 1: Method from `AerodynamicCoefficientsEvaluator`.

It is registered as a callback via `FluidDynamics::set_end_timestep_callback`, so it executes automatically after each successful NS time step. Results are then appended to a CSV file (`force.csv`) with columns `time`, `F_lift`, `F_drag`, `C_L`, `C_D`.

2.1.5. Configuration

The turbulence model is selected via the parameter `"Turbulence model": "Spalart-Allmaras"` in the JSON configuration file.

For backward compatibility, `FluidDynamicsSA` also accepts the legacy value `"None"`, overriding it internally with `Turbulence::SpalartAllmaras` after `parse_parameters`.

The `enum class Turbulence` in `fluid_dynamics.hpp` was extended accordingly:

```
enum class Turbulence { None, VMS_LES, SpalartAllmaras };
```

Listing 2: Extension of the class `Turbolence`.

2.2. Computational framework and execution workflow

This section details the computational environment and the end-to-end workflow established for the simulation.

2.2.1. HPC platform and software stack

All simulations were executed on **Leonardo** at CINECA (Bologna), targeting the `dcgp_usr_prod` partition (Intel Sapphire Rapids nodes, 112 cores per node) under the EuroHPC allocation `EUHPC_D35_025`, with job submission managed by the SLURM scheduler.

The simulation framework is `LifeX` 2.1, introduced in Section 2.1, compiled against `deal.II`9.7.1 with **Trilinos** as the linear-algebra backend, MPICH 4.0 for distributed-memory parallelism, and HDF5 1.14.5 for field output in XDMF format. Meshes are generated externally with Gmsh and read by `LifeX` in MSH 2.2 format.

The full software stack is delivered to the compute nodes through an Apptainer image (`lifex_env.sif`) maintained by the `LifeX` developers, which removes the need for any host-level `module load` of HPC libraries; the build procedure and the rationale behind this choice are detailed in Section 2.2.2.

The resources allocated for each run of the simulation campaign are summarized in Table 2.

Component	**Specification**
Cluster	Leonardo @ CINECA (EuroHPC)
Partition	`dcgp_usr_prod`
CPU	Intel Sapphire Rapids
Cores per node	112
MPI ranks per simulation	8
Wall-time per run	up to 24 h
Software stack	LifeX 2.1 / deal.II 9.7.1 / Trilinos / MPICH 4.0 / HDF5 1.14.5
Container runtime	Apptainer 1.4.5 (image `lifex_env.sif`)

Table 2: Computational resources for the simulation campaign on Leonardo.

2.2.2. Containerization and build process

The compute nodes of Leonardo do not expose a pre-configured HPC software stack through the `module` command, which only loads base system libraries. A native build of `LifeX` on the cluster would therefore require manually compiling `deal.II` 9.7 together with all its required backends[1], an operation that is both fragile and not portable across nodes. The entire toolchain was therefore packaged into an Apptainer image and executed through the container runtime available on the compute nodes (Apptainer 1.4.5), so that the build environment is identical to the execution environment.

[1] `Trilinos` (used by `LinearAlgebraTrilinos` and the `sacado_dfad` AD operators), PETSc, p4est, VTK, Boost, and HDF5

Identifying an image that simultaneously satisfies all `LifeX` dependencies required iterating over several candidates, summarized in Table 3, encountering a series of incompatibilities:

- the Spack stack pre-installed on Leonardo provides `dealii@9.6.2`, but only with the `~trilinos` variant, which is incompatible with `LifeX` 2.1;
- the official `dealii/dealii` Docker images expose `deal.II` 9.7 with all required backends, but lack the `boost_filesystemConfig.cmake` component required by the `LifeX` CMake configuration;
- an earlier MOX `mk` container shipping `deal.II` 9.5.1 fails to compile `LifeX` 2.1 due to API changes in `get_mpi_communicator`, `CellStatus`, and `AffineConstraints` that occurred upstream between 9.5 and 9.7

The image finally adopted, `mk-dealiimaster_latest.sif`, is the MOX MK container with the `deal.II`module tracking the upstream master branch (`deal.II` 9.7), together with matching `vtk`, `boost`, `trilinos`, `petsc`, `p4est`, and `hdf5` modules, activated inside the container by sourcing `/u/sw/etc/profile` and loading the relevant modules.

Stack	Components	Outcome
Spack `dealii@9.6.2` (Leonardo)	deal.II `~trilinos`, PETSc, MPICH	Missing Trilinos/Sacado required by LifeX 2.1
`dealii/dealii` Docker images (`v9.7.1-jammy`, `master-jammy`, `v9.7.1-noble`)	deal.II 9.7.1 with Trilinos, PETSc, p4est, VTK, HDF5	`boost_filesystemConfig.cmake` missing in the image
MOX `mk` container (deal.II 9.5.1)	deal.II 9.5.1, Boost 1.76, VTK, HDF5	API mismatch: LifeX 2.1 requires deal.II $\geq$ 9.7
`mk-dealiimaster_latest.sif`	MK toolchain with deal.II master ($\geq$ 9.7), Boost, VTK, Trilinos, PETSc, p4est, HDF5	**Adopted** for the simulation campaign

Table 3: Container strategies evaluated for building LifeX on Leonardo.

The CMake configuration is invoked through `apptainer exec -cleanenv -bind`, which strips host environment contamination and mounts the workspace inside the container. Because the simulation campaign is purely two-dimensional, the build is configured with `-DLIFEX_DIM=2` and restricted to the two example targets actually used, `example_fluid_dynamics_airfoil` and `example_fluid_dynamics_external_flow`. A global `make all` cannot be used in 2D because some files in the test suite (e.g. `tests/multidomain_laplace/laplace.cpp`) rely on overloads of `GridTools::rotate` that exist only for three-dimensional triangulations, and the resulting compilation error stops the whole build.
The complete build script is reported in Listing 3.

```
#!/bin/bash
#SBATCH -A EUHPC_D35_025
#SBATCH -p dcgp_usr_prod
#SBATCH --time=00:24:00
#SBATCH -N 1
#SBATCH --ntasks-per-node=112
#SBATCH --job-name=lifex_build

set -euo pipefail

WORK_DIR="$HOME/Ice_acceleration_model_on_airplane_wing"
SIF="${WORK_DIR}/containers/mk-dealiimaster_latest.sif"
SRC_SUBDIR="external/lifex-public"
BUILD_SUBDIR="external/lifex-public/build_2d"

rm -rf "${WORK_DIR}/${BUILD_SUBDIR}"

singularity exec --cleanenv \
  --bind "${WORK_DIR}:/work" \
  --bind /dev/shm:/dev/shm \
  "${SIF}" bash -c "
    source /u/sw/etc/profile
```

```
    module load gcc-glibc dealii vtk

    cmake -B /work/${BUILD_SUBDIR} -S /work/${SRC_SUBDIR} \
      -DCMAKE_BUILD_TYPE=Release \
      -DLIFEX_DIM=2 \
      -DDEAL_II_DIR=\${mkDealiiPrefix} \
      -DLin_ALG=Trilinos

    cmake --build /work/${BUILD_SUBDIR} \
      --target example_fluid_dynamics_airfoil \
               example_fluid_dynamics_external_flow \
      -j 112
  "
```

Listing 3: SLURM build script for LifeX on Leonardo (`job_build_leonardo.sh`). The Apptainer image is invoked through the legacy `singularity` command alias.

2.2.3. Mesh generation and pre-processing pipeline

The computational meshes are generated with Gmsh from parametric `.geo` scripts, driven by a set of Python generators (`generate_geo*.py`, `naca_generator_v*.py`) that expose the airfoil profile (NACA 0006 through NACA 0018), the domain extent, and the boundary-layer structure as input parameters.

The resulting mesh is hybrid: a structured layer of quadrilaterals wraps the airfoil surface to resolve the boundary layer, while the outer region is filled with unstructured triangles. Boundary entities follow a fixed tagging convention (`1` = inlet, `2` = outlet, `3` = airfoil wall) matching the tags expected by the `LifeX` configuration (Section 2.1), and the mesh is exported in the MSH 2.2 format read natively by `deal.II`.

In accordance with the wall-resolved turbulence modeling approach described in Section 2.1.3, the near-wall region is carefully refined. The height of the first cell layer is adjusted until the estimated dimensionless wall distance satisfies the criterion $y^+ \lesssim 1$ at a Reynolds number of $\mathrm{Re} = 10^6$. This ensures that the use of a homogeneous Dirichlet condition for $\tilde{\nu}$ is physically valid.

Because a single malformed mesh wastes hours of allocated CPU time, every mesh is screened by a pre-processing pipeline before submission. Four independent Python checks are run in sequence:

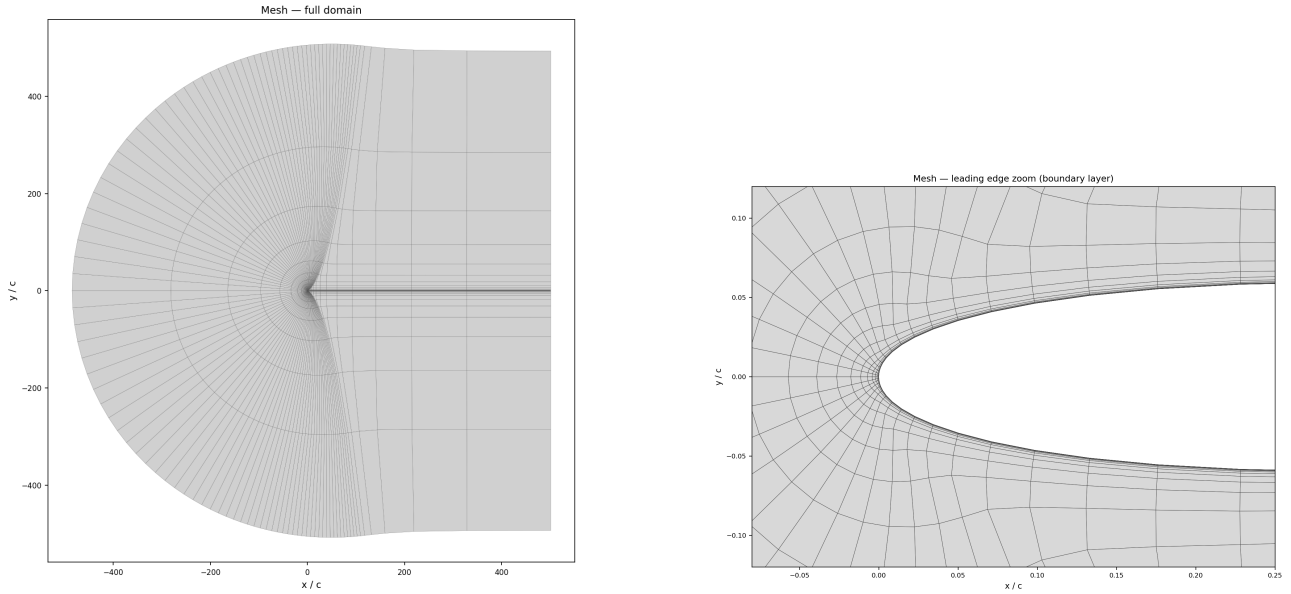
- `mesh_info.py` reports the node and element counts grouped by element type and physical tag and verifies that the inlet, outlet, and airfoil tags are all present;
- `check_extents.py` computes the bounding box of each tagged boundary, confirming that the inlet, outlet and airfoil entities are positioned where the geometry expects them;
- `check_jacobians.py` reads the mesh through `meshio` and computes the signed area of every quadrilateral via the cross product of its diagonals, flagging inverted or non-convex cells that would produce a negative Jacobian;
- `check_yplus.py` estimates the wall-normal y^+ at the first node by measuring the height of the cells adjacent to the airfoil edges and applying a flat-plate skin-friction correlation ($u_\tau/U_\infty \approx 0.036$), consistent with the estimate in Section 2.1.3.

Any mesh that fails one or more of these checks is automatically discarded and regenerated. An example of a mesh that has passed this quality control pipeline, used for the NACA 0012 simulations, is shown in Figure 2.

2.2.4. Simulation execution and post-processing

Each simulation is launched as a single-node SLURM job that runs the `LifeX` executable inside the Apptainer container with `mpirun -n 8`, distributing the mesh across eight MPI ranks on the `dcgp_usr_prod` partition.

The workspace is bind-mounted into the container and the run-time configuration is supplied as a JSON file, following the parameter structure described in Section 2.1: the turbulence model is activated through `"Turbulence`



(a) Overall domain (b) Leading edge detail

Figure 2: Computational mesh for the NACA 0012 airfoil. (a) Overall domain, with a structured quadrilateral boundary layer around the profile and an unstructured triangular outer region. (b) Detail of the wall-resolved boundary layer at the leading edge, refined to give $y^+ \lesssim 1$ at $\mathrm{Re} = 10^6$.

`model": "Spalart-Allmaras"`, and the Reynolds number is set implicitly through the kinematic viscosity ($\nu = 10^{-6}$ with $U_\infty = 1$ and chord $c = 1$, giving $\mathrm{Re} = 10^6$). Before execution, the configuration file is copied into the run directory `output/run_${SLURM_JOB_ID}/` as `config_used.json`, so that every result remains paired with the exact parameters that produced it.

During the run, the solver generates two primary forms of output: a time history of the aerodynamic coefficients, which is appended to `force.csv`, and the full velocity, pressure, and turbulent-viscosity fields, saved as HDF5 files with corresponding XDMF descriptors for visualization in ParaView. The complete SLURM submission script orchestrating this process is reported in Listing 5.

```
#!/bin/bash
#SBATCH -A EUHPC_D35_025
#SBATCH -p dcgp_usr_prod
#SBATCH --time=24:00:00
#SBATCH -N 1
#SBATCH --ntasks-per-node=8
#SBATCH --job-name=lifex_run
#SBATCH -o output/run_%j/solver.log

set -euo pipefail

WORK_DIR="$HOME/Ice_acceleration_model_on_airplane_wing"
SIF="${WORK_DIR}/containers/mk-dealiimaster_latest.sif"
EXEC="/work/external/lifex-public/build_2d/examples/fluid_dynamics_airfoil
      /lifex_example_fluid_dynamics_airfoil"
CONFIG="/work/cases/naca0012/config_new_method.json"
RUN_DIR="${WORK_DIR}/output/run_${SLURM_JOB_ID}"

mkdir -p "${RUN_DIR}"
cp "${WORK_DIR}/cases/naca0012/config_new_method.json" "${RUN_DIR}/
    config_used.json"

singularity exec --cleanenv \
  --bind "${WORK_DIR}:/work" \
```

```
  --bind /dev/shm:/dev/shm \
  "${SIF}" bash -c "
    source /u/sw/etc/profile
    module load gcc-glibc dealii vtk
    export LD_LIBRARY_PATH=/work/external/lifex-public/build_2d/lib:\
       $LD_LIBRARY_PATH

    cd /work/output/run_${SLURM_JOB_ID}
    mpirun -n 8 ${EXEC} ${CONFIG}
  "
```

Listing 4: SLURM run script for a single simulation (`job_run_leonardo.sh`).

The field output is inspected in ParaView, while the coefficient time histories are plotted with the `plot_forces.py` script (matplotlib with the `Agg` backend, so that figures are rendered without a display on the compute nodes).

Figures 3 and 4b show representative output for the NACA 0012 at $\alpha = 4°$ and $\mathrm{Re} = 10^6$. The pressure coefficient field (Figure 3) exhibits the suction peak on the upper surface near the leading edge that generates the net lift; the velocity field (Figure 4a) shows the flow accelerating over the suction side and the thin viscous wake downstream of the trailing edge; the spanwise vorticity (Figure 4b) isolates the shed boundary layers in the wake.

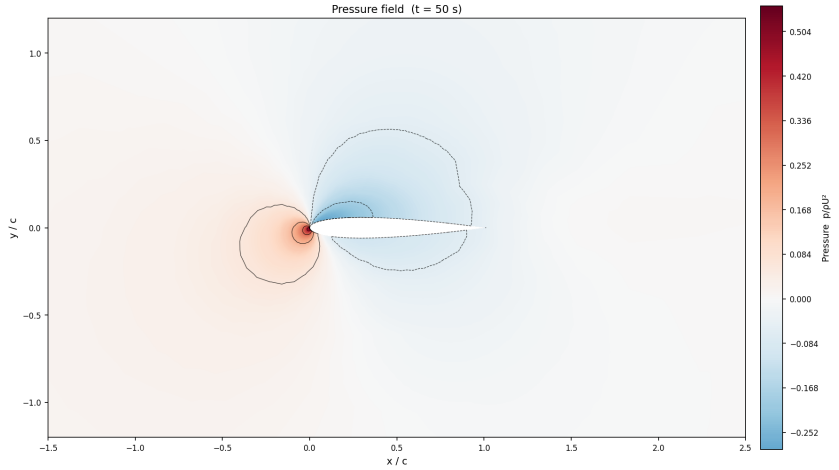
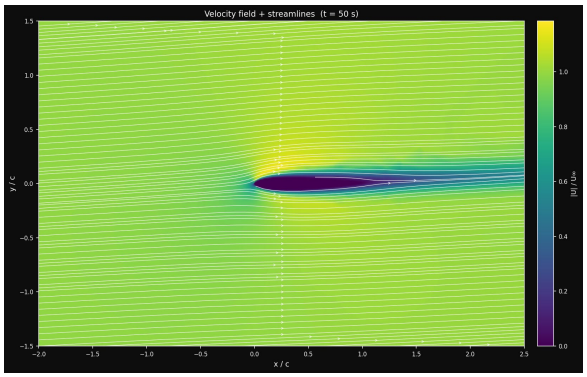
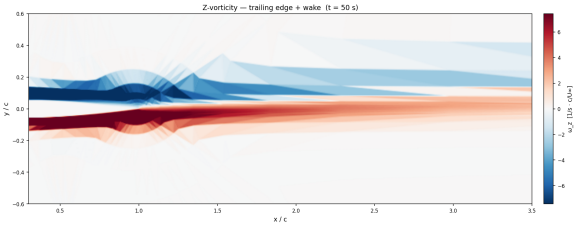


Figure 3: Pressure coefficient field around the NACA 0012 at $\alpha = 4°$, $\mathrm{Re} = 10^6$. The suction peak on the upper surface near the leading edge and the stagnation region on the lower side generate the net lift.



(a) Velocity magnitude with superimposed streamlines.



(b) Spanwise vorticity in the wake.

Figure 4: Flow field around the NACA 0012 at $\alpha = 4°$, $\mathrm{Re} = 10^6$. (a) The flow accelerates over the suction side and produces a thin viscous wake downstream of the trailing edge. (b) The spanwise vorticity isolates the shed boundary layers.

2.2.5. Dataset generation for the surrogate model

The simulation is not an end in itself: its role is to build the training set for the convolutional surrogate model presented in Section 3.

To build a comprehensive dataset, the parameter space was systematically explored. For each airfoil profile in a family of symmetric NACA sections (NACA 0006 to NACA 0018), the angle of attack was swept over the range $\alpha \in \{-14°, -12°, \ldots, +14°\}$ using templated configuration files (`config_isa_aoa_*.json`).

For each configuration, the pipeline produces a single label: the converged lift coefficient, C_L. These labels populate the `Y_cl` array of the final dataset, which is stored in the compressed NumPy format (`.npz`). The corresponding inputs are the Signed Distance Function representation of each geometry and the scalar pair of physical parameters (Re, α).

Computing the lift labels with the turbulence-resolved solver, rather than with a low-fidelity panel method, is what allows the surrogate to be trained against high-fidelity data in the high-incidence regime where thin-airfoil theory no longer holds.

The same workflow extends to the iced-airfoil geometries. A modified leading edge produced by an icing model could be created in future workflows and processed through the identical execution and post-processing chain, so that the dataset can later be augmented with iced profiles without any change to the simulation pipeline. The complete workflow is summarized in Figure 5.

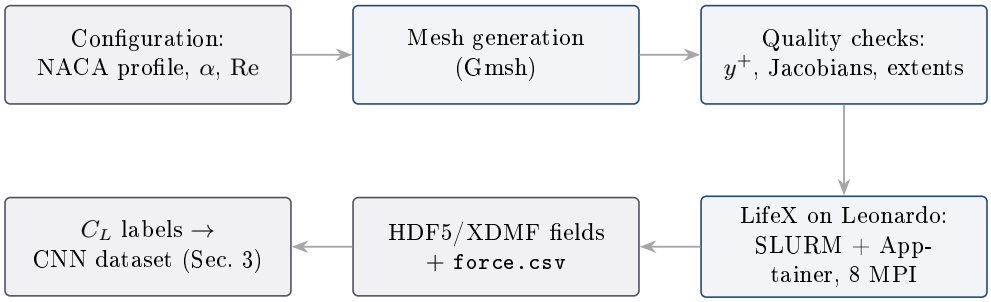


Figure 5: End-to-end simulation pipeline (top row left to right, then bottom row right to left). Mesh generation and quality control run interactively on the login node; the solver runs as a SLURM batch job inside the Apptainer container on the compute nodes; the resulting lift coefficients feed the convolutional surrogate model of Section 3.

3. Convolutional Neural Network

Building upon the dataset established in the preceding Section 2, this Section introduces a surrogate model designed to approximate the results of CFD simulations with a significant reduction in computational time.

Our approach is centered on a **Convolutional Neural Network (CNN)**, a deep learning architecture adept at learning the complex physical relationships that govern aerodynamic performance directly from simulation data.

Specifically, the network is designed to predict the lift coefficient (C_L) as a function of three key inputs:

1. the airfoil shape, represented as a **Signed Distance Function**;
2. the angle of attack (α);
3. the Reynolds number (Re).

By training on the generated simulation results, the CNN effectively learns to emulate the fluid dynamics solver, enabling near-instantaneous predictions.

The following sections will provide a comprehensive overview of this CNN model, from its underlying mathematical principles to its custom implementation designed for high-performance computing.

3.1. Mathematical Background

This section details the mathematical foundations of our surrogate modeling approach.
We first introduce the **Signed Distance Function (SDF)**, which transforms the airfoil's complex geometry into a structured grid suitable for a neural network. We then provide a comprehensive breakdown of the **Convolutional Neural Network architecture**, explaining the mathematical operations within each layer and the physics-informed training algorithm used to learn the mapping from geometry and flow conditions to the aerodynamic lift coefficient.

3.1.1. Geometric Representation with Signed Distance Function

To construct a deep learning model capable of predicting aerodynamic coefficients, we need a robust method for representing the geometry of the airfoil $\Omega \subset \mathbb{R}^2$ and its boundary $\partial\Omega$ in a format suitable for Convolutional Neural Networks. Traditional boundary representations, such as coordinates of boundary points, are challenging to process with standard convolutional operators because they lack fixed spatial grids. To address this limitation, we represent the airfoil geometry using a **Signed Distance Function**, obtaining for each airfoil the result shown in Figure 6.

Figure 6: The Signed Distance Field representation of a NACA 0012 airfoil, discretized on a uniform grid. Negative values (blue) correspond to the airfoil interior, positive values (red) to the exterior while the boundary is represented by the zero-level contour (black line).

The SDF $\Phi : \mathbb{R}^2 \to \mathbb{R}$ maps any point $P = (x, z)$ in the spatial domain to its signed distance from the boundary $\partial\Omega$:

$$\Phi(P) = \begin{cases} -d(P, \partial\Omega) & \text{if } P \in \Omega \\ d(P, \partial\Omega) & \text{if } P \notin \Omega \end{cases} \tag{10}$$

where $d(P, \partial\Omega)$ represents the Euclidean distance from the point P to the boundary $\partial\Omega$:

$$d(P, \partial\Omega) = \inf_{Q \in \partial\Omega} \|P - Q\|_2 \tag{11}$$

Using this definition, the boundary of the airfoil is implicitly defined as the zero-level set of the function:

$$\partial\Omega = \{P \in \mathbb{R}^2 \mid \Phi(P) = 0\} \tag{12}$$

while the interior corresponds to negative values ($\Phi(P) < 0$) and the exterior to positive values ($\Phi(P) > 0$).

In our numerical implementation, the boundary $\partial\Omega$ is defined as a closed polygon composed of an ordered set of n vertices $\{V_0, V_1, \ldots, V_{n-1}\}$.
To guarantee scale invariance, the coordinates of the airfoil profile are first normalized by the chord length c, defined as the Euclidean distance between the leading edge point $(x_{\min}, z|_{x_{\min}})$ and the trailing edge point $(x_{\max}, z|_{x_{\max}})$:

$$c = \sqrt{(x_{\max} - x_{\min})^2 + (z_{\text{at } x_{\max}} - z_{\text{at } x_{\min}})^2} \tag{13}$$

where $x_{\max}$, $x_{\min}$, $z_{\max}$, and $z_{\min}$ are the extreme coordinates computed over the boundary vertices. Each vertex is then scaled by dividing its coordinates by c:

$$V_i = \left(\frac{x_{V_i}}{c}, \frac{z_{V_i}}{c}\right) \tag{14}$$

The boundary $\partial\Omega$ is then represented as the union of n linear segments $S_i = [V_i, V_{i+1}]$ (with $V_n = V_0$). The minimum distance from a grid point P to the boundary $\partial\Omega$ is computed by finding the minimum distance to each individual segment S_i:

$$d(P, \partial\Omega) = \min_{i=0,\dots,n-1} d(P, S_i) \tag{15}$$

For a given segment S_i defined by the endpoints $A = V_i$ and $B = V_{i+1}$, any point on the infinite line passing through A and B can be parameterized as $P(t) = A + t(B - A)$ for $t \in \mathbb{R}$. The orthogonal projection of the point P onto this line is obtained by minimizing the distance $\|P - P(t)\|^2$, which yields the projection parameter:

$$t^* = \frac{(P - A) \cdot (B - A)}{\|B - A\|_2^2} \tag{16}$$

To restrict the projection to the finite line segment S_i, t^* is clamped to the interval $[0, 1]$:

$$t = \max(0, \min(1, t^*)) \tag{17}$$

The projection point on the segment is then $P_{\text{proj}} = A + t(B - A)$, and the distance to the segment is:

$$d(P, S_i) = \|P - P_{\text{proj}}\|_2 \tag{18}$$

To determine the sign of the distance field, we implement the **Winding Number Algorithm** to check whether a grid point P lies inside or outside the closed boundary. The winding number $wn(P)$ computes the number of times the boundary curve wraps around the point P. For a closed polygon, it is computed by casting a horizontal ray starting from P and pointing in the positive x-direction, and counting the signed crossings of this ray with the polygon edges:

- an **upward crossing** occurs if an edge $S_i = [V_i, V_{i+1}]$ crosses the horizontal ray from below, i.e., $V_{i,z} \leq z_P < V_{i+1,z}$ and P lies to the left of the directed line segment;
- a **downward crossing** occurs if an edge S_i crosses the horizontal ray from above, i.e., $V_{i+1,z} \leq z_P < V_{i,z}$ and P lies to the right of the directed line segment.

The relative position of the point P with respect to the directed segment $[A, B]$ is determined mathematically using the 2D cross product:

$$\text{cross}(P, A, B) = (A_x - x_P)(B_z - z_P) - (B_x - x_P)(A_z - z_P) \tag{19}$$

If $\text{cross}(P, A, B) > 0$, the point P lies to the left of the directed segment, whereas if $\text{cross}(P, A, B) < 0$, it lies to the right. The winding number is incremented by 1 for every upward crossing and decremented by 1 for every downward crossing. A point P is strictly inside the domain Ω if and only if the total winding number is non-zero:

$$P \in \Omega \iff wn(P) \neq 0 \tag{20}$$

Once the signed distance is computed for any point, we project it onto a discrete uniform grid. The spatial domain is discretized into a Cartesian grid of size $N \times N = 150 \times 150$ over the bounding box $[X_{\min}, X_{\max}] \times [Z_{\min}, Z_{\max}]$, with:

$$X_{\min} = -0.15, \quad X_{\max} = 1.01, \quad Z_{\min} = -0.12, \quad Z_{\max} = 0.12 \tag{21}$$

The grid spacing is defined by:

$$\Delta x = \frac{X_{\max} - X_{\min}}{N - 1}, \quad \Delta z = \frac{Z_{\max} - Z_{\min}}{N - 1} \tag{22}$$

For each grid node indexed by (row, col) for $row, col = 0, \dots, N - 1$, the corresponding spatial coordinates are given by:

$$x_{col} = X_{\min} + col \cdot \Delta x, \quad z_{row} = Z_{\max} - row \cdot \Delta z \tag{23}$$

This discretized representation yields a 2D matrix of shape 150×150, where each entry contains the signed distance value. This matrix provides a spatially structured representation of the geometry that preserves boundary details, making it perfectly suited as an input to the convolutional branch of the neural network.

3.1.2. Convolutional Neural Networks for Aerodynamic Prediction

The architecture of our **Convolutional Neural Network** is designed as a multi-input model to effectively process both the spatial geometric data and the scalar physical parameters. A high-level schematic of this design, inspired by the model presented by Cuozzo et al. (2026), is depicted in Figure 7.

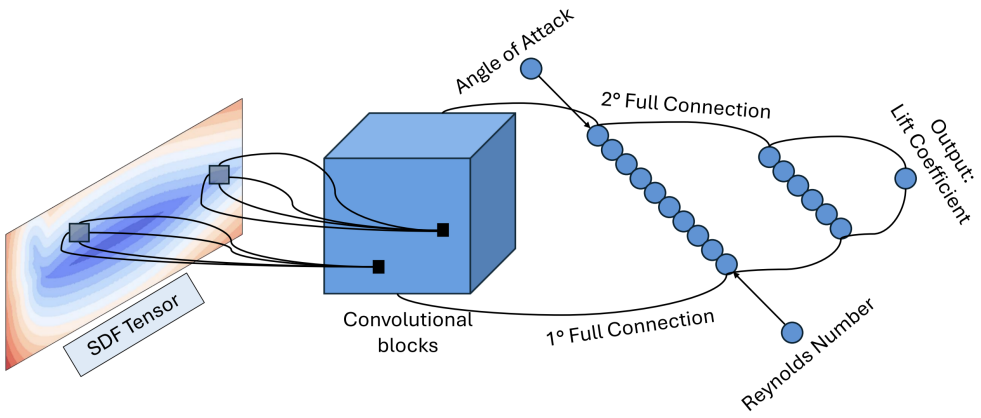


Figure 7: Architecture of the Convolutional Neural Network used for aerodynamic prediction.

The network is logically divided into three main components:

1. a convolutional feature extractor, responsible for interpreting the airfoil geometry. It accepts the 2D SDF matrix as input and processes it through a series of `Conv2DLayer`s, each followed by a `LeakyReLULayer` activation. This stack of layers hierarchically extracts relevant spatial features, such as leading-edge curvature and surface details. The resulting multi-channel feature maps are then transformed into a one-dimensional vector by a `FlattenLayer`, producing a learned representation of the airfoil's shape;

2. a feature fusion stage, in which, the flattened geometric feature vector from the convolutional branch is concatenated with the scalar physical parameters (the angle of attack and the Reynolds number) tensor using a `ConcatenateLayer`. This creates a single feature vector that contains comprehensive information about both the airfoil's shape and the current flow conditions;

3. finally the unified vector is passed to the regression head, which consists of a sequence of `DenseLayer`s. This fully-connected network maps the combined high-level features to a single scalar output, which represents the predicted lift coefficient, C_L.

Convolutional Layer

The **Convolutional Layer** (`Conv2DLayer`) is an operator specifically designed to process grid-like data such as the 2D SDF. Its primary function is to detect local spatial patterns within the input by applying a set of learnable filters. The difference with the dense layer, which treats all input pixels independently, is that a convolutional layer preserves the spatial relationship between pixels by operating on localized neighborhoods.

At its core, the operation is a discrete 2D convolution, which can be understood as a *sliding window dot product*. A small matrix of learnable weights, known as the kernel (K), slides over the input feature map (I). At each position, an element-wise multiplication between the kernel and the overlapping region of the input is performed, and the results are summed up to produce a single pixel in the output feature map (O).

For a single-channel input I of size $H \times W$ and a kernel of size $F \times F$, the value of the output O at position (i, j) is given by:

$$O(i,j) = \sum_{m=0}^{F-1} \sum_{n=0}^{F-1} I(i-m, j-n) \cdot K(m,n) + b \tag{24}$$

where b is a learnable scalar bias term applied to the entire output feature map.

The key principle here is parameter sharing: the *same* kernel K and bias b are used across all positions of the input, allowing the network to detect the same feature regardless of where it appears in the SDF[2].

In deep networks, layers typically process multi-channel inputs. If the input feature map has C_{in} channels[3], the kernel must be three-dimensional, with a size of $F \times F \times C_{in}$. The convolution is then performed across all channels simultaneously. The results from each channel are then summed together to produce a single value for the 2D output map.

To generate an output with C_{out} feature maps, we simply apply C_{out} independent kernels. Each kernel is responsible for learning a different spatial feature, allowing the layer to build a complete representation of the input geometry.

The behavior of the convolution is controlled by two hyperparameters:

- the stride (S) defines the step size the kernel matrix as it traverses the input grid. While a unit stride ($S = 1$) evaluates adjacent localized neighborhoods sequentially, a stride of $S > 1$ skips spatial coordinates, downsampling the spatial resolution of the resulting feature map;
- padding (P)[4] consists in appending a perimeter of zero-valued elements around the input grid boundaries. This can be used to control the output dimensions and mitigate the loss of information at the edges.

The dimensions of the output feature map ($H_{out} \times W_{out}$) are determined by the input dimensions ($H_{in} \times W_{in}$), the kernel size (F), the stride (S) and the padding (P) using the following formula:

$$W_{out} = \frac{W_{in} - F + 2P}{S} + 1 \quad \text{and} \quad H_{out} = \frac{H_{in} - F + 2P}{S} + 1 \tag{25}$$

From an implementation perspective, this operation is typically realized as a set of nested loops that iterate over each position in the output map, each element of the kernel, and each input channel, performing the required multiply-accumulate operations[5].

Non-Linear Activation (Leaky ReLU)

After each `Conv2DLayer` or `DenseLayer` performs its linear operation, an activation function applies a non-linear transformation element-wise to its output. Without this, a deep neural network, regardless of its number of layers, would behave as a single, large linear model. The introduction of non-linearity is what gives the network its power to approximate arbitrarily complex, non-linear relationships, such as the one between airfoil geometry and lift coefficient.

In this project, a **Leaky Rectified Linear Unit** (Leaky ReLU) is implemented as the `LeakyReLULayer`. This function, shown in Figure 8, is a popular and effective choice in modern deep learning due to its computational efficiency and its ability to mitigate some of the common problems encountered during training.

The Leaky ReLU function is defined mathematically as:

$$\text{LeakyReLU(x)} = \begin{cases} x & \text{if } x > 0 \\ \alpha x & \text{if } x \leq 0 \end{cases} \tag{26}$$

where α is a small, positive constant (a hyperparameter, typically set to a value like 0.01).

The function behaves as a simple identity for all positive input values. For negative input values, instead of mapping them to zero like the standard ReLU function, it allows a small, non-zero gradient to pass through.

The primary advantage of the Leaky ReLU over the standard ReLU is its handling of negative inputs. In a standard ReLU, if a neuron's input is consistently negative, its gradient will always be zero. This means the

[2] In more details, if the kernel learns to recognize a leading edge curve of a profile, it will successfully detect the curve whether it appears at the top of the grid or in a bottom corner.

[3] This is not our case, since in the SDF we have only one input channel ($C_{in} = 1$).

[4] In our implementation, padding is set to 0.

[5] We will explain in a better way highlighting our C++ implementation in Section 3.2.3 regarding our implementation in C++

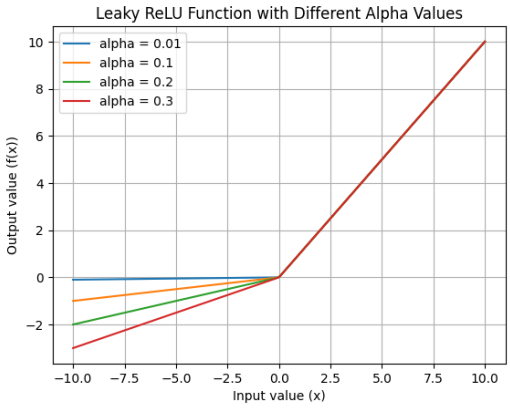


Figure 8: Example of Leaky ReLU function with different α.

neuron's weights will never be updated, and it can effectively "die", ceasing to contribute to the network's learning process. The Leaky ReLU prevents this "dying ReLU" problem by ensuring that a small gradient always exists, even for negative inputs.

From the backpropagation algorithm, the gradient of the Leaky ReLU is simple and computationally inexpensive to calculate:

$$\frac{d}{dx}\text{LeakyReLU(x)} = \begin{cases} 1 & \text{if } x > 0 \\ \alpha & \text{if } x \leq 0 \end{cases} \tag{27}$$

During the backward pass of training, this straightforward derivative is used to scale the incoming gradient from the subsequent layer:

- if the input to the activation function was positive during the forward pass, the gradient passes through unchanged;
- if it was negative, the gradient is scaled down by the factor α.

This simple conditional check makes the backward pass for the `LeakyReLULayer` extremely fast, contributing to the overall efficiency of the training process.

Structural Layers

After the convolutional network has extracted a hierarchy of spatial features, the resulting multi-channel feature maps must be prepared for integration with the scalar parameters and for final processing by the regression head. This preparation is handled by two complementary **structural layers**, the `FlattenLayer` and the `ConcatenateLayer`, that are parameter-free: they do not contain any learnable weights or biases since their sole purpose is to rearrange data tensors.

Flatten Layer

The `FlattenLayer` links the convolutional layers and the fully-connected layers. Convolutional layers output a 4D tensor with a shape of `[Batch Size, Channels, Height, Width]`, preserving spatial information. However, dense layers require their input to be a 2D tensor of shape `[Batch Size, Features]`, where each row is a one-dimensional vector representing a single sample.

The flatten operation reshapes an input tensor T_{in} with dimensions $[B \times C_{out} \times H_{out} \times W_{out}]$ into an output tensor T_{out} with dimensions $[B \times D]$, where the new feature dimension D is the product of the input's channel, height and width dimensions:

$$D = C_{out} \times H_{out} \times W_{out} \tag{28}$$

During the forward pass, this is a straightforward memory rearrangement.
During the backward pass for backpropagation, the `FlattenLayer` performs the inverse operation: it receives an incoming gradient tensor of shape $[B \times D]$ and reshapes it back to the original 4D shape $[B \times C_{out} \times H_{out} \times W_{out}]$. This ensures that the gradients are correctly propagated back to the preceding convolutional layer in the required spatial format.

Concatenate Layer

Once the geometric features have been flattened into a vector, they must be combined with the scalar physical parameters by the `ConcatenateLayer`, that takes multiple input tensors and joins them together along a specified axis[6].

Mathematically, given the flattened geometric feature tensor T_{flat} of shape $[B \times D_1]$ and the scalar input tensor T_{scalar} (containing angle of attack and Reynolds number) of shape $[B \times D_2]$, the Concatenate Layer produces an output tensor T_{concat} of shape $[B \times (D_1 + D_2)]$. This operation creates a unified feature vector that encodes both geometric and physical information for the network to make a prediction.

During backpropagation, the process is reversed. The incoming gradient tensor is split according to the original input sizes:

- the first D_1 elements of the gradient vector are passed back to the `FlattenLayer`'s branch;
- the D_2 elements corresponding to the scalar inputs are also computed, although they terminate at the input layer.

After this fusion step, the resulting unified tensor is ready to be processed by the final regression head of the network.

Dense Layer and final regression

The final computational stage of the network is handled by the **Dense Layer** (`DenseLayer`), known as a fully-connected layer. Its role is to learn the final mapping from the feature space to the target scalar value (the lift coefficient C_L).

A dense layer applies a linear transformation to its input vector. For an input vector $\mathbf{x} \in \mathbb{R}^{D_{in}}$, the layer computes an output vector $\mathbf{y} \in \mathbb{R}^{D_{out}}$ using a learnable weight matrix $W \in \mathbb{R}^{D_{out} \times D_{in}}$ and a learnable bias vector $\mathbf{b} \in \mathbb{R}^{D_{out}}$. The operation is defined as:

$$\mathbf{y} = W\mathbf{x} + \mathbf{b} \tag{29}$$

This operation performs a weighted sum of all input features to compute each output feature. Every neuron in the input layer is connected to every neuron in the output layer, allowing the network to model complex interactions between all elements of the input feature vector.

This layer is used in the regression head, which consists of a sequence of `DenseLayer`s, each followed by a `LeakyReLULayer` activation:

- the initial dense layers take concatenated feature vector as input. Their purpose is to find and amplify the non-linear correlations between the learned geometric features from the SDF and the raw physical parameters (α and Re);
- the last layer is configured to have a single output neuron ($D_{out} = 1$) and does not have a subsequent activation function. This single neuron's output directly corresponds to the network's prediction for the lift coefficient (C_L).

During the backward pass, the dense layer must compute three distinct gradients based on the incoming gradient from the subsequent layer $\frac{\partial L}{\partial \mathbf{y}}$, where L is the loss:

- the gradient with respect to the weights (W) is calculated as the outer product of the incoming gradient and the layer's input vector from the forward pass:

$$\frac{\partial L}{\partial W} = \frac{\partial L}{\partial \mathbf{y}} \cdot \underbrace{\mathbf{x}^{\mathbf{T}}}_{\frac{\partial \mathbf{y}}{\partial \mathbf{W}}} \tag{30}$$

- the gradient with respect to the biases (b) is equal to the incoming gradient:

$$\frac{\partial L}{\partial \mathbf{b}} = \frac{\partial L}{\partial \mathbf{y}} \tag{31}$$

[6] In our case, this is the feature axis (axis 1)

- the gradient with respect to the input (x) must be passed back to the preceding layer. It is computed by multiplying the transposed weight matrix by the incoming gradient:

$$\frac{\partial L}{\partial \mathbf{x}} = W^T \cdot \frac{\partial L}{\partial \mathbf{y}} \tag{32}$$

These calculations are the fundamental operations that enable the network's regression head to learn from the training data.

Network Training

The process of training the neural network involves iteratively adjusting its parameters to minimize a loss function, which quantifies the discrepancy between the model's predictions and the desired outcomes. Our approach embeds physical knowledge directly into the training process, making our model a type of **Physics-Informed Neural Network (PINN)**. This is achieved through a composite loss function that balances empirical data accuracy with a theoretical physical constraint.

The total loss, L, is a weighted sum of data-driven term and a physics-informed term:

$$L = L_{\text{data}} + \lambda L_{\text{physics}} \tag{33}$$

where λ is a hyperparameter that controls the strength of the physical regularization.

The first component is the standard **Mean Squared Error** (MSE), which measures the difference between the network's predicted lift coefficients (P_i) and the ground-truth values from the CFD simulations (T_i):

$$L_{\text{data}} = \frac{1}{N} \sum_{i=1}^{N} (P_i - T_i)^2 \tag{34}$$

This term ensures the network learns to faithfully replicate the high-fidelity simulation data generated for this work.

The second component acts as a physical constraint, guiding the network's predictions to be consistent with established aerodynamic theory. This term is based on **Thin-Airfoil Theory**, which provides an analytical approximation for the lift coefficient at small angles of attack. The general form of this relationship is $C_L = 2\pi(\alpha - \alpha_{L=0})$, where $\alpha_{L=0}$ is the zero-lift angle of attack.

For symmetric airfoils, such as those in our dataset, the zero-lift angle of attack is, by definition, $\alpha_{L=0} = 0$. This simplifies the theoretical relationship to the linear form:

$$C_L \approx 2\pi\alpha \tag{35}$$

The physical loss in the network leverages this simplified form, penalizing deviations from this theoretical relationship:

$$L_{\text{physics}} = \frac{1}{N} \sum_{i=1}^{N} M_i \cdot (P_i - 2\pi\alpha_i)^2 \tag{36}$$

where M_i is a mask function, introduced because Thin-Airfoil Theory is only valid in the linear flight regime. Therefore, the mask is defined as:

$$M_i = \begin{cases} 1 & \text{if } |\alpha_i| \leq 10^\circ \\ 0 & \text{if } |\alpha_i| > 10^\circ \end{cases} \tag{37}$$

This ensures the physical constraint is only active at low angles of attack.
In the non-linear, high-angle-of-attack regime, the mask deactivates this term ($M_i = 0$), forcing the network to rely entirely on the empirical CFD data, which accurately captures the complex flow separation phenomena.

By explicitly building in the assumption of symmetry, this physics-informed regularizer provides a robust theoretical anchor for the network's predictions without being applied in regimes where its underlying assumptions would be violated.

Optimization with Adam

Once the backpropagation algorithm has computed the gradient of the loss function with respect to every trainable parameter in the network ($\nabla L(\theta)$, where θ represents all weights and biases), the final step in the learning process is to update these parameters. This is done using the **Adam (Adaptive Moment Estimation)** optimizer, which guides the parameters toward a minimum in the loss landscape.

Adam combines the advantages of the **RMSprop** and **Momentum** optimization methods. It maintains an exponentially decaying moving average of both the past gradients (first moment, the momentum term) and the past squared gradients (second moment, the adaptive learning rate term).

Let $g_t = \nabla L(\theta_{t-1})$ be the gradient at timestep t. The optimizer updates two moving averages:

- first moment (momentum):

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \tag{38}$$

- second moment (adaptive learning rate):

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \tag{39}$$

where β_1 and β_2 are hyperparameters (typically close to 1, e.g., 0.9 and 0.999 respectively).

These raw moment estimates are then bias-corrected to account for their initialization at zero:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t} \quad \text{and} \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \tag{40}$$

Finally, the parameters are updated using these corrected estimates. The update rule for a parameter θ is:

$$\theta_t = \theta_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon} \tag{41}$$

where η is the learning rate and ε is a small constant (e.g., 10^{-8}) to prevent division by zero.

The key intuition is that the update step is scaled by both a running average of the gradients (providing momentum to navigate flat regions) and an adaptive, per-parameter learning rate (scaling down updates for frequently changing parameters and scaling up updates for sparse ones). This adaptive nature makes Adam robust and generally leads to faster convergence compared to standard stochastic gradient descent.

3.2. Code Implementation

We implemented our CNN and SDF generator using an **object-oriented paradigm**. Figure 9 provides a comprehensive **UML class diagram** detailing the structural layout, member attributes and method signatures of the system.

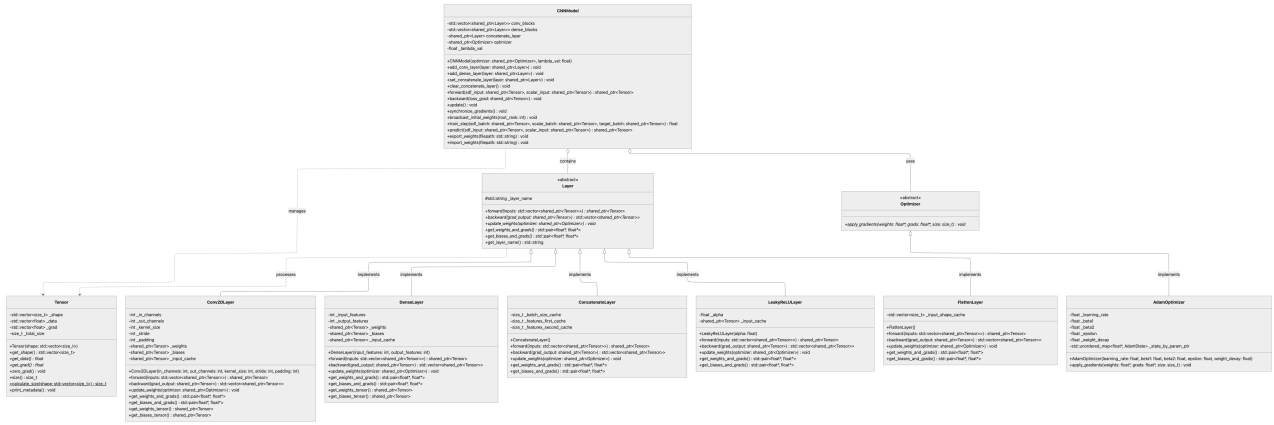


Figure 9: UML class diagram illustrating the object-oriented design of the C++ Convolutional Neural Network.

In the following sections, we decide to highlight the most interesting implementations of these components, detailing key choices and memory optimization layouts, as well as providing an explanation of the core functions implemented.

3.2.1. Geometric Preprocessing: SDF Generator

The generation of the Signed Distance Function is encapsulated within the `SDFGenerator` class, which maps airfoil geometries to structured 2D tensors suitable for convolutional processing.

To maximize CPU cache efficiency and minimize cache misses during spatial calculations, boundary vertices are stored in a `std::vector<Point>`, where `Point` is a simple structure holding `x` and `z` coordinates. This choice creates a single contiguous array for the boundary data.

Upon initialization, the constructor ensures scale invariance (as detailed in Section 3.1.1) by calling two private helper methods: `computeCord()` calculates the chord length c using `std::hypot` to prevent numerical overflow[7] and `normalizePoint()` scales all vertex coordinates accordingly[8].

With a normalized boundary in place, the `SDFGenerator` computes the signed distance for each point on the target grid. This is achieved through a set of optimized geometric kernel functions:

- `computeDistance(...) const` calculates the minimum Euclidean distance from a grid point to a boundary segment (as it is shown in Equations (16) and (17)). It includes a safety check for degenerate edges (where the two end-point of the segment coincide) to prevent division-by-zero errors;
- `getMinDistance(...) const` iterates through all boundary edges of the airfoil, calling for each one `computeDistance` and returning the absolute minimum distance found. Additionally, we used `static_cast<int>(airfoil.boundary.size())` to safely convert the unsigned vector size to a signed integer, preventing signed/unsigned comparison warnings;
- finally, the `windingAlgorithm(...) const` implements the Winding Number Algorithm previously explained (Section 3.1.1) to verify whether a grid point p lies inside or outside the closed boundary. This is computed by analyzing edge crossings and using the 2D cross product (Equation (19))[9], returning `true` if the point is inside the domain.

Once these values are computed for every grid point, the `mappingToMatrix` function converts the resulting 1D flattened SDF data into the required 150×150 2D matrix format and writes it to a file, utilizing the **RAII (Resource Acquisition Is Initialization)** paradigm via `std::ofstream outfile(filename)` that automatically closes the file descriptor when `outfile` goes out of scope, in order to prevent system resource leaks in case of exceptions.

The most computationally intensive part of the process is executing these kernels for each one of the $150 \times 150 = 22,500$ points on the grid. This task is a classic example of an *embarrassingly* parallel problem and it is implemented in the `generateTensorMPI` method, the details of which are provided in Section 3.3.

3.2.2. Core abstractions and memory management

To achieve high computational throughput while preserving a flexible design, the system introduces two core abstractions: a unified multi-dimensional tensor representation and a polymorphic layer interface.

Tensor

The foundational data structure of our CNN implementation is the `Tensor` class, which manages both forward activation data (`_data`) and backward gradients (`_grad`) as multi-dimensional arrays of single-precision floating-point numbers.

[7] This function scales intermediate values before performing the sum of squares, avoiding the premature overflow or underflow that can occur when computing the norm directly as $\sqrt{\Delta x^2 + \Delta z^2}$

[8] This normalization step will be valuable for future implementations aimed at studying the effect of ice accumulation on the lift coefficient.

[9] Implemented through the `crossProduct(Point p, Point a, Point b) const` with each point passed by value, in order improve performance for small `structs` by allowing them to be passed directly in CPU registers, avoiding memory indirection.

First of all, the constructor `Tensor(std::vector<size_t> shape)` initializes the shape descriptor and sets the `_total_size` member through a member **initializer list**. It pre-allocates and zero-initializes the storage vectors using `std::vector::resize`, that guarantees contiguous memory allocation. Direct access to this underlying memory is provided via the `get_data()` and `get_grad()` methods, which return raw pointers (`float*`) using the `.data()` member function.

Beyond raw pointer access, the class provides safe, native array-like indexing for flattened 1D elements through overloaded `operator[]` methods, offering both mutable and read-only (`const`-qualified) references. Utility functions are also optimized: gradient values are efficiently reset before each backward pass using the `std::fill` algorithm within the `zero_grad()` method.

Furthermore, the total element count is computed by `calculate_size()`, which is declared as a `static` method[10] and accepts a constant reference to avoid the overhead of copying the shape vector.

Finally, the class strictly adheres to `const` correctness, ensuring that utility methods like `print_metadata()` are executed safely without modifying the object's internal state.

Layer

The abstract base class `Layer`, defined in `Layer.hpp`, establishes a unified interface for all CNN components by leveraging runtime polymorphism to decouple the high-level network topology from the specific implementations of individual layers.

Runtime polymorphism is achieved through virtual functions and dynamic dispatch. When a class declares virtual methods, the compiler generates a virtual method table (`vtable`) containing pointers to these functions. Each object instance is implicitly equipped with a pointer (`vptr`) to its class's `vtable`. During execution, when a virtual method is called via a base class pointer, the program resolves the call dynamically: it dereferences the object's `vptr` to access the `vtable` and executes the overridden method corresponding to the actual derived type.

This mechanism is fundamental to our design. Rather than relying on conditional logic to handle the different mathematical operations, polymorphism allows the model to store and manage all layers uniformly within a single container (`std::vector<std::shared_ptr<Layer>`). In addition, this design ensures high scalability and the effortless integration of new layer types simply by inheriting from the `Layer` base class.

Following the idea of implementing the polymorphism, we declared a virtual destructor `~Layer() = default`, ensuring that when a derived layer object is destroyed via a base class pointer, the derived class's destructor is executed, preventing resource leaks of dynamically allocated memory.

The core operations of the network are declared as pure virtual functions (indicated by the `= 0` suffix), which must be overridden by every concrete layer subclass. Specifically, we implemented:

- `forward`, that computes forward activations, and `backward`, that performs backpropagation, both accepting and returning `std::shared_ptr` in order to guarantee automatic memory deallocation when the tensors are no longer needed;
- `get_weights_and_grads()` and `get_biases_and_grads()` that returns `std::pair<float*, float*>` packaging the weight/bias arrays and their corresponding gradients into a single return value. Returning non-const pointers allows the optimizer to modify the parameter values directly in place.

Finally, architectural decoupling is achieved through a forward declaration of the `Optimizer` class; this resolves potential circular dependencies while allowing the `update_weights(std::shared_ptr<Optimizer>)` method to delegate parameter updates to external optimization algorithms seamlessly.

3.2.3. Computational and activation layers

The computational layers manage the network's mathematical transformations and contain all trainable parameters, while activation layers introduce the necessary non-linear activation boundaries.

[10] This allows the method to be invoked without a class instance, for example, within a member initializer list.

DenseLayer

The concrete `DenseLayer` class implements the fully-connected linear transformation $\mathbf{y} = W\mathbf{x} + \mathbf{b}$, where W is the weight matrix and $\mathbf{b}$ is the bias vector, as described in details in the Section 3.1.2.

First of all, the constructor `DenseLayer(int input_features, int output_features)` allocates the weight and bias tensors utilizing `std::make_shared` rather than direct initialization via `std::shared_ptr(new Tensor(...))`. While this last one forces two separate heap allocations[11], `std::make_shared` performs a **single contiguous memory allocation** for both the reference-counting control block and the managed `Tensor` object, in order to reduce heap fragmentation and improve CPU cache locality by keeping the control metadata and the tensor data adjacent in memory.

To ensure gradient stability at the start of training, the weights are initialized according to the **Xavier/Glorot scheme**, drawing from a uniform distribution bounded by $\pm\sqrt{6/(N_{\text{in}} + N_{\text{out}})}$ using `std::default_random_engine` and `std::uniform_real_distribution<float>`.

During the `forward` pass, the input tensor is explicitly cached (`_input_cache`) to satisfy the mathematical requirements of backpropagation, as the input vector $\mathbf{x}$ must be retained to compute the weight updates as $\frac{\partial L}{\partial W} = \mathbf{x} \otimes \frac{\partial L}{\partial \mathbf{y}}$.

Moreover, the linear transformation $Y = XW^T + \mathbf{b}$ is evaluated through three nested loops:
- the outermost loop iterates over the batch size b;
- the middle loop iterates over the output features j;
- the innermost loop iterates over the input features i to evaluate the dot product $Y_{b,j} = \sum_i X_{b,i}W_{j,i} + b_j$.

The weight matrix W is stored in a transposed format of shape $(N_{\text{out}}, N_{\text{in}})$. This ensures that as the innermost loop increments its index over the input features, it accesses both the input tensor X and the weight tensor W with a unit stride. This design maximizes spatial locality, reduces cache misses, and enables the compiler to generate efficient SIMD vector instructions.

To conclude, also the `backward` method computes the gradients through three distinct sets of nested loops:
1. the gradient with respect to the inputs is calculated as $dX_{b,i} = \sum_j dY_{b,j}W_{j,i}$. In this loop structure, the weight matrix is traversed column-wise, resulting in non-contiguous memory access;
2. the gradient with respect to the weights is accumulated as $dW_{j,i} = \sum_b dY_{b,j}X_{b,i}$ using loops over j, i, and the batch dimension b. The innermost loop iterates over b, calculating the outer product of the output gradients and cached inputs;
3. the gradient with respect to the biases is accumulated as $dB_j = \sum_b dY_{b,j}$ by summing the output gradients across all batch elements for each output feature.

LeakyReLULayer

The concrete `LeakyReLULayer` class implements homonymous element-wise activation function shown in Equation (26). The negative slope parameter α is configured into the constructor `LeakyReLULayer(float alpha = 0.05f)`.

The interesting thing in this code is the use of the ternary operator:
- in the `forward` method, the element-wise activation is evaluated inside a contiguous loop using raw pointers `out_ptr[i] = (in_ptr[i] > 0.0f) ? in_ptr[i] : _alpha * in_ptr[i]`;
- in the `backward` method, that is used to evaluate the gradient with respect to the input by scaling the output gradient as shown in Equation (27), implemented as `din_ptr[i] = dout_ptr[i] * ((cache_ptr[i] > 0.0f) ? 1.0f : _alpha)`.

Because the derivative of the activation function is conditioned on the sign of the original input activations, caching the forward input tensor inside `_input_cache` is necessary to retrieve the values during the backward pass.

To conclude this class, because the activation layer contains no learnable parameters, the overridden method `update_weights` is implemented as a no-op. Moreover, the parameter accessors `get_weights_and_grads()` and `get_biases_and_grads()` return a pair of type-safe null pointers, `{nullptr, nullptr}`.

[11] One for the object and one for its control block.

Conv2DLayer

The concrete `Conv2DLayer` class extends the abstract `Layer` interface to perform 2D spatial convolutions on four-dimensional tensors, which logically represent **batch samples, channels, height and width**.

Upon instantiation, the constructor initializes the convolutional kernels using a **Gaussian Xavier scheme** to ensure variance stability across deep network layers. The standard deviation for this distribution is computed as $\sigma = \sqrt{2/(F_{\text{in}} + F_{\text{out}})}$, where the fan-in and fan-out dimensions are defined as $F_{\text{in}} = C_{\text{in}} \times K^2$ and $F_{\text{out}} = C_{\text{out}} \times K^2$, with K denoting the kernel size.

During the `forward` pass, the spatial dimensions of the output tensor are explicitly calculated using Equation (25) and the resulting tensor is allocated contiguously via `std::make_shared`. Because the system flattens all multi-dimensional tensors into 1D contiguous vectors for optimal memory layout, the layer utilizes a private inline helper function, `idx4`, to resolve 4D indices to 1D offsets. This enforces a strict row-major mapping:

$$\text{Offset}(a, b, c, d) = [(a \times \text{dim}_b + b) \times \text{dim}_c + c] \times \text{dim}_d + d \tag{42}$$

where a, b, c, d represent the target indices across the batch, channel, and spatial dimensions.

The forward convolution operation is executed through seven-nested loop. The outer four loops iterate over the batch elements n, output channels oc and spatial output coordinates oh and ow, while the inner three loops traverse the input channels ic and kernel dimensions kh and kw. To handle boundaries without incurring the overhead of allocating memory for physical zero-padding, the implementation uses implicit padding. Source coordinates are dynamically evaluated at each step as $ih = oh \times S + kh - P$ and $iw = ow \times S + kw - P$:

- if these fall outside the input boundaries, the multiplication is safely bypassed using a `continue` statement;
- otherwise, the flattened indices are retrieved via `idx4`, and the kernel-input products are accumulated into a running sum initialized with the appropriate channel bias.

Finally, the `backward` method evaluates gradients using two distinct loop structures:

- the first structure computes the parameter gradients: the bias gradient (dB) aggregates the output gradients directly, while the weight gradient (dW) accumulates the product of the output gradients and the cached forward inputs;
- the second structure computes the gradient with respect to the input (dX) by performing a full convolution between the output gradients and the flipped kernel filters. To achieve this, the loop accesses the weights using inverted spatial coordinates:

$$kh_{\text{orig}} = K - 1 - kh, \quad kw_{\text{orig}} = K - 1 - kw \tag{43}$$

3.2.4. Structural Layers

The **structural layers** are parameter-free components designed to rearrange and route data tensors across the branches of the network.

FlattenLayer

The `FlattenLayer` class extends the base `Layer` interface to perform a logical reshaping of the 4D feature maps (generated by the convolutional branch) into a 2D tensor suitable for the subsequent `ConcatenateLayer`.

The `forward` method calculates the flattened size $F = C \times H \times W$ and allocates a 2D output tensor of shape $[B \times F]$ via `std::make_shared`. Since the flatten operation is basically a reshape, the data is copied from the input to the output tensor sequentially in a single loop, preserving the element order across contiguous memory arrays.

The `backward` method receives the 2D gradient tensor `grad_output` (of shape $[B \times F]$), validates the batch size consistency, allocates a 4D gradient tensor matching the cached shape `_input_shape_cache`[12] and performs the inverse sequential copy `grad_in_ptr[i] = grad_out_ptr[i]`, reconstructing the 4D gradient layout for the preceding convolutional layer.

[12] This member attribute of type `std::vector<size_t>` preserves the original spatial dimensions ($C \times H \times W$) to reconstruct and propagate the gradient back.

ConcatenateLayer

The `ConcatenateLayer` class is used to obtain a single tensor that combines the information from the **Convolutional Layer** and the scalar values of the Reynolds number and angle of attack.

Beyond the mathematical formalism about the shape of the matrices described in a detailed way in Section 3.1.2, the merge operations are implemented with nested loops:

- the outer loop iterates over the batch size n, computing the row offsets for the first input (`n * _features_first_cache`), the second input (`n * _features_second_cache`) and the output (`n * (_features_first_cache + _features_second_cache)`);
- two inner loops then copy the data, ensuring row-wise contiguous layout.

During the `backward` method, this operation is inverted. The layer receives a combined gradient tensor of shape $[B \times (D_1 + D_2)]$ and must route the gradients back to their respective origins, reversing the copy logic: for each batch element, it copies the first D_1 components of the output gradient to the first input gradient buffer and the remaining D_2 components to the second input gradient buffer. Both elements are then returned in a standard vector `{grad_first, grad_second}`.

3.2.5. Training pipeline, loss and optimizers

The network's optimization and training pipeline is coordinated by mathematical loss functions and state-tracking optimization solvers.

Loss

Unlike the object-oriented structure of the computational layers, the **physics-informed loss function** and its corresponding gradient are implemented as non-member utilities within the `Loss` namespace. This design choice avoids the overhead of object instantiation and maintains a functional style for stateless operations.

In the `forward` method[13], the composite physics-informed loss, described in Section 3.1.2, is computed in a single loop using raw pointers retrieved via `get_data()` to ensure contiguous memory traversal. When computing the physics-informed penalty, the masking logic from Equation (37) is applied: the linear regime limit is saved in an anonymous namespace at the beginning of the file, in order to restrict the scope of this constant. The average data and physics terms are computed and normalized by the number of elements using `static_cast<float>(n)`

Finally, the `backward` method is implemented in a similar way.

Optimizer and Adam Optimizer

Similar to the layer architecture, the optimization logic is strictly decoupled from the neural network topology via an abstract `Optimizer` base class. This unified interface ensures that different numerical solvers can be swapped seamlessly, exposing a single pure virtual function (`apply_gradients(float* weights, float* grads, size_t size)`) to handle dynamic dispatch interface that must be implemented by concrete subclasses.

The only concrete implementation of this interface is the `AdamOptimizer`, which executes the **Adaptive Moment Estimation** algorithm detailed in Section 3.1.2.

To map specific weight and bias arrays to their corresponding optimization states without coupling the layers to the optimizer, the class utilizes a parameter-to-state lookup map as `std::unordered_map<float*, AdamState> _state_by_param_ptr`[14], that provides average constant-time $\mathcal{O}(1)$ lookup complexity. Since weight and bias arrays are allocated once and reside in contiguous memory locations throughout the lifetime of the network, their raw memory addresses (`float*`) serve as unique keys. This pointer-based hashing avoids the overhead of index-based tracking or string-based lookups and enables on-the-fly initialization of the `AdamState` upon the first optimization step.

[13] That accepts predictions, targets and angles of attack (`alphas`) tensors as `const std::shared_ptr<Tensor>&`.

[14] Where `AdamState` is a private structure that contains the first moment vector `std::vector<float> m`, the second moment vector `std::vector<float> v`, and an unsigned integer `timestep` to keep track of the parameter update steps.

CNNModel

The concrete class `CNNModel` coordinates the execution, training and state management of the entire neural network. It aggregates the layers of the convolutional branch and the dense branch, manages feature fusion, interfaces with the optimizer and implements utilities for distributed data-parallel training and weight serialization.

The `forward` method propagates input tensors sequentially through the model. To maximize efficiency, the iteration over the layer containers (`conv_blocks` and `dense_blocks`) utilizes `const auto&`, thereby avoiding the performance overhead associated with incrementing reference counters on the underlying shared pointers. Conversely, the `backward` method employs reverse iterators to correctly propagate gradients upstream in strict accordance with the chain rule.

Another interesting detail is that in the `update` method gradient values are clipped to the interval $[-1.0, 1.0]$ to prevent gradient explosion[15]. This is executed via a local lambda function (`clip_layer`) containing a nested `clip_tensor` lambda, which directly modifies the elements in-place using standard pointer-based loops.

Whenever layer-specific parameters must be accessed for weight updates, gradient synchronization, or serialization, the model dynamically resolves the abstract base pointer types through runtime downcasting via `std::dynamic_pointer_cast` (e.g., casting from `std::shared_ptr<Layer>` to `std::shared_ptr<Conv2DLayer>`).

Finally, the model implements binary weight export and import methods using `std::ofstream` and `std::ifstream`, opened with the `std::ios::binary` flag. Binary data is written and read directly via type-unsafe low-level operations utilizing `reinterpret_cast<const char*>` and `reinterpret_cast<char*>` to map raw byte buffers, which minimizes I/O latency. During the import phase, after reading the shapes and element counts from the file, weight vectors are populated using `std::move` (from the `<utility>` header) to transfer the ownership of heap-allocated arrays into the target structures, bypassing expensive memory allocation and copying. To illustrate this efficient binary format, Table 4 details the layout of the file header and the metadata corresponding to the initial `Conv2DLayer`.

Table 4: Example layout of the binary weight file header, showing the global count of layers followed by the name and weight/bias tensor metadata of the first layer (`Conv2DLayer`).

Byte Offset	Field Name	Value / Example
`0x00-0x07`	Total layers in model (N_{layers})	`8`
`0x08-0x0F`	Layer name length (L_{name})	`39`
`0x10-0x36`	Layer name string	`"Conv2DLayer(1,8,5, stride=5, padding=0)"`
`0x37`	Weights presence flag	`true` (`0x01`)
`0x38-0x3F`	Weights tensor rank (R_W)	`4`
`0x40-0x5F`	Weights tensor shape	`[8, 1, 5, 5]`
`0x60-0x67`	Total weights count (N_W)	`200`
`0x68-0x387`	Weights raw data	`[-0.0125, 0.0841, ...]`
`0x388`	Biases presence flag	`true` (`0x01`)
`0x389-0x390`	Biases tensor rank (R_B)	`1`
`0x391-0x398`	Biases tensor shape	`[8]`
`0x399-0x3A0`	Total biases count (N_B)	`8`
`0x3A1-0x3C0`	Biases raw data	`[0.0, 0.0, ...]`

3.2.6. Data Pipelines: `NPZDataLoader`

The `NPZDataLoader` class manages the retrieval, normalization, and partitioning of datasets stored in the compressed NumPy (`.npz`) format. This format was explicitly selected because it packages multiple named multidimensional arrays within a single ZIP archive, acting as an efficient bridge between Python-based preprocessing scripts (such as `build_dataset.py`) and the C++ training pipeline without the severe parsing overhead

[15] This choice has been made after training the model for the first time, where we have noticed very strong fluctuations of the loss value during the training process.

of text-based formats like CSV.

Specifically, the dataset is structured into four core arrays:
- `X_sdf`, representing the 150×150 SDF of the airfoil geometries;
- `X_scalars`, storing the Reynolds number (Re) and the angle of attack (α);
- `Y_cl`, containing the ground-truth lift coefficient (C_L) labels computed from the Navier-Stokes simulations;
- `metadata`, containing the specific NACA airfoil profile labels.

This class provides a complete input pipeline, converting these raw data arrays into normalized, formatted tensor batches for network training.

To import the data without introducing heavy external C++ NumPy parsing libraries, the internal reading method establishes a read-only POSIX pipe to execute a brief Python script, that utilizes Python's native numpy library to load the specified array, converts it to raw single-precision bytes and streams the output directly to the standard output. The C++ program then intercepts this byte stream, reading it sequentially into a standard vector buffer before safely closing the process and verifying its exit status.

Once the arrays are loaded into memory, the constructor applies Z-score standardization across the SDF, scalar parameters and target lift coefficients, to prevent numerical instability during the training phase. Then, the class handles the sequential feeding of this data during training. The batching process partitions the normalized dataset into mini-batches by maintaining an internal cursor that tracks the current read position. For each requested batch, it allocates new tensor objects and utilizes standard library copying algorithms to transfer the exact slice of data from the internal buffers to the newly created tensors.

3.2.7. Main execution of the CNN

The main execution entry point of the neural network workflow is defined in `main.cpp`. It coordinates the loading of datasets, the structural composition of the CNN layers and the training and validation loops over successive epochs.

The program begins by instantiating the training and testing pipelines using `NPZDataLoader` to load the compressed NumPy datasets. It then creates a shared instance of the `AdamOptimizer` wrapped in a `std::shared_ptr` via `std::make_shared` with a designated learning rate of 10^{-5}. This optimizer is passed to initialize the `CNNModel` instance.

The architecture of the neural network is composed using the model's builder methods:

1. the convolutional branch is constructed by adding a `Conv2DLayer` with 1 input channel, 8 output channels, a 5×5 kernel, and a stride of 5, followed by a `LeakyReLULayer` (with an activation slope $\alpha = 0.05$) and a `FlattenLayer` to reduce the spatial feature maps to a one-dimensional representation of size 7200;

2. the feature fusion stage is enabled by registering a `ConcatenateLayer` via `set_concatenate_layer()`, which merges the geometry features with the two physical flow parameters (Reynolds number and angle of attack);

3. the fully-connected regression head is built using three sequential `DenseLayer`s of dimensions $7202 \to 128$, $128 \to 64$, and $64 \to 1$, separated by `LeakyReLULayer` activations, resulting in a single scalar prediction representing the lift coefficient (C_L).

The training process iterates for 100 epochs. Within each epoch, it resets the data loaders and processes the dataset in mini-batches. For each batch, it invokes `train_step` on the model instance to run the forward pass, evaluate the PINN loss function, compute parameter gradients via backpropagation and update the weights. After the training phase of each epoch, it runs a validation pass by propagating the validation samples through the network via `forward()` to monitor generalization and check for overfitting by computing the MSE against the validation targets.

3.3. Parallelism

To achieve high-performance capabilities and scale the computational workflow to large-scale grids and datasets, the codebase employs distributed-memory parallelism using the **Message Passing Interface (MPI)** standard.

The parallelization strategy is implemented at two distinct stages: the geometric preprocessing phase (for parallel SDF generation) and the neural network training phase (for distributed data-parallel training).

3.3.1. Parallel SDF Generation

The computation of the SDF, as described in Section 3.1.1, is an embarrassingly parallel task, since the evaluation of the signed distance for each point on the 150×150 grid is independent of all other points.

To do this, upon initializing the MPI environment via `MPI_Init`, the total number of grid points ($N_{\text{total}} = 22,500$) is partioned by the communicator size (S, retrieved using `MPI_Comm_size`). Because there could be some cases where N_{total} is not perfectly divisible by S, the remainder ($R = N_{\text{total}} \bmod S$) is distributed among the first R processes[16]. Each process (with rank ID r, retrieved via `MPI_Comm_rank`) computes its local count and offset as:

$$N_{\text{local}} = \lfloor N_{\text{total}}/S \rfloor + \mathbb{I}(r < R) \quad \text{and} \quad O_{\text{local}} = r\lfloor N_{\text{total}}/S \rfloor + \min(r, R) \tag{44}$$

where $\mathbb{I}$ is the indicator function.

Each process independently computes the SDF values for its assigned local grid points. To reconstruct the complete 2D matrix, the local chunks must be gathered into a global vector of size N_{total}, using `MPI_Allgatherv`, which performs a vector gather-to-all communication. The collective call requires `recvcounts`, which stores the number of elements expected from each rank, and `displs`, which specifies the starting byte displacement of each segment in the global buffer. `MPI_Allgatherv` ensures that all processes receive the full SDF image.

Finally, the root process (rank 0) writes the complete matrix to disk via the `mappingToMatrix` method, while other ranks proceed to the next file or terminate safely via `MPI_Finalize`.

3.3.2. Distributed Data-Parallel Training

The training of the Convolutional Neural Network is parallelized in `CNN/main.cpp` and `CNNModel.cpp` using a data-parallel training strategy to scale to large datasets and decrease training epoch time.

In `CNN/main.cpp`, data parallelism is implemented by distributing the training batch across multiple processes. The global mini-batch size is scaled according to $B_{\text{global}} = B_{\text{local}} \times S$, with a local batch size of $B_{\text{local}} = 64$. To prevent redundant memory duplication, each process uses a custom `slice_batch_tensor` method to extract only its specific segment of the global dataset, beginning at the index offset $r \times B_{\text{local}}$.

To guarantee consistent optimization across the distributed environment, the training pipeline enforces MPI synchronization at three critical stages:

1. initially, the root process generates the network weights and distributes them to all other ranks using `MPI_Bcast` on the raw tensor buffers, ensuring an identical starting state[17] across the entire communicator;
2. during the backpropagation phase, each process computes gradients based on its local batch slice. Before applying the Adam optimization step, these local gradients are aggregated via an in-place `MPI_Allreduce` operation. By utilizing the `MPI_SUM` operator directly on the existing memory buffers, the system efficiently avoids redundant data allocations. The accumulated gradients are then divided by the total number of processes S to compute a global average, which guarantees that every rank applies the exact same mathematical update to its local parameters;
3. finally, at the conclusion of each epoch, the local training losses, validation losses, and processed sample counts are gathered back to the root process via `MPI_Reduce`, allowing it to calculate and report the consolidated global statistics.

3.4. Results

Given the significant model size of 930,513 trainable parameters, performing the training process on standard personal computers was computationally impractical within a reasonable timeframe. To overcome this lim-

[16] This partitioning guarantees perfect load balancing, as the workload of any two processes differs by at most a single grid point.

[17] With all the weights initialized to the same value.

itation, the training was conducted on the high-performance computing (HPC) resources of the **CINECA Leonardo cluster**. Specifically, we utilized a full single node configured with two Intel Xeon Platinum 8480+ (Sapphire Rapids) processors, providing a total of 112 physical cores, backed by 512 GiB of DDR5 RAM operating at 4800 MHz. This hardware acceleration reduced the total execution time from several hours to approximately five minutes.

```
#!/bin/bash
#SBATCH --job-name=cnn_training
#SBATCH --account=EUHPC_D35_025
#SBATCH --partition=dcgp_usr_prod
#SBATCH --nodes=1
#SBATCH --ntasks-per-node=1
#SBATCH --cpus-per-task=112
#SBATCH --time=01:00:00
#SBATCH --output=cnn_%j.out
#SBATCH --error=cnn_%j.err


cd /leonardo/home/userexternal/gdonnine/CNN/
   Ice_acceleration_model_on_airplane_wing/CNN

module purge
module load python/3.11.7
module load gcc/11.3.0
module load intel-oneapi-compilers
module load intel-oneapi-mpi

srun ./cnn_mpi_executable
```

Listing 5: SLURM run script for CNN training (`run_cnn.sh`).

The network was trained for 100 epochs on a synthetically generated dataset of 2D SDF, representing five **symmetric NACA airfoil profiles** (NACA 0006, 0008, 0010, 0012 and 0018) across varying angles of attack (α) and Reynolds numbers (Re). Partitioned via an 80/20 split, the dataset yielded 1,713 training samples and 429 validation samples[18].

Initial Unstable Training Run

In our first training attempt, we adopted a standard learning rate of 10^{-3}, without applying input feature normalization or gradient clipping. This configuration resulted in severe numerical instability during the optimization process. As illustrated in Figure 10, the training and validation losses exhibit chaotic, erratic oscillations, fluctuating wildly by over seven orders of magnitude (ranging from 10^{-1} to 10^{6} in log scale). The presence of massive loss spikes[19] is a clear indicator of exploding gradients. Without gradient clipping, large backpropagated gradients led to excessive parameter updates, pushing the weights into unstable regions of the loss landscape and preventing the optimizer from settling into a local minimum.

Optimized Successful Training Run

To resolve the numerical instability and ensure smooth convergence, three key modifications were implemented in the training pipeline:

1. the Adam optimizer's learning rate (η) was reduced from 10^{-3} to 10^{-5};
2. standard Z-score normalization was applied during the data-loading phase to scale the inputs (SDF and physical scalars) and targets (lift coefficients) to zero-mean and unit-variance;
3. gradient clipping was enabled in the update phase to clamp all gradient components strictly within the interval $[-1.0, 1.0]$ (as detailed in Section 3.2.5).

These modifications successfully stabilized the training trajectory.

[18] All of which have been made publicly available on Kaggle

[19] Such as the prominent peaks around epochs 40, 60, 70 and 85.

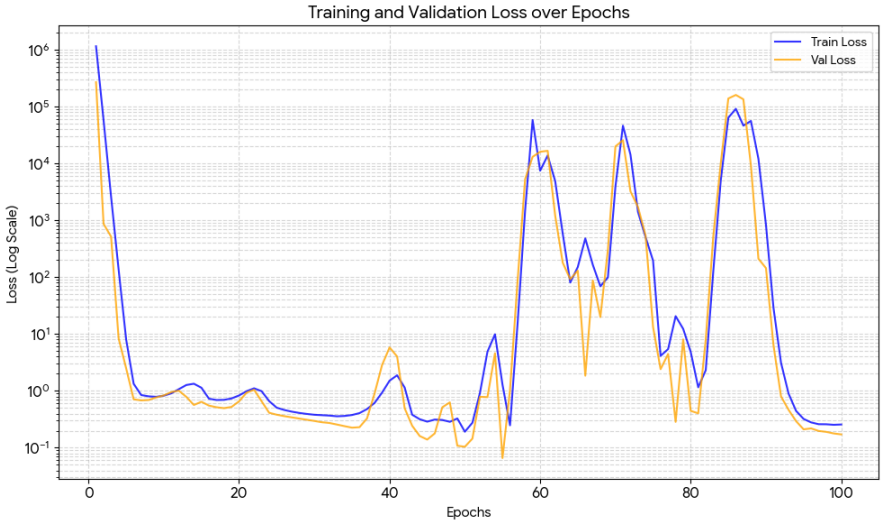


Figure 10: Log-scale evolution of the training and validation losses over 100 epochs during the initial unstable training attempt. The extreme oscillations and sudden spikes up to 10^6 highlight the numerical instability caused by an excessively high learning rate and the absence of gradient clipping or feature normalization.

As shown in Figure 11, the loss curves exhibit highly stable, monotonic convergence. The left plot (Full View) shows the rapid decay of the loss during the initial epochs, indicating that the network quickly learned the dominant physical relationships. The right plot (Zoomed View, Epochs 5-100) highlights the smooth, continuous reduction of both training and validation losses, with no signs of overfitting. The validation loss consistently tracks slightly lower than the training loss; this is a common characteristic when model regularization (such as L2 weight decay) is active during training or when the z-score normalization effectively centers the distribution of test cases.

This optimized training process completed in likely 5 minutes and 11 seconds, achieving a final training loss of 0.0104216 and a validation loss of 0.00784775, demonstrating both the efficiency and accuracy of our implementation.

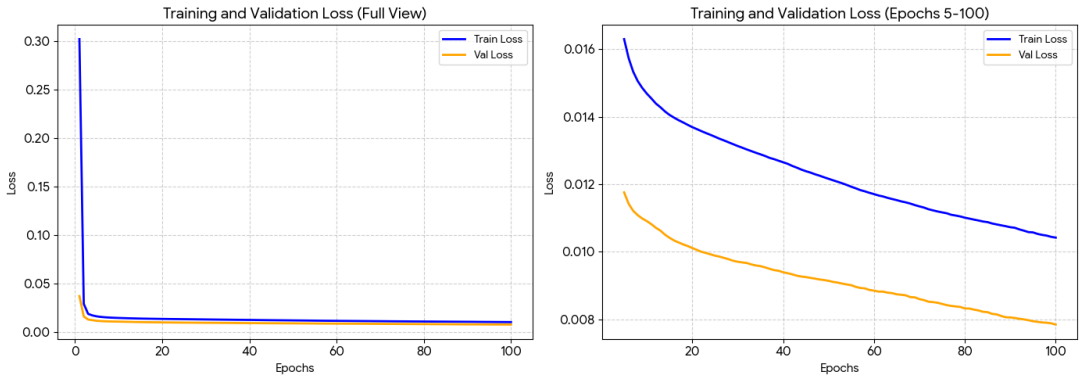


Figure 11: Evolution of training and validation losses for the optimized training run. The full view (left) illustrates the rapid initial convergence, while the zoomed-in view from epoch 5 to 100 (right) highlights the smooth, monotonic decrease of both metrics, confirming the effectiveness of Z-score normalization and gradient clipping.

4. Tuning

The Convolutional Neural Network, described and implemented in Chapter 3, provides the starting point for the numerical study presented in this Chapter. Although the initial model is already able to approximate the

lift coefficient from the airfoil geometry and the flow parameters, we decided to understand which choice of architecture, optimization algorithm, regularization, activation function and learning rate allow the surrogate model to obtain more accurate and more reliable predictions. For this reason, we performed a sequence of controlled tuning experiments. In each experiment one group of quantities was varied, while the other settings were kept fixed. The results were then analysed not only through the final validation error, but also through the evolution of the training objective, the physical validation error, the gradient norms, the actual parameter updates and the activation distributions.

The order of the study follows the development of the model:

- first, we investigated the topology of the network, because the feature representation and the capacity of the regression head influence every subsequent experiment;
- then, we considered dropout, the optimizer, the weight of the SIMM physical term, and L1 and L2 regularization;
- next, we compared the available activation functions;
- as last thing, we studied the magnitude and the schedule of the learning rate.

The last part of the Chapter (Section 4.9) describes the ordinary training run performed with the production executable and collects the conclusions that can be drawn from the complete tuning.

4.1. Cross-validation and diagnostic methodology

All the tuning experiments use the training dataset stored in `dataset/cnn_dataset_train.npz`[20], which contains 1713 samples. Each sample consists of a 150×150 Signed Distance Function[21], the scalar pair (Re, α) and the corresponding lift coefficient C_L. The separate file `dataset/cnn_dataset_test.npz`, containing 429 samples, is not used to compare candidates. In the intended procedure, it is loaded only after a candidate has been selected and the selected model has been refitted on the complete training dataset. This separation is important because using test data during the search would turn the final error into a selection quantity rather than an independent estimate.

The comparisons are performed with the project implementation of `RandomKFold`. The samples are shuffled with a fixed seed and divided into five folds. An important detail is that the data is spli on a per-sample basis, meaning individual simulation results are distributed randomly rather than grouping all data from a single airfoil into one fold. This means that for any given fold, data from the same airfoil (e.g., NACA 0012) can appear in both the training and validation sets. In this way, the tunings experiments measure the model's ability to interpolate within the dataset, rather than its capacity for generalization to entirely new airfoil families.
In every fold, approximately eighty percent of the samples are used for training and the remaining twenty percent for validation. In the recorded runs, the validation sets contain 342 (or 343 samples) and the training sets contain 1370 (or 1371 samples). The fold plan is generated once and reused for every candidate within a tuning experiment, guaranteein that every candidate model is benchmarked on the exact same data for a direct comparison.

The experimental procedure enforces the following key principles, in order to ensure reproducible comparisons. First of all, the normalization statistics (such as mean and standard deviation) are calculated exclusively from the training data. These same statistics are then used to scale the validation data. We also added a minor preprocessing step to convert the angle of attack from degrees to radians, which is necessary for the physics-informed loss function.
Next, every trial is completely independent. For each candidate model and each fold, a new instance of the network and optimizer is created from scratch, in order to not carry over any learned parameters, optimizer moments or any other state from previous runs.
Finally, the distributed training framework is designed to be consistent. The global batch size is fixed for all experiments, and increasing the number of MPI processes simply divides this global batch into smaller sub-batches for each worker. This ensures that the results and metrics are independent of the number of parallel ranks used.

With this experimental framework established, we evaluate each candidate using a primary selection metric, supplemented by a series of diagnostic quantities to analyze model behavior.

The primary selection quantity is the physical-unit mean squared error on the validation samples. If $\hat{C}_{L,i}$ is the

[20] Both the training and test datasets are generated according to the procedure detailed in Section 2.
[21] See Section 3.1.1 for details about the implementation.

predicted coefficient and $C_{L,i}$ is the CFD value, the physical MSE of fold k is computed as:

$$\mathrm{MSE}_{\mathrm{val}}^{(k)} = \frac{1}{n_k} \sum_{i \in \mathcal{V}_k} \left(\hat{C}_{L,i} - C_{L,i} \right)^2 \tag{45}$$

where $\mathcal{V}_k$ is the validation subset and n_k is its number of samples.
The final score for a candidate is the sample-weighted average of this MSE across all five folds, as returned by `CrossValidator`. This distinction matters because the training objective contains a data term and, in some experiments, a physics term expressed in fold-specific standardized target units, while Equation (45) is expressed in the squared physical lift-coefficient units. To assess the sensitivity of a model to the specific data split, the standard deviation of the MSE across folds is also reported alongside the mean.

Beyond the primary selection metric, we monitor several diagnostic quantities to ensure numerical stability and understand the training dynamics. The training objective is monitored through `train_objective`, while the physical training and validation errors are read from `epoch_metrics.csv`. The cross-validation history is normally stored every ten epochs, although diagnostics contain a validation value at every epoch.
The following diagnostic quantities are used throughout the chapter:

- the gradient norm is read from rows with `parameter_scope=all` and it is measured after the MPI weighted gradient aggregation and before clipping and the optimizer update. This metric indicates whether gradients are naturally vanishing or exploding;
- the parameter update ratio is the actual movement of the parameters relative to their norm, computed as:

$$r_\theta = \frac{\|\theta_{\mathrm{after}} - \theta_{\mathrm{before}}\|_2}{\max\left(\|\theta_{\mathrm{before}}\|_2, 10^{-12}\right)}, \tag{46}$$

  where θ_{before} and θ_{after} are the parameter vectors before and after the optimizer update, respectively. A healthy value typically hovers around 10^{-3}. Excessively high values point to numerical instability or an overly aggressive learning rate, whereas excessively low values indicate training stagnation. The 10^{-12} term serves as a safety epsilon to prevent division by zero.
- the pre-activation and post-activation statistics are computed for every layer, including the mean, variance, minimum, maximum, and fixed-bin histograms of the activation tensors;
- the activation statistics are distinguished between the pre-activation tensor entering a non-linearity and the post-activation tensor returned by it. Their mean, variance, minimum, maximum and fixed-bin histograms are used to identify signal attenuation, saturation, or growth of the activation scale.

In all cases, we checked the raw CSV files for non-finite values and for late-epoch increases before describing a run as numerically stable.

4.2. Topology tuning

The first tuning activity concerned the topology of the network, as it dictates the model's fundamental capacity to learn from the data. The architecture has two key components:

- a convolutional feature extractor responsible for extracting meaningful geometric features from the 150×150 SDF input;
- a dense head that fuses these geometric features with the scalar flow parameters (Reynolds number and angle of attack) to predict the final lift coefficient.

The main focus is to find the right balance. A model with insufficient capacity (too few layers or neurons) may fail to capture the non-linear relationship between an airfoil's shape and its aerodynamic performance. Conversely, an overly complex model risks poor generalization, fitting noise in the training data rather than the underlying physical principles, while also increasing computational cost.

This type of structural comparison is consistent with the surrogate model motivation introduced in Chapter 3. The reference to the role of activation and gradient propagation in deep networks discussed by Glorot and Bengio Glorot and Bengio (2010) is kept here only as motivation for studying signal propagation in principle, since that work considers saturating nonlinearities and initialization and does not predict a kernel size or a head width for this SDF trunk.

Therefore, this experiment systematically explores different architectural choices, such as the convolutional kernel size, the width and depth of the dense head, to identify a topology that optimally balances representational power with generalization performance. All other hyperparameters, such as the optimizer and loss function, were held constant to isolate the impact of the architecture itself.

For the convolutional trunk, we evaluated different kernel sizes, which directly govern the spatial resolution of the resulting feature maps and, consequently, the input dimensionality to the dense head. For a two-dimensional convolution with input size $H_{\text{in}} \times W_{\text{in}}$, kernel size K, stride S, and zero padding P, the output dimensions are

$$H_{\text{out}} = \frac{H_{\text{in}} - K + 2P}{S} + 1, \qquad W_{\text{out}} = \frac{W_{\text{in}} - K + 2P}{S} + 1. \tag{47}$$

The topology experiment used $P = 0$ and set the stride equal to the kernel size, with eight output channels in every candidate. The two main alternatives for the trunk were a 5×5 kernel (producing a 30×30 feature map, leading to 7202 inputs for the dense head) and a 3×3 kernel (producing a 50×50 feature map, leading to 20002 inputs for the dense head).

For the dense head, we explored variations in both width (the number of neurons in each layer) and depth (the number of hidden layers). A head with widths $(w_1, \ldots, w_m)$ then applies successive transformations $\mathbf{h}_{j+1} = \sigma(W_j \mathbf{h}_j + \mathbf{b}_j)$ with LeakyReLU nonlinearity σ and ends with a linear scalar output. Increasing the widths mainly affects the capacity of the first dense transformation, while adding hidden layers changes the depth of the nonlinear mapping. The parameter count makes this distinction concrete. With the 5×5:

- the baseline head $(128, 64)$ holds 930305 parameters;
- the intermediate head $(512, 256)$ holds 3819521 parameters;
- the selected head $(1024, 512, 256, 128)$ holds 8065025 parameters;
- the widest depth two head $(2048, 1024)$ holds 16850945 parameters.

Each dense layer contributes $(D_{\text{in}} + 1)D_{\text{out}}$ weights and biases and the convolution contributes $(K^2 + 1) \times 8$ parameters.

To systemically evaluate these architectural trade-offs, we designed a comprehensive grid search. The search contained 18 candidates and therefore 90 fold evaluations. Each fold was trained following the same recipe:

- 100 epochs with global batch size 64;
- Adam optimizer with learning rate 10^{-5};
- a SIMM physics weight of 0.25;
- LeakyReLU activation with negative slope 0.05.

The baseline was `conv5x5-dense-128-64`. The entire experiment was conducted following the rigorous cross-validation framework, evaluation metrics and rules detailed in Section 4.1.

The official results and logs for this specific search are committed in the CINECA Leonardo log `run_leonardo_51807287.log` and its corresponding summary files. It should be noted that detailed per-epoch diagnostic logging (e.g. gradient norms, activation statistics) was disabled for this particular grid search. The only convergence histories are the public ten epoch checkpoints printed to the log.

The results of the 90-fold evaluation, summarized in Table 5, reveal the trends regarding the impact of network architecture on prediction accuracy. The ranked performance of all candidates is shown in Figure 12.

Candidate	Mean validation MSE	Fold standard deviation
`conv5x5-dense-1024-512-256-128`	0.004560	0.000946
`conv5x5-dense-1024-512-256-128-64`	0.004626	0.000918
`conv5x5-dense-1024-512`	0.004851	0.000911
`conv5x5-dense-512-256-128-64`	0.004872	0.001125
`conv5x5-dense-512-256`	0.005052	0.001067
`conv5x5-dense-1536-768`	0.005307	0.001177
`conv5x5-dense-2048-1024`	0.005673	0.001466
`conv5x5-dense-128-64` (baseline)	0.006010	0.000823
`conv3x3-dense-128-64`	0.006330	0.001099
`conv5x5-dense-64-32`	0.006425	0.001292
`conv3x3-dense-512-256`	0.006459	0.001318

Table 5: Representative topology candidates in ranked order. The ranking was performed over the complete 18 candidate grid by sample-weighted mean physical validation MSE. The omitted intermediate candidates fall between the rows shown and do not change the conclusions.

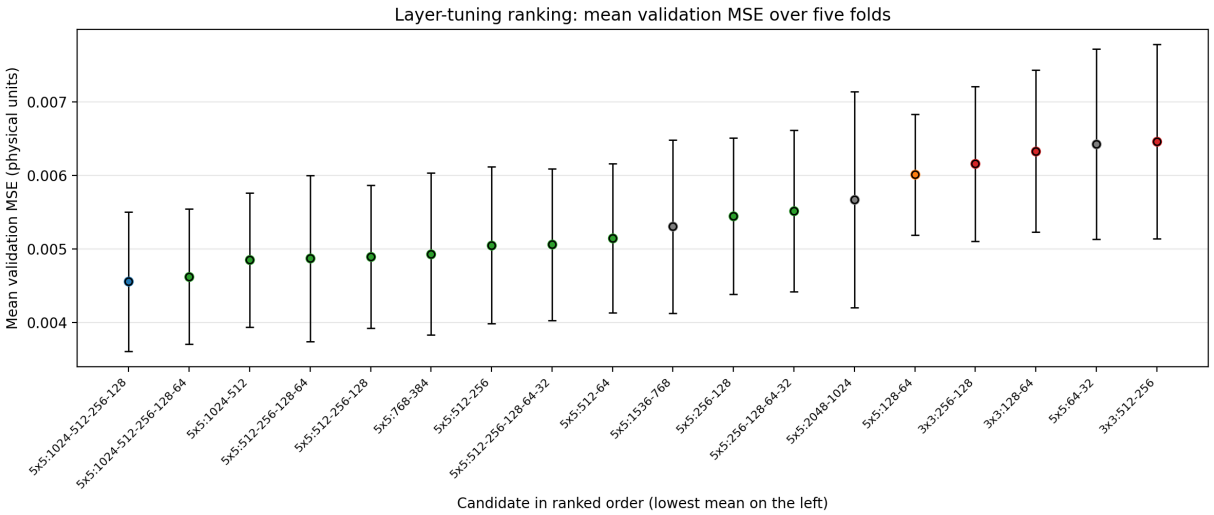


Figure 12: Layer tuning ranking by sample-weighted mean physical validation MSE over five folds. Error bars show the fold standard deviation. The leftmost candidate is the selected `conv5x5-dense-1024-512-256-128`. The plot summarizes `summary.csv` and does not imply a fitted scaling law.

The lowest observed validation MSE was obtained by `conv5x5-dense-1024-512-256-128`, with 0.004560 ± 0.000946. This represents a substantial improvement over the baseline `conv5x5-dense-128-64`, which achieved 0.006010 ± 0.000823, outperforming it consistently across all five cross-validation folds. The mean paired difference between the winner and the baseline is -0.001450, indicating an advantage of the selected topology. After being refit on the entire training dataset, this winning topology achieved an MSE of 0.002805 on the untouched test dataset.

The analysis shows that both network width and depth were crucial for achieving this performance, but with diminishing returns.
Deeper networks consistently proved more effectivee at learning the complex relationship between airfoil geometry and lift coefficient. We can see that at entry width $(512, 256)$, increasing the head from two to four hidden layers reduced the mean error from 0.005052 to 0.004872, while at entry width $(1024, 512)$ the corresponding increase in depth reduced it from 0.004851 to 0.004560.
Increasing width also helped at fixed depth, but the gains became smaller and then reversed. At a fixed depth of two layers, the MSE decreased steadily as the head widened from $(64, 32)$ (with an MSE of 0.006425) to $(1024, 512)$ (with an MSE of 0.004851). However, this trend then reversed. Increasing the width further to $(1536, 768)$ caused the error to rise again to 0.005307, and further to $(2048, 1024)$ with an MSE of 0.005673. This indicates that the model was becoming too large and was beginning to lose its generalization capability.
For this reason, the best performance is not achieved by maximizing either width or depth in isolation, but by finding an effective interaction between the two.

The kernel comparison gives a second clear result. Across all paired comparison, the 5×5 convolution consistently outperformed the 3×3 convolution.
This trend is clearly visible in the paired comparisons shown in Figure 13. For instance, when paired with the $(512, 256)$ dense head, the 5×5 kernel achieved a mean MSE that was lower by approximately 0.001407 compared to the 3×3 kernel. This outcome is likely due to the larger receptive field of the 5×5 kernel. With a larger window, the model can capture broader spatial patterns and more complex geometric details from the SDF input in a single step, useful for the prediction of the lift coefficient.
The convergence history, shown in Figure 14, further supports this conclusion. Models using the 5×5 kernel consistently achieved lower validation MSEs throughout the training process, indicating that they were able to learn more effectively from the data.

However, it is important to acknowledge several limitations inherent to this specific experimental setup:

- as already said in Section 4.1, this grid search was executed without recording detailed per-epoch diagnostics. While all runs completed successfully without signs of divergence, this prevents a deeper analysis of phenomena like vanishing or exploding gradients for these specific candidates;
- each candidate was trained only once per fold with a fixed random seed. Consequently, architecture

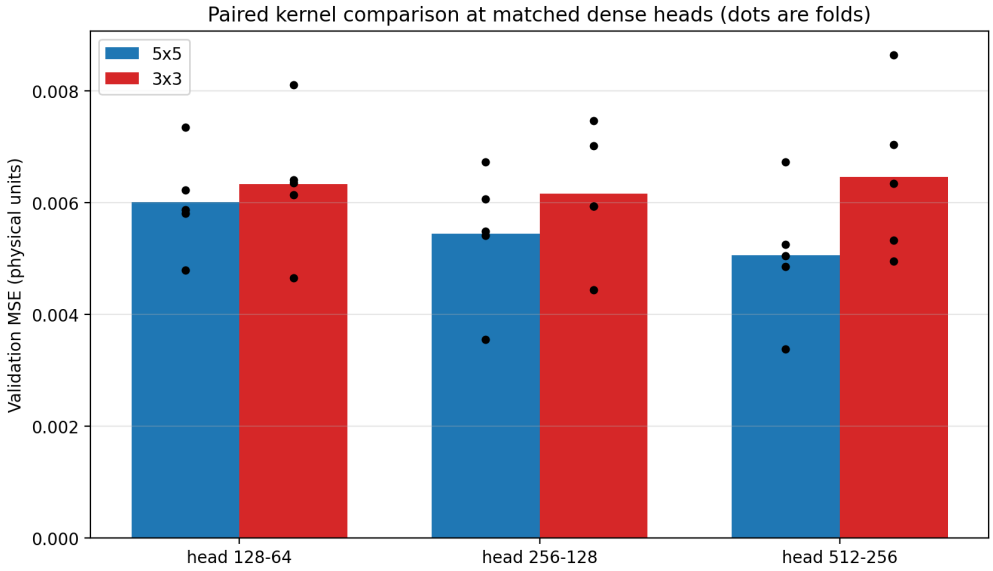


Figure 13: Paired kernel comparison at three matched dense heads. Bars show the sample-weighted mean physical validation MSE and dots show the five per fold values from `aggregated.csv`. All three pairs favour 5×5 with stride five over 3×3 with stride three at zero padding.

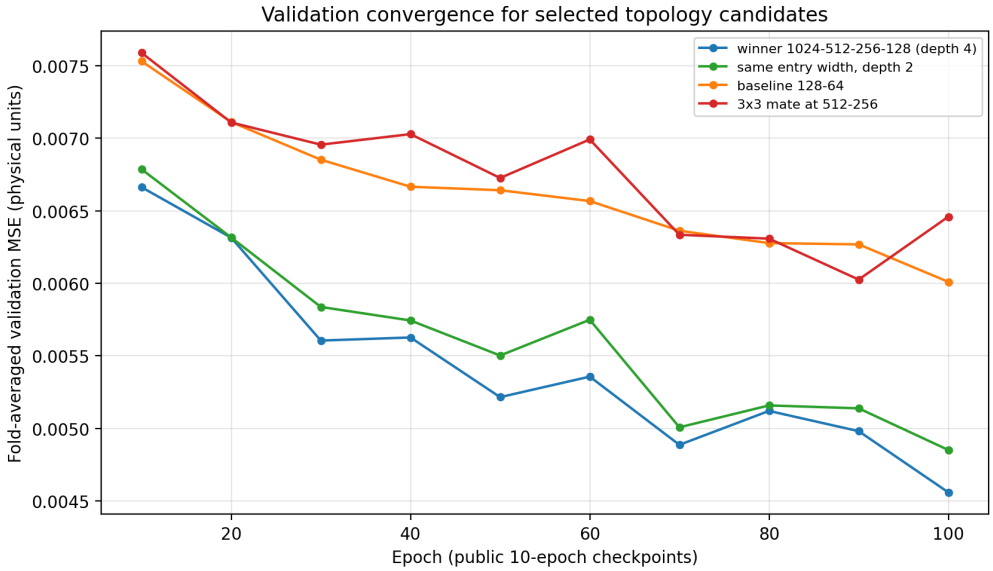


Figure 14: Fold averaged physical validation MSE at public checkpoints every ten epochs, parsed from `run_leonardo_51807287.log`. Curves are shown for the winner `conv5x5-dense-1024-512-256-128`, the depth two candidate `conv5x5-dense-1024-512`, the baseline `conv5x5-dense-128-64`, and the 3×3 mate `conv3x3-dense-512-256`. The training objective is omitted because it uses different standardized units.

difference cannot be separated from training seed noise;

- as established in our methodology (Section 4.1), the training and validation sets are split at the sample level, meaning that data from the same airfoil can appear in both sets. This setup measures interpolation within the dataset rather than generalization to unseen airfoil families;
- the grid search was limited to a specific range of kernel sizes, head widths, and depths. While the selected candidate performed best within this grid, it is possible that other configurations outside this range could yield even better results.

Cross-validation winner: `conv5x5-dense-1024-512-256-128` with physical validation MSE 0.004560 ± 0.000946 and untouched test MSE 0.002805 after refit. Production configuration: the ordinary model built by `make_single_trial` in `CNN/main.cpp` uses a 5×5 convolution with eight channels and stride five, LeakyReLU with $\alpha = 0.05$, and the smaller dense head $(128, 64, 1)$, with source defaults Adam learning rate 10^{-3}, physics weight 0.10, no dropout,

no L1 or L2 penalty, gradient clipping at 1, batch size 257, and seed 42. The production choice therefore differs from the topology winner in head capacity, learning rate, physics weight, and batch size. To conclude, we retain the topology winner as the lowest observed candidate of this grid, while the numbers reported for the final long training run in Section 4.9 describe the recorded production configuration and must not be read as a test score for the large topology winner.

4.3. Dropout tuning

Dropout is a regularization technique designed to prevent overfitting in large networks by randomly deactivating neurons during training, which discourages complex co-adaptations between them Srivastava et al. (2014). Typically, it is most effective when there is a clear gap between training and validation performance. The preceding regularization experiment, however, showed only a small training to validation gap relative to the fold variability. The dropout experiment therefore tests a specific question: can stochastic masking improve validation accuracy when the available evidence does not indicate a large deterministic overfitting gap?

During training, a random subset of activations in a layer is set to zero. The remaining are scaled up by the reciprocal of the survival probability to maintain the expected magnitude of the layer's output. This process, which effectively trains a large ensemble of thinned subnetworks, can be expressed mathematically as:

$$\widetilde{z}_j = \frac{m_j}{1-p} z_j, \qquad m_j \sim \text{Bernoulli}(1-p) \tag{48}$$

where p is the probability of dropping an element and $1-p$ is the survival probability. The same scaled mask is used during backpropagation, so that:

$$\frac{\partial L}{\partial z_j} = \frac{m_j}{1-p} \frac{\partial L}{\partial \widetilde{z}_j} \tag{49}$$

At inference, the mask is disabled and the layer is the identity, so no additional rescaling is needed.

In the implementation, a `DropoutLayer` is inserted after every hidden `LeakyReLULayer` within the dense regression head. It is not applied to the final scalar output. The tested grid was $p \in \{0, 0.1, 0.2, 0.3, 0.5\}$, including the baseline case of no dropout.

To isolate the effect of dropout, all other hyperparameters were held constant, retaining the winning topology from Section 4.2. The experiment strictly followed the established cross-validation protocol detailed in Section 4.1, using the Adam optimizer (learning rate of 10^{-5}), a SIMM physics weight of 0.25, batch size 64 and a 100-epoch budget across identical data folds.

For rigorous reproducibility, the dropout masks are generated deterministically from a combination of the fold seed, epoch, sample row, and feature index. This design ensures that the stochastic mask remains identical regardless of the number of parallel MPI ranks employed.
Crucially, the baseline candidate ($p = 0.0$) retains the dropout layer as an identity mapping rather than omitting it. This guarantees that all candidates share identical parameter initializations and random number sequences. Consequently, the resulting baseline performance is strictly paired within this sweep and should not be compared directly against standalone runs configured without the identity layer.

Candidate selection used the sample-weighted physical-unit validation MSE returned by `CrossValidator`. The official aggregate file is `cv_summary.csv`; the paired differences, final training MSE, and training to validation gaps are read from `sweep_dropout.csv`. The two files differ slightly in their arithmetic means because the official score weights the 342 and 343 sample validation folds, while the sweep CSV is also used to preserve the fold-by-fold paired inputs. Table 6 reports the complete grid. The last three columns are descriptive quantities and do not replace the physical validation MSE as the selection metric.
The lowest validation error was achieved with dropout disabled ($p = 0$), yielding a mean MSE of $0.005833893 \pm 0.001006787$. As shown in Figure 15, performance systematically degraded as the dropout rate increased. Even a small rate of $p = 0.1$ increased the error by nearly 15%, while the strongest rate of $p = 0.5$ more than doubled the validation MSE.

This conclusion was remarkably consistent across all data splits. As visualized in the paired difference plot (Figure 16), no non-zero dropout rate improved performance in any of the five cross-validation folds.

This outcome directly answers the experimental question posed in our motivation. For the baseline model ($p = 0.0$), the gap between training and validation error was already very small (+0.000413), indicating no sig-

p	Mean val. MSE	Fold std.	Paired Δ	Mean train MSE	Val. − train
0.0	0.005834	0.001007	0.000000	0.005421	+0.000413
0.1	0.006686	0.001060	+0.000852	0.006390	+0.000296
0.2	0.007630	0.000861	+0.001796	0.007236	+0.000395
0.3	0.008467	0.001141	+0.002632	0.008098	+0.000369
0.5	0.011764	0.002603	+0.005930	0.011354	+0.000411

Table 6: Complete recorded dropout sweep. Mean validation MSE and fold standard deviation are the sample-weighted values from `cv_summary.csv`. Paired differences, training MSE, and validation minus training gaps are calculated from `sweep_dropout.csv`; a positive paired difference means that the rate is worse than the $p = 0$ reference.

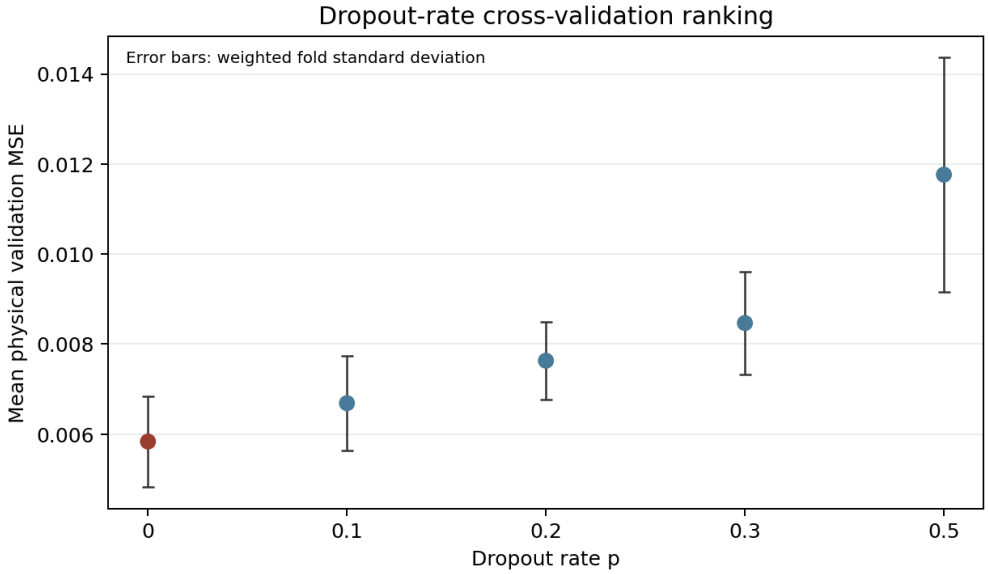


Figure 15: Dropout-rate ranking from the official `cv_summary.csv`. Points are sample-weighted mean physical validation MSE and error bars are the weighted standard deviations over the five folds. The $p = 0$ reference is highlighted in red.

nificant overfitting problem. While a small dropout rate ($p = 0.1$) did slightly reduce this gap (to +0.000296), this minor improvement came at the cost of a much larger increase in the overall validation error.

This pattern strongly suggests that, for this well-specified architecture and dataset, dropout did not act as a useful regularizer. Instead, it functioned primarily as optimization noise, hindering the model's ability to converge to an accurate solution within the fixed 100-epoch training budget. Therefore, the evidence from this experiment supports disabling dropout entirely for the production model.
A closer look at the training dynamics, presented in Figure 17, reinforces this conclusion. The training history reveals that all candidates converge stably, but to different levels: the final mean is approximately 0.0123 for $p = 0$ and 0.1142 for $p = 0.5$. These objective values are in standardized SIMM units and are not numerical alternatives to the physical validation MSE.
Next, as seen in the right panel, the validation error for the baseline $p = 0$ model consistently remains the lowest throughout the 100 epochs (from 0.0071253 at epoch 10 to 0.0058346 at epoch 100). Conversely, increasing the dropout rate raises the final error value, with the $p = 0.5$ candidate converging to a solution approximately twice as poor as the baseline (the MSE during validation ranges from 0.0227140 at epoch 10 to 0.0117650 at epoch 100). The curves contain small fold-averaged fluctuations, particularly for the nonzero rates, but none shows a late unbounded increase.

To understand why dropout degraded performance, we analyzed the detailed training diagnostics, summarized in Figure 18. These diagnostics confirm that the poorer performance was not caused by numerical instability but by the disruptive effect of dropout on the optimization process.
We can see that no run was numerically unstable. Looking at the bottom right plot, while stronger dropout rates led to larger initial gradient transients (peaking around 47.444652 for $p = 0.5$, compared to 37.841272 for

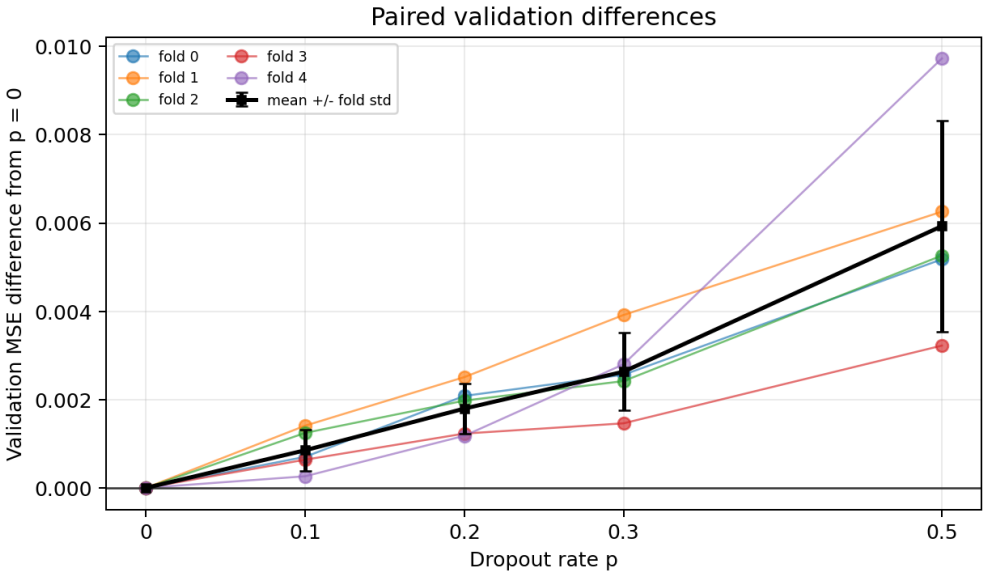


Figure 16: Paired physical validation MSE differences relative to the $p = 0$ reference, computed from `sweep_dropout.csv`. Coloured curves show the five zero-based folds, while black markers and error bars show the mean paired difference and its sample standard deviation. Positive values indicate deterioration relative to the reference.

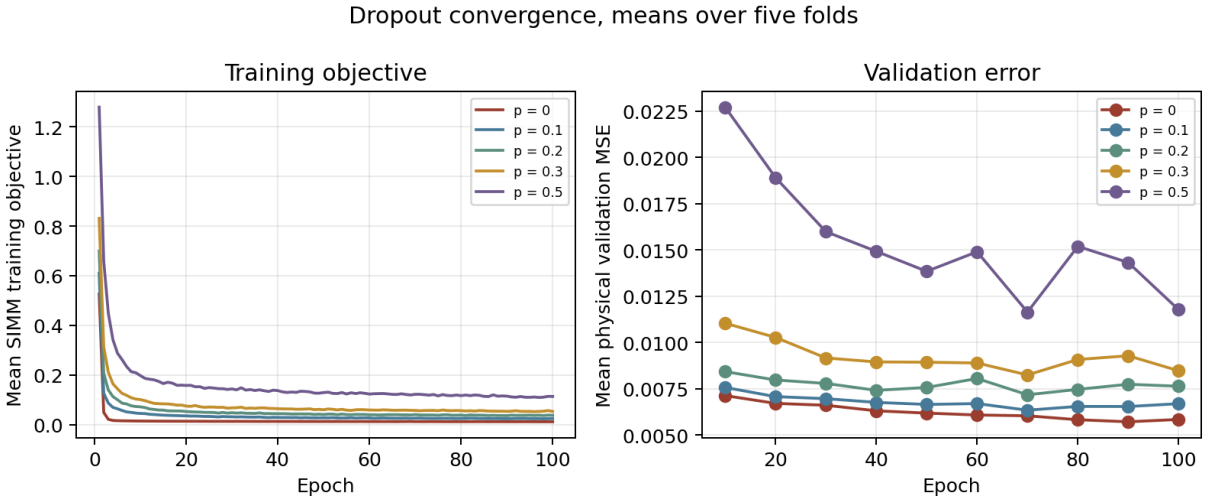


Figure 17: Means over the five recorded folds for the training SIMM objective and the physical validation MSE. The training objective is recorded at every epoch, whereas the validation values in the committed diagnostic files occur at ten-epoch checkpoints. The two panels use separate units, and the curves describe the recorded dropout sweep.

$p = 0.0$), these were controlled, early-epoch events. All gradients remained finite and decreased substantially as training progressed, confirming that the high validation error was not the result of a failed or divergent run.

Looking at the other diagnostic panels, late in training, both the average gradient norm and the parameter update ratio systematically increase with the dropout rate. This indicates that dropout forces the optimizer to take larger, more aggressive steps even late in training. However, the post-activation variance shows the opposite trend: at epoch 100, the signal variance decreases as the dropout rate increases. This suggests that while dropout forces larger parameter updates, it may also lead to weaker overall signal propagation through the network.

Taken together, these diagnostics paint a clear picture. Dropout did not act as a helpful regularizer; instead, it introduced optimization noise that led to larger, less efficient parameter updates while simultaneously attenu-

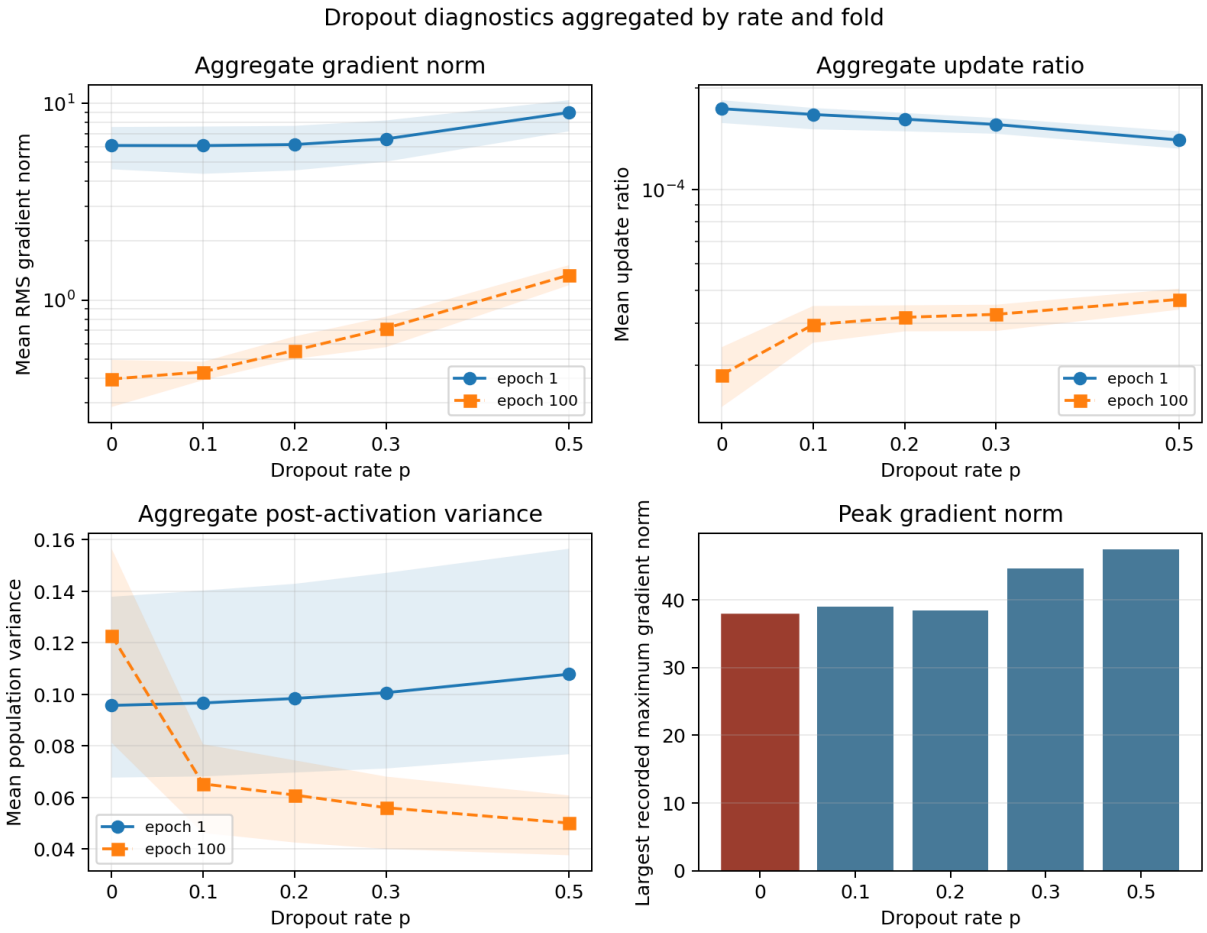


Figure 18: Diagnostics for the recorded dropout sweep. The aggregate gradient norm, actual update ratio, and post-activation variance panels show means with minimum to maximum bands over five folds at epochs 1 and 100; the horizontal axis is dropout rate and the first two vertical axes are logarithmic. The final panel shows the largest recorded `maximum_norm` with `parameter_scope=all` over all folds and epochs for each dropout rate. Diagnostic rows are aggregated and individual layers are not distinguished.

ating the network's internal signal. This combination fully explains the degraded validation performance.

Therefore, the cross-validation winner is unequivocally `dropout rate = 0.0`, with a final validation MSE of $0.005833893 \pm 0.001006787$. This choice aligns with the default CNN production configuration, which also disables dropout. The evidence from this experiment strongly supports retaining $p = 0$ for the final model, as every non-zero rate worsened the paired validation result without providing a meaningful reduction in the already-small overfitting gap.

4.4. Optimizer comparison

The optimizer determines how the gradient produced by the SIMM loss is converted into a change of the trainable parameters. This choice is important for the present CNN because the convolutional feature extractor and the dense regression head have parameters with different dimensions and gradient histories. A fixed learning rate can therefore lead to different effective progress depending on whether the method retains a direction from previous batches or scales each coordinate using past squared gradients. AdaGrad formalizes cumulative coordinate adaptation Duchi et al. (2011), while Adam combines first and second moment estimates with bias correction Kingma and Ba (2015). We tested whether these mechanisms, together with momentum, improve the physical validation error within the same finite training budget. The result is a comparison at one learning rate and is not a claim that an optimizer is universally preferable.
Let $g_{t,j}$ denote the gradient for parameter component j after the MPI weighted aggregation and after the loss-gradient contributions have been combined. The implementation clips each component immediately before the

optimizer is called, so that the gradient actually consumed by every candidate is

$$\widetilde{g}_{t,j} = \mathrm{clip}(g_{t,j}, -c, c), \qquad c = 1. \tag{50}$$

The clipping is element-wise rather than a global norm operation. In this experiment the L1 and L2 loss weights and the optimizer weight decay are all zero, so no additional regularization term changes $g_{t,j}$. For vanilla SGD, the update is

$$\theta_{t+1,j} = \theta_{t,j} - \eta \widetilde{g}_{t,j}. \tag{51}$$

For SGD with momentum, the implementation stores a velocity $u_{t,j}$ and applies

$$u_{t,j} = \mu u_{t-1,j} + \widetilde{g}_{t,j}, \qquad \theta_{t+1,j} = \theta_{t,j} - \eta u_{t,j}, \qquad \mu = 0.9. \tag{52}$$

There is no Nesterov step in the project implementation. AdaGrad stores the cumulative squared gradient,

$$G_{t,j} = G_{t-1,j} + \widetilde{g}_{t,j}^2, \qquad \theta_{t+1,j} = \theta_{t,j} - \frac{\eta \widetilde{g}_{t,j}}{\sqrt{G_{t,j} + \varepsilon}}, \qquad \varepsilon = 10^{-8}, \tag{53}$$

whereas RMSprop replaces the cumulative sum with an exponential average,

$$s_{t,j} = \rho s_{t-1,j} + (1-\rho)\widetilde{g}_{t,j}^2, \qquad \theta_{t+1,j} = \theta_{t,j} - \frac{\eta \widetilde{g}_{t,j}}{\sqrt{s_{t,j} + \varepsilon}}, \qquad \rho = 0.9. \tag{54}$$

Finally, Adam stores both a first moment $m_{t,j}$ and a second moment $v_{t,j}$:

$$\begin{aligned} m_{t,j} &= \beta_1 m_{t-1,j} + (1-\beta_1)\widetilde{g}_{t,j}, & v_{t,j} &= \beta_2 v_{t-1,j} + (1-\beta_2)\widetilde{g}_{t,j}^2, \\ \widehat{m}_{t,j} &= \frac{m_{t,j}}{1-\beta_1^t}, & \widehat{v}_{t,j} &= \frac{v_{t,j}}{1-\beta_2^t}, \\ \theta_{t+1,j} &= \theta_{t,j} - \eta \frac{\widehat{m}_{t,j}}{\sqrt{\widehat{v}_{t,j}} + \varepsilon}, & \beta_1 &= 0.9, \quad \beta_2 = 0.999, \quad \varepsilon = 10^{-8}. \end{aligned} \tag{55}$$

In all five equations $\eta = 10^{-5}$. The state variables are initialized to zero for every newly constructed optimizer, and Adam starts with $t = 0$. They persist across mini-batches within one fold, but the construction of a fresh model and a fresh `Optimizer` by `ModelFactory` prevents state from being shared between candidates or folds. The learning rate is constant because no scheduler is attached to the `TrialConfig`; it is not an unrecorded learning-rate schedule.

The fixed model was the selected topology from the previous experiment, `conv5x5-dense-1024-512-256-128`. It maps the 150×150 SDF through `Conv2D` with eight channels, a 5×5 kernel, stride five, and zero padding, followed by LeakyReLU with $\alpha = 0.05$ and flattening. The flattened features are concatenated with the two scalar inputs, (Re, α), and passed through the dense widths $(1024, 512, 256, 128)$ and a final scalar output. The loss was the data MSE plus a SIMM physics residual with weight 0.25; the angle of attack was converted from degrees to radians for that residual, and dropout and L1 and L2 penalties were disabled. The comparison contained five candidates and 25 fold evaluations. It used the 1713-sample training dataset, while the 429-sample test dataset was excluded from candidate selection. Five sample-level folds were generated by `RandomKFold` with shuffling and seed 42. Their validation sizes were 343, 343, 343, 342, and 342 samples, with complementary training sizes of 1370, 1370, 1370, 1371, and 1371. The same fold plan was used for every candidate, and normalization was fitted on each fold's training indices only. The global batch size was 64, giving 22 balanced optimizer steps per epoch, 2200 steps per fold, and 55000 optimizer steps across the comparison. Training shuffled the fold indices at each epoch, and validation and public history were recorded at the configured ten-epoch interval. Early stopping was disabled. MPI distributes the global batches across ranks and synchronizes gradients using the actual sample counts, so the nominal batch size is not multiplied by the number of ranks.

The selection score is the physical-unit validation MSE in Equation (45), not the numerical value of the standardized SIMM objective. The current optimizer-comparison result directory does not contain a diagnostics-enabled `cv_summary.csv`; therefore, the aggregate values in Table 7 were recomputed from its 25 `fold_results.csv` rows with the validation sample counts above. The mean and deviation use the sample-weighted population formulas implemented by `CrossValidator`. This is different from the experiment's `analyze.py` script, whose summary plot uses an unweighted mean and a sample standard deviation. The ranking is unchanged, but the latter deviation is not used here.

Adam was the cross-validation winner with $0.004426734 \pm 0.000793966$ physical validation MSE. It was the lowest-scoring candidate in each zero-based fold, with values 0.002879, 0.004510, 0.004897, 0.004806, and 0.005044. RMSprop was the closest alternative at $0.005215589 \pm 0.000584618$, with fold values from 0.004518 to 0.006173. Thus Adam reduced the weighted mean relative to RMSprop by approximately 15.1% and relative

Optimizer	Mean validation MSE	Fold standard deviation	Minimum fold	Maximum fold
Adam	0.004427	0.000794	0.002879	0.005044
RMSprop	0.005216	0.000585	0.004518	0.006173
AdaGrad	0.006184	0.001108	0.004239	0.007439
SGD with momentum	0.007150	0.001264	0.004791	0.008473
SGD	0.017924	0.005568	0.013063	0.028647

Table 7: Complete optimizer comparison at learning rate 10^{-5}. The means and fold deviations are sample-weighted physical validation MSE values, with the deviation defined as the weighted population standard deviation over the five folds. The minimum and maximum columns show the corresponding unweighted extrema of the fold scores.

to SGD with momentum by approximately 38.1%. AdaGrad and momentum SGD occupied intermediate positions. Plain SGD was not a failed run, since all five folds reached epoch 100 with finite values, but it remained much less accurate at this rate and had the largest fold deviation. Its zero-based fold 3 score, 0.028647420, was the largest value in the comparison. This is evidence of poorer performance and less uniform fold results for this split, not by itself proof of a general optimizer sensitivity.

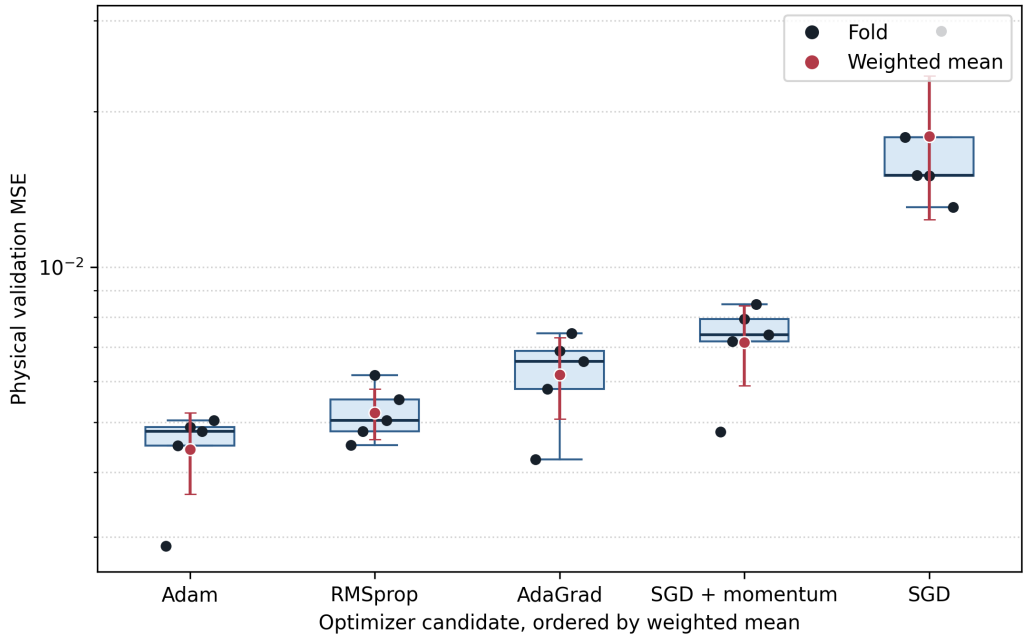


Figure 19: Optimizer ranking from the five-fold comparison at learning rate 10^{-5}. Dark points are individual physical validation MSE values, boxes summarize the five folds, and red points with bars show the sample-weighted mean and weighted population standard deviation. The vertical axis is logarithmic.

The physical training MSE recorded after the final fold epoch was also lower for the better validation candidates, but it must be interpreted separately from the SIMM objective. Using training-sample-weighted means from `fold_results.csv`, the final physical training MSE was 0.003842 for Adam, 0.004814 for RMSprop, 0.005930 for AdaGrad, 0.007027 for momentum SGD, and 0.017337 for SGD. The corresponding validation means are those in Table 7. In particular, the small Adam training to validation difference is not evidence that the optimizer alone controls generalization; it shows that the selected method reached a lower physical training error within the same 100-epoch budget.

The checkpoint histories show how this separation developed. Averaged with the same validation sample weights, Adam's physical validation MSE decreased from 0.006205 at epoch 10 to 0.004800 at epoch 50 and 0.004427 at epoch 100. It was not strictly monotone: the mean rose from 0.004506 at epoch 80 to 0.004597 at epoch 90 before reaching its lowest recorded value at epoch 100. RMSprop reached 0.004682 at epoch 80 and then increased to 0.006073 at epoch 90 before finishing at 0.005216. By contrast, the fold-averaged curves for AdaGrad, momentum SGD, and SGD decreased at the recorded checkpoints, but remained above Adam at epoch 100. Plain SGD fell from 0.273765 at epoch 10 to 0.017924 at epoch 100, which indicates progress without convergence to the error scale reached by the other candidates. Figure 20 shows these physical validation histories on a logarithmic axis. The SIMM training objective is intentionally not placed on the same axis, because it is

expressed in standardized units and includes the physics term.

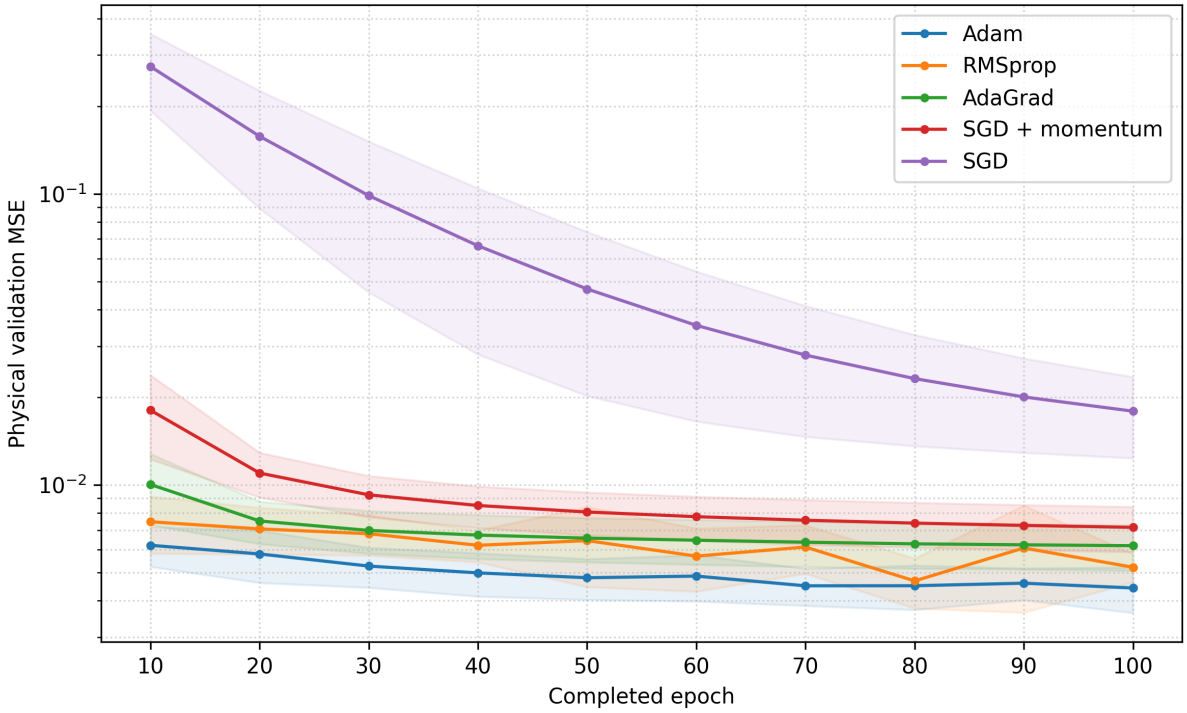


Figure 20: Physical validation MSE at the ten-epoch history checkpoints for the five optimizer candidates. Lines and shaded regions are the sample-weighted means and weighted population standard deviations over the five folds. The vertical axis is logarithmic; the SIMM training objective and the test set are not shown.

The diagnostic conclusion is limited by the provenance of the available files. The matching 10^{-5} comparison was run without the `-diagnostics` option, so none of the structured diagnostic CSV files was produced for these candidates. Consequently, no claim about layer-wise gradient flow, actual parameter movement, activation attenuation, or histogram tails can be made for these five candidates. A separate `run/` directory does contain such diagnostics, but its metadata identifies a different 10^{-3} comparison and an unknown source revision. Those files are not combined with the 10^{-5} results. The two figures above therefore summarize only the compatible fold scores and histories. All 25 compatible fold records are finite and successful, but the absence of matching diagnostics prevents a stronger numerical stability conclusion.

After selecting Adam by cross-validation, the recorded refit reported a physical MSE of 0.00365482 on the 429-sample test dataset in `summary.txt`. The test data were loaded only after the cross-validation search and did not influence the optimizer ranking. There is, however, a limitation in the experiment driver: it passes the test dataset to `Trainer::fit` as its validation dataset and evaluates it after every epoch, rather than literally evaluating it once. Early stopping was disabled, so these repeated measurements did not affect parameter updates or candidate selection, but the value should be described as a post-selection test check with this qualification rather than as evidence used to choose Adam.

The cross-validation winner is therefore **Adam at learning rate** 10^{-5}. Its physical validation MSE was $0.004426734 \pm 0.000793966$ under the fixed large topology, loss, fold plan, and 100-epoch budget. The production configuration is related but not identical: `make_single_trial` in `CNN/main.cpp` constructs Adam, but its source defaults use learning rate 10^{-3}, physics weight 0.10, the smaller dense head $(128, 64, 1)$, global batch size 257, 200 epochs, gradient clipping at 1, and ordinary-training early stopping with a 10 percent holdout, a minimum of 20 epochs, patience of 20 epochs, a maximum overfit ratio of 0.15, and best-weight restoration. The production path also uses the source default seed 42 and no dropout or L1 or L2 penalty. Thus the optimizer algorithm agrees with the selected candidate, while the experiment does not establish the performance of the complete production configuration at the production learning rate. To conclude, Adam is retained as the optimizer choice because it won every compatible validation fold and achieved the lowest observed mean without a failed candidate, whereas the alternatives either made slower progress or reached a higher final error. A new comparison over learning rates and repeated seeds would be required before extending this conclusion beyond the tested setting.

4.5. Physics-weight tuning

The SIMM term was tuned because it is the only part of the objective that encodes an aerodynamic relation not learned directly from the CFD targets. A positive physics weight can improve the low-angle regime if the thin-airfoil relation supplies useful information, but it can also pull the prediction away from the simulation data when the relation is only approximate. The experiment therefore asks whether the prior improves the physical-unit validation error, rather than assuming that a larger physics contribution is beneficial. The search was performed over the training dataset only, and the untouched test dataset was not used to choose the weight. The implementation evaluates the loss in the fold-specific standardized target space. Let $C_{L,i}$ be the physical target, let μ_y and s_y be the target mean and standard deviation fitted on the training part of a fold, and let a_i be the angle of attack in radians. The standardized target and prediction are $y_i = (C_{L,i} - \mu_y)/s_y$ and $\widehat{y}_i$, while the standardized physical target is

$$y_i^{\text{phys}} = \frac{2\pi a_i - \mu_y}{s_y}. \tag{56}$$

The mask is evaluated on the unstandardized angle,

$$M_i = \begin{cases} 1, & |a_i| \le 0.174533 \text{ rad}, \\ 0, & |a_i| > 0.174533 \text{ rad}, \end{cases} \tag{57}$$

which corresponds to $|\alpha| \le 10^\circ$. The actual batch objective is

$$\mathcal{L} = \frac{1}{N}\sum_{i=1}^{N}(\widehat{y}_i - y_i)^2 + \lambda \frac{1}{N}\sum_{i=1}^{N} M_i(\widehat{y}_i - y_i^{\text{phys}})^2, \tag{58}$$

where the physics sum is divided by the full batch size, not by the number of masked samples. The prediction gradient used by backpropagation is consequently

$$\frac{\partial \mathcal{L}}{\partial \widehat{y}_i} = \frac{2}{N}\left[(\widehat{y}_i - y_i) + \lambda M_i(\widehat{y}_i - y_i^{\text{phys}})\right]. \tag{59}$$

The validation score is converted back to physical units by multiplying the standardized prediction residual by s_y before squaring. Therefore, the SIMM objective and the selection metric have different units and cannot be ranked by their numerical magnitudes.

The sweep evaluated seven values, $\lambda \in \{0, 0.05, 0.1, 0.25, 0.5, 1, 2\}$, for 35 candidate-fold evaluations. Adam used a constant effective learning rate of 9.9999997×10^{-6}, the global batch size was 64, the epoch budget was 100, the fold count was five, the base seed was 42, and element-wise gradient clipping used a threshold of 1. No learning-rate schedule or early stopping was enabled. The fold plan was a shuffled sample-level split of the 1713-sample training dataset, with validation sizes 343, 343, 343, 342, and 342. Normalization was fitted on each fold's training indices only, and the test dataset was not loaded during the sweep.

There is an important provenance limitation. The current experiment source constructs the large head `conv5x5-dense-1024-5` but the retained diagnostic metadata for this completed sweep records the earlier head `conv5x5-dense-128-64`. The raw CSV and diagnostic files are therefore reported as an archival run of the physics-weight question, not silently relabelled as measurements from the current source recipe. The figures aggregate over folds and over diagnostic rows without displaying a layer index, so they describe global numerical behaviour and do not imply that incompatible layer topologies were grouped together. The production model is read separately from `CNN/main.cpp` below.

λ	Mean validation MSE	Fold standard deviation	Δ_λ	Improved folds
0.00	0.005600634	0.001099091	0	–
0.05	0.005637647	0.001136246	+0.000037038	1/5
0.10	0.005629100	0.001116111	+0.000028457	1/5
0.25	0.005706661	0.001195944	+0.000106089	1/5
0.50	0.006025918	0.001193569	+0.000425322	0/5
1.00	0.006581512	0.001350790	+0.000981093	0/5
2.00	0.007406998	0.001519330	+0.001806727	0/5

Table 8: Recorded physics-weight sweep. The mean and deviation are the sample-weighted physical validation values in `cv_summary.csv`. Δ_λ is the paired candidate minus reference validation MSE, so positive values are worse than $\lambda = 0$.

The lowest recorded physical validation MSE was obtained at $\lambda = 0$, with $0.005600634 \pm 0.001099091$. The two closest nonzero candidates, 0.10 and 0.05, were worse by 2.8457×10^{-5} and 3.7038×10^{-5}, respectively,

and each improved on the reference in only one of five folds. The stronger values were worse in every fold. None of the nonzero candidates satisfies the pre-registered paired rule of improving in at least four folds with an improvement larger than its own paired spread. Thus the mean ranking and the paired analysis lead to the same data-only choice, while the fold deviations show that the two smallest differences are small relative to split variability.

The angle split tests whether a possible benefit is concentrated where the prior is active. Across the five validation folds there were 1442 low-angle samples and 271 high-angle samples. For $\lambda = 0$, the count-weighted MSE was 0.004096627 in the low-angle group and 0.013603506 in the high-angle group. At $\lambda = 2$, the corresponding values were 0.005843330 and 0.015727326. Both groups therefore worsened as the prior became strong; the effect was not a selective improvement in the masked regime. Figure 21 shows this split together with the complete validation ranking and representative convergence histories.

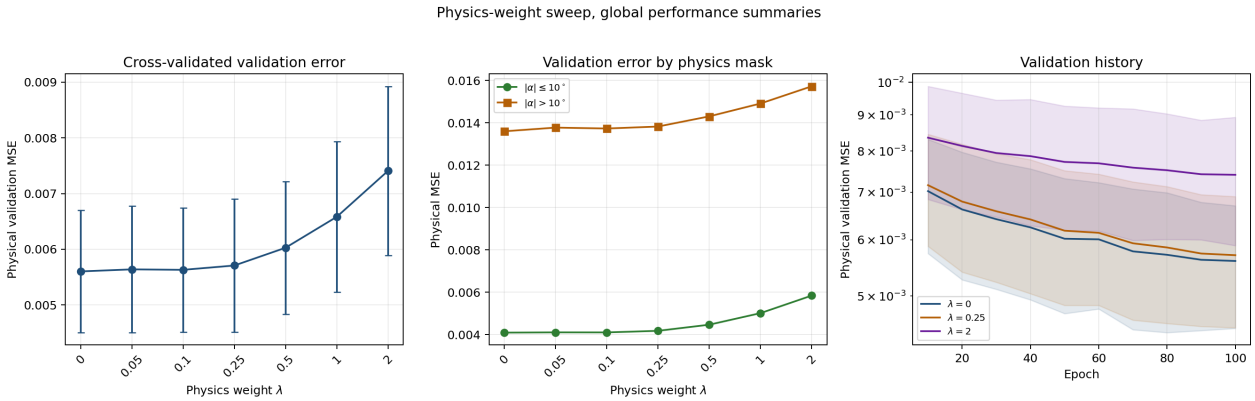


Figure 21: Global summaries of the recorded physics-weight sweep. The first panel shows sample-weighted mean physical validation MSE with fold deviations, the second separates the count-weighted low-angle and high-angle validation errors, and the third shows physical validation histories for $\lambda = 0$, 0.25, and 2 with fold-spread bands. The training SIMM objective is not placed on the validation-MSE axis.

The convergence curves are consistent with the final ranking. The fold-averaged validation MSE for $\lambda = 0$ decreased from approximately 0.007025 at epoch 10 to 0.005602 at epoch 100, whereas the corresponding values for $\lambda = 2$ were 0.008356 and 0.007407. Every candidate completed 100 epochs with finite values. The training objective started and ended higher as λ increased, from approximately 0.599 to 0.0104 for $\lambda = 0$ and from 1.393 to 0.0168 for $\lambda = 2$ when averaged over folds. These are standardized SIMM objectives, so their difference from the physical validation MSE is not a generalization gap.

The diagnostic files provide a numerical stability check without replacing the validation criterion. All numeric fields in the 35 fold directories are finite, the effective learning rate is constant, and the learning-rate step files contain no schedule entries. When the rows with `parameter_scope=all` are combined through $G = \sqrt{\sum_\ell \mathrm{rms\_norm}_\ell^2}$, the epoch-one mean increases from about 18.76 at $\lambda = 0$ to 45.36 at $\lambda = 2$ as the prior is strengthened, while the corresponding epoch-100 values are 0.842 and 1.071. The maximum recorded step norm increases from 56.87 to 149.44, but the largest late-epoch value for $\lambda = 2$ is only 4.20. This is an early transient and not a late exploding-gradient trend. The weights-only actual update ratio also decreases over training, from about 1.69×10^{-4} to 2.43×10^{-5} for $\lambda = 0$. Weight denominators did not produce near-zero events in this scope. Bias-scope ratios were not used because their small denominators can create very large ratios without comparable absolute parameter movement.

The post-activation variance remains finite and changes only modestly in the global aggregate. Averaged over the recorded activation rows and folds, it changes from approximately 0.1092 to 0.1113 for $\lambda = 0$ and from 0.1092 to 0.1114 for $\lambda = 2$. These values do not establish that the physics term causes signal attenuation or amplification. Figure 22 reports the gradient aggregate, actual weights-only update ratio, and post-activation variance without grouping the curves by model layer. The plot is consequently a check of fold and training-time behaviour, not a layer-wise claim.

The cross-validation winner is therefore $\lambda = 0$ for the recorded five-fold run, with physical validation MSE $0.005600634 \pm 0.001099091$. The production configuration must be stated separately. In the current `CNN/main.cpp`, `make_single_trial` uses the smaller dense head $(128, 64, 1)$, Adam at the source-default learning rate 10^{-3}, physics weight 0.10, global batch size 257, 200 epochs, gradient clipping at 1, and no L1 or L2 penalty. The experiment README calls 0.25 a production default, but the source file is authoritative for the current executable. The ordinary final run used the production path and therefore does not provide an independent test estimate for the selected $\lambda = 0$. The documented physics result supports removing the SIMM term under the

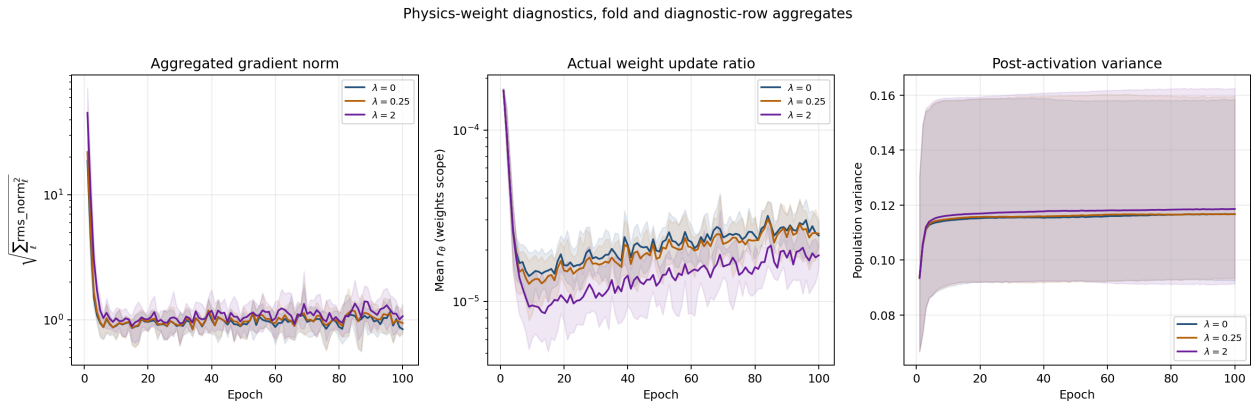


Figure 22: Layer-agnostic diagnostics for the recorded physics-weight sweep. Curves are means over folds and comparable diagnostic rows, with minimum to maximum fold bands. The gradient panel uses the square root of the sum of squared per-layer RMS norms for the all-parameter scope, the update panel uses the actual weights-scope update ratio, and the variance panel uses post-activation population variance. No layer index is shown.

recorded low-rate, 100-epoch sweep, but a rerun is required before extending that conclusion from the archived small-head measurements to the current large-head source recipe.

4.6. Weight regularization

Weight regularization was tested because the dense head contains millions of trainable weights, so reducing unnecessary parameter magnitude could in principle improve validation performance. L1 regularization is associated with sparsity and L2 regularization with continuous shrinkage Tibshirani (1996); Hoerl and Kennard (1970). These expectations motivate the experiment, but they do not determine the result: a penalty can also slow optimization or introduce bias when the unregularized model is not overfitting within the available training budget.

For every convolutional and dense weight tensor W_ℓ, the implementation defines the two penalties as

$$R_{\mathrm{L1}} = \lambda_1 \sum_{\ell} \sum_{w \in W_\ell} |w|, \qquad R_{\mathrm{L2}} = \lambda_2 \sum_{\ell} \sum_{w \in W_\ell} w^2. \tag{60}$$

Biases are excluded, the sums are not normalized by parameter count or batch size, and the two coefficients were varied independently. The corresponding gradient contribution is

$$\frac{\partial (R_{\mathrm{L1}} + R_{\mathrm{L2}})}{\partial w} = \lambda_1 \operatorname{sign}(w) + 2\lambda_2 w, \qquad \operatorname{sign}(0) = 0. \tag{61}$$

The trainer adds this contribution after synchronized MPI gradient averaging and before element-wise clipping and the Adam update. It is therefore different from Adam weight decay. The recorded `train_objective` remains the SIMM objective; the final L1 and L2 penalty values are reported separately.

The current experiment source fixes `conv5x5-dense-1024-512-256-128`, LeakyReLU with $\alpha = 0.05$, Adam at 10^{-5}, physics weight 0.25, global batch size 64, seed 42, gradient clipping at 1, and 100 epochs. It evaluates eight L1 values, $\{0, 6.75 \times 10^{-7}, 2.13 \times 10^{-6}, 6.75 \times 10^{-6}, 2.13 \times 10^{-5}, 6.75 \times 10^{-5}, 2.13 \times 10^{-4}, 6.75 \times 10^{-4}\}$, and eight L2 values, $\{0, 10^{-4}, 3.16 \times 10^{-4}, 10^{-3}, 3.16 \times 10^{-3}, 10^{-2}, 3.16 \times 10^{-2}, 10^{-1}\}$. Each axis is a separate one-dimensional sweep with zero as its own reference, giving 16 candidates and 80 fold evaluations. The test dataset was not part of the search.

The numerical provenance differs from the physics sweep. The current result directory does not contain the raw regularization CSVs or metadata. The complete candidate values and aggregate results below are the documented Reference Results in `CNN/experiments/regularization_tuning/README.md`, which identifies a large-topology CINECA run. Raw CSVs retained in an older Git revision belong to the earlier $(128, 64)$ head and are not merged with these values. The tables should therefore be read as a documented historical result, not as a newly recomputed sample-weighted summary. This limitation does not change the direction of the documented ranking, but it prevents an independent reconstruction of every fold statistic from the current worktree.

λ_1	Mean validation MSE	Fold standard deviation	Δ_{λ_1}	Improved folds
0	0.004419	0.000794	0	–
6.75×10^{-7}	0.004424	0.000788	+0.0000053	2/5
2.13×10^{-6}	0.004436	0.000800	+0.0000178	2/5
6.75×10^{-6}	0.004478	0.000811	+0.0000588	0/5
2.13×10^{-5}	0.004540	0.000832	+0.0001209	0/5
6.75×10^{-5}	0.004684	0.000902	+0.0002656	0/5
2.13×10^{-4}	0.005100	0.000950	+0.0006810	0/5
6.75×10^{-4}	0.006462	0.001189	+0.0020431	0/5

Table 9: Documented L1 reference sweep on the large topology. Values are transcribed from the experiment README because the corresponding raw result tree is absent. Δ_{λ_1} is the documented candidate minus reference mean validation MSE.

In both documented axes the zero-penalty candidate had the lowest observed validation MSE. The nearest L1 candidates were only 5.3×10^{-6} and 1.78×10^{-5} worse in the reported mean and improved in two folds, so neither meets the four-fold paired criterion or exceeds its own paired spread. The nearest L2 candidate, 10^{-4}, improved in one fold and was worse by 6.44×10^{-5} in the reported mean. The degradation then increased across the tested grids. The penalties were active rather than numerically inert: the documented $\sum w^2$ decreased from approximately 2993 at zero penalty to 587 for the largest L1 value and 582 for the largest L2 value, while $\|w - w_0\|$ increased from about 0.91 to 41.18 and 36.46, respectively. At the same time, the validation minus training gap decreased, but the training MSE rose faster, indicating that shrinkage restricted the fit more than it corrected overfitting in this fixed 100-epoch regime.

The recorded diagnostic summaries, also preserved only in the README, are consistent with this interpretation. The largest gradient norm is reported as approximately 22.54 across the L1 axis and most of the L2 axis, with only the strongest L2 values increasing it to 24.08. The reported post-activation variance changes from 0.1441

λ_2	Mean validation MSE	Fold standard deviation	Δ_{λ_2}	Improved folds
0	0.004419	0.000794	0	–
10^{-4}	0.004483	0.000819	+0.0000644	1/5
3.16×10^{-4}	0.004580	0.000859	+0.0001613	0/5
10^{-3}	0.004768	0.000932	+0.0003489	0/5
3.16×10^{-3}	0.005092	0.000992	+0.0006730	0/5
10^{-2}	0.005902	0.001100	+0.0014836	0/5
3.16×10^{-2}	0.006994	0.001217	+0.0025749	0/5
10^{-1}	0.008453	0.001080	+0.0040345	0/5

Table 10: Documented L2 reference sweep on the large topology. Values are transcribed from the experiment README because the corresponding raw result tree is absent. Δ_{λ_2} is the documented candidate minus reference mean validation MSE.

at zero L1 to 0.1654 at the largest L1 and from 0.1441 at zero L2 to 0.1686 at the largest L2. Because the underlying diagnostic CSVs are absent, these values are not used to make a layer-wise stability claim. No new regularization diagnostic plot is included; the tables and provenance statement are the reproducible evidence that remains in the current repository.

The documented cross-validation winner is therefore $\lambda_1 = 0$ and $\lambda_2 = 0$, with reported mean physical validation MSE 0.004419 ± 0.000794. The current production source also sets both penalties to zero, so the direction of this choice agrees with `CNN/main.cpp`. The production run nevertheless differs in head width, learning rate, physics weight, batch size, and epoch or early-stopping policy, and no regularization-specific test estimate exists. To conclude, the retained evidence supports not adding L1 or L2 at Adam learning rate 10^{-5} and 100 epochs. A new diagnostics-enabled run would be required to attach stronger numerical stability claims or to test whether the answer changes after the production learning rate and training budget are used.

4.7. Activation-function tuning

The activation function determines the nonlinear transformation applied after an affine layer and therefore affects both the signal passed to the next layer and the gradient returned during backpropagation. This choice is particularly relevant for the selected topology, which contains one activation after the convolution and four more in the dense head. We therefore compared bounded functions, a rectifier with an inactive negative branch, and LeakyReLU functions with three negative slopes. The purpose was not to assume that one family is universally better, but to measure which transformation gives the lowest physical validation error under the fixed architecture and training budget. Saturating nonlinearities can reduce the derivative when their input has large magnitude, whereas a rectifier retains a unit derivative on its positive branch; these are established mechanisms for changes in signal and gradient propagation Glorot and Bengio (2010).

$$
\begin{aligned}
f_{\mathrm{ReLU}}(x) &= \begin{cases} x, & x > 0, \\ 0, & x \le 0, \end{cases} \qquad f'_{\mathrm{ReLU}}(x) = \begin{cases} 1, & x > 0, \\ 0, & x \le 0, \end{cases} \\
f_{\alpha}(x) &= \begin{cases} x, & x > 0, \\ \alpha x, & x \le 0, \end{cases} \qquad f'_{\alpha}(x) = \begin{cases} 1, & x > 0, \\ \alpha, & x \le 0. \end{cases}
\end{aligned}
\tag{62}
$$

Here α is the fixed negative slope of a LeakyReLU layer. Thus, negative inputs are removed by ReLU and attenuated rather than removed by LeakyReLU. The code evaluates the non-positive branch also at $x = 0$, which fixes the derivative convention used in the recorded backward pass. The smooth alternatives were implemented as

$$
\begin{aligned}
f_{\mathrm{Tanh}}(x) &= \tanh(x), & f'_{\mathrm{Tanh}}(x) &= 1 - \tanh^2(x), \\
f_{\mathrm{Sigmoid}}(x) &= \frac{1}{1 + \exp(-x)}, & f'_{\mathrm{Sigmoid}}(x) &= f_{\mathrm{Sigmoid}}(x)\,(1 - f_{\mathrm{Sigmoid}}(x))\,.
\end{aligned}
\tag{63}
$$

Their outputs are bounded, and the derivatives become small in the corresponding saturation regions. The final dense layer remains linear, so these transformations are used only in the convolutional branch and in the four hidden dense layers.

The fixed topology was `conv5x5-dense-1024-512-256-128`. More precisely, the 150×150 SDF was passed through `Conv2DLayer(1,8,5, stride=5, padding=0)`, which produces $8 \times 30 \times 30 = 7200$ features. The two scalar inputs are then concatenated, so the dense sequence is `DenseLayer(7202,1024)`, `DenseLayer(1024,512)`, `DenseLayer(512,256)`, `DenseLayer(256,128)`, and a final `DenseLayer(128,1)`. The same candidate activation was used after the convolution and after each of the four hidden dense layers. This detail matters because the comparison changes the complete nonlinear path rather than one isolated layer.

The search used the six candidates `tanh`, `sigmoid`, `relu`, `leakyrelu-alpha-0.01`, `leakyrelu-alpha-0.05`, and `leakyrelu-alpha-0.1`. Adam was fixed at learning rate 10^{-5} with $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$, and zero optimizer weight decay. The SIMM loss used physics weight 0.1, while the L1 and L2 weights and dropout rate were zero. Each fold ran for 100 epochs with global batch size 64, element-wise gradient clipping at 1, shuffled samples, and no early stopping. The recorded run used five sample-level folds generated with shuffling and seed 42, and 64 MPI ranks. The validation sets contained 343, 343, 343, 342, and 342 samples, with complementary training sizes of 1370, 1370, 1370, 1371, and 1371. Each fold therefore processed 22 balanced global batches per epoch, or 2200 optimizer steps. The 30 candidate-fold fits consequently accounted for 66000 optimizer steps before the final refit. The validation and public history interval was ten epochs, and the diagnostics used 64 fixed histogram bins over $[-10, 10]$ with out-of-range values placed in the edge bins. The effective learning rate in the diagnostics was constant at 9.9999997×10^{-6} and `learning_rate_steps.csv` contained only its header, so no schedule was present.

The fold plan was reused for all candidates, and normalization statistics were fitted on each fold's training indices only. A fresh model and a fresh Adam optimizer were constructed for every candidate and fold. The 1713-sample training NPZ was the only dataset used for candidate selection; the 429-sample test NPZ was loaded by the driver only after the best candidate had been identified and refitted on all training samples. The source revision recorded in the diagnostic metadata is `8aada70b492a`, whereas the current checkout is `fe71f0964ca4`; the numerical values below are therefore taken from the committed format-1 artifacts and are not silently reinterpreted using the newer diagnostic schema.

Selection used the physical-unit validation MSE defined in Equation (45). The official `cv_summary.csv` aggregates the fold errors with validation-sample weights, and `CrossValidator::tune` selects the smallest resulting mean. The standardized SIMM training objective is not substituted for this score and is not compared numerically with it. Moreover, the implementation selects on the mean validation MSE alone. Gradient, update, and activation diagnostics are used below as post hoc evidence about the numerical behaviour, not as an additional undocumented selection criterion. The complete ranking is reported in Table 11.

Activation	Mean validation MSE	Fold standard deviation
LeakyReLU, $\alpha = 0.05$	0.004427	0.000865
LeakyReLU, $\alpha = 0.01$	0.004448	0.000876
ReLU	0.004454	0.000876
LeakyReLU, $\alpha = 0.1$	0.004545	0.000966
Tanh	0.005466	0.001045
Sigmoid	0.008971	0.001250

Table 11: Complete activation-function comparison on the fixed large topology. Means and deviations are the sample-weighted physical validation MSE and weighted population standard deviation from `cv_summary.csv`; the ranking was performed over all six candidates.

The lowest observed mean was obtained by LeakyReLU with $\alpha = 0.05$, with physical validation MSE 0.0044274 ± 0.0008653. Its five fold values were 0.00447944, 0.00573559, 0.00399854, 0.00312070, and 0.00480010. The nearest candidate was $\alpha = 0.01$, whose mean was 0.0044477 ± 0.0008761, only 2.03×10^{-5} higher, or approximately 0.46% relative to the selected mean. ReLU followed at 0.0044543 ± 0.0008759. The selected candidate was best in folds 1, 3, and 4 using zero-based fold indices, while $\alpha = 0.01$ was best in fold 0 and ReLU in fold 2. Thus, the selection is a small aggregate advantage within the ReLU family rather than a uniform win in every split. The fold deviations are much larger than the difference between the first three means, so we refer to $\alpha = 0.05$ as the selected or lowest observed candidate, not as a statistically established optimum. No repeated seed was used.
The separation from the bounded alternatives was larger. Tanh reached $0.005465620520 \pm 0.001045446677$, about 23.4% above the selected mean, while Sigmoid reached $0.008971404356 \pm 0.001250088488$, about twice the selected mean. Figure 23 displays the official weighted means and deviations. All 30 candidate-fold records completed with finite metrics, and no candidate was removed from the ranking because of a failed or divergent run.

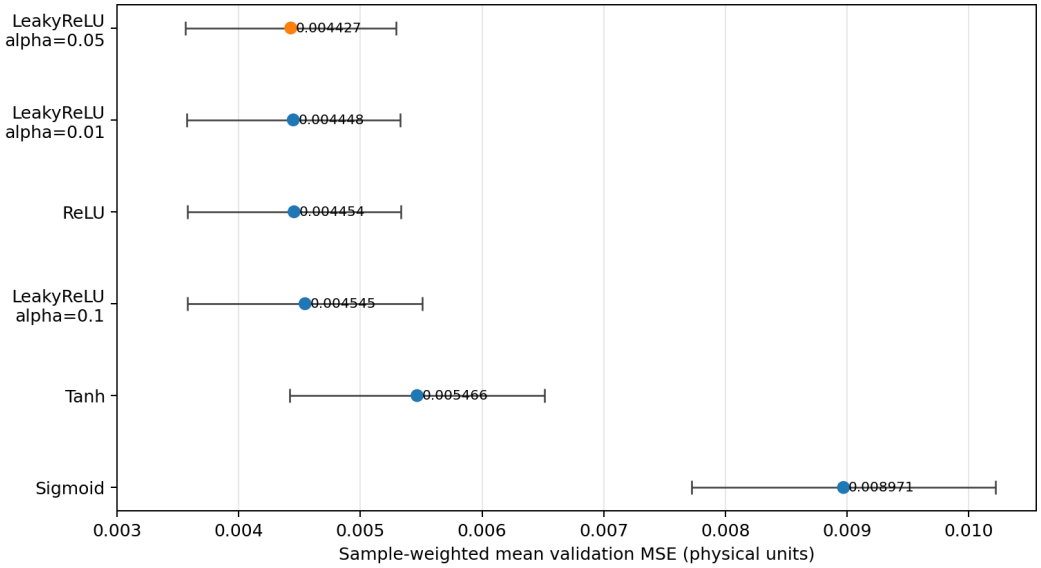


Figure 23: Complete activation-function ranking from the official `cv_summary.csv`. Points are sample-weighted mean physical validation MSE values and horizontal error bars are the corresponding weighted population standard deviations over the five folds. The orange point identifies the lowest observed candidate, LeakyReLU with $\alpha = 0.05$.

The checkpoint histories explain how the final separation developed. The fold-weighted validation MSE for the selected candidate decreased from 0.00655666 at epoch 10 to 0.00505364 at epoch 50 and 0.00442742 at epoch 100. The corresponding values for Tanh were 0.00801736, 0.00635208, and 0.00546562. Sigmoid was markedly slower at the beginning, decreasing from 0.410526 at epoch 10 to 0.0132361 at epoch 50 and 0.00897140 at epoch 100. Its training SIMM objective also fell from 0.888182 to 0.0272476 and then 0.0178482, whereas the selected candidate went from 0.0130666 to 0.0103440 and then 0.00891624. These objectives are standardized SIMM quantities and are kept separate from the physical validation curve. The candidate-level validation means were

lowest at the final recorded checkpoint, and none of the histories showed a late unbounded increase. Figure 24 shows the fold-weighted physical validation histories on a logarithmic vertical axis.

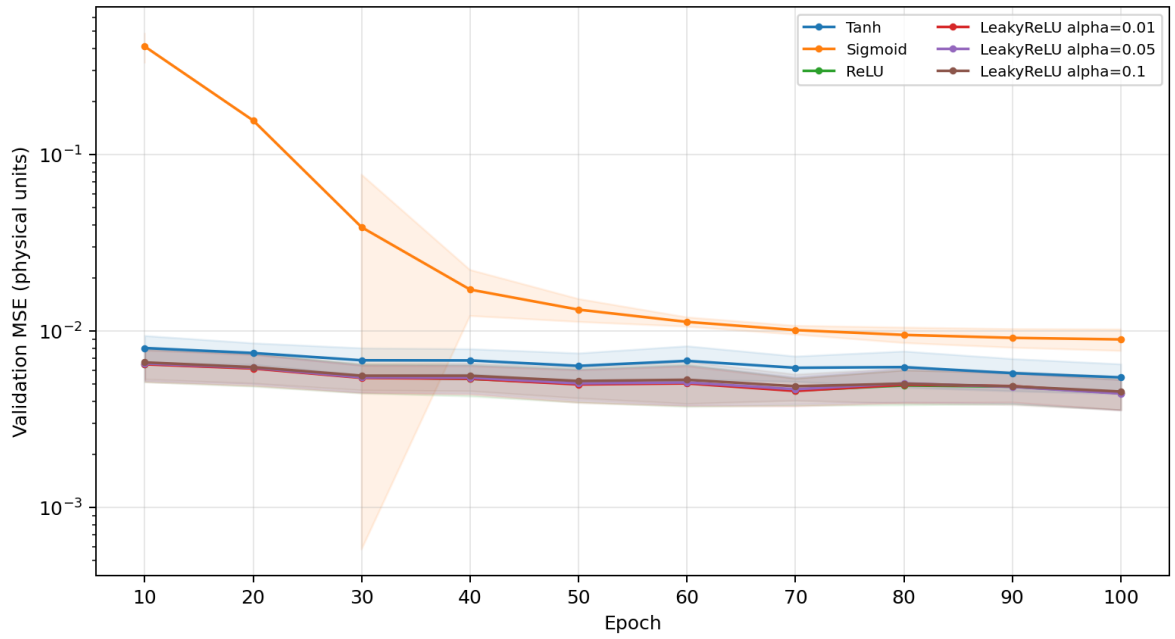


Figure 24: Fold-weighted physical validation MSE at the ten-epoch checkpoints for the six activation candidates. Shaded regions show the positive part of the weighted population standard deviation band over the five folds, and the vertical axis is logarithmic. The standardized SIMM training objective is not plotted on this axis.

The recorded diagnostics provide a numerical stability check, but they do not change the selection rule. For gradient flow, we use rows with `parameter_scope=all` and the stored `maximum_norm` for peak checks. The largest value for the selected candidate was 20.1379 at epoch 1, zero-based fold 2, and stable model layer 4, the first dense layer. The corresponding peaks were 20.8124 for ReLU, 20.6424 for $\alpha = 0.01$, and 20.7899 for $\alpha = 0.1$. Tanh was the numerical outlier with a peak of 111.508 at epoch 1, zero-based fold 1, also at layer 4. Sigmoid had a lower peak of 18.2200, but this does not imply better optimization because its early validation curve was much worse. All of these maxima occurred at the first epoch rather than late in training, and the selected candidate's fold-averaged RMS gradient norms at stable trainable layer indices $(0, 4, 6, 8, 10, 12)$ decreased from $(1.596, 7.795, 3.352, 2.284, 1.619, 0.938)$ at epoch 1 to $(0.156, 0.640, 0.302, 0.242, 0.198, 0.133)$ at epoch 100. This pattern is consistent with an initial transient followed by settling, not with a late exploding gradient.
For parameter motion, we use `parameter_scope=weights` and the actual `mean_ratio` in Equation (46). The selected candidate reached a maximum weight update ratio of 4.50124×10^{-4} at epoch 1, zero-based fold 3, layer 4. The associated mean pre-update norm was 42.3385 and the mean update norm was 0.0190575, with no near-zero denominator event. The neighbouring LeakyReLU candidates had comparable maxima, 4.47063×10^{-4} for $\alpha = 0.01$ and 4.42262×10^{-4} for $\alpha = 0.1$, while the largest value over the grid was 6.15050×10^{-4} for Sigmoid. Each run also contains six epoch-1 events in the bias-only rows because the denominator is protected by 10^{-12} while the initially zero biases have a very small norm. The corresponding weight and combined-layer rows have no such events. We therefore do not interpret the large bias-only ratios as unstable parameter movement. The small weight ratios, together with the absence of non-finite diagnostics and late gradient growth, support the conclusion that the selected score was not produced by an anomalously large update.
The activation statistics show the transformation expected from the equations. For the selected LeakyReLU, the post-activation population variances were pooled over folds from the recorded population moments as follows:

epoch	layer 1	layer 5	layer 7	layer 9	layer 11
1	0.109805	0.092960	0.065048	0.045128	0.030745
50	0.109755	0.102640	0.084950	0.067045	0.051060
100	0.108244	0.100116	0.082939	0.065922	0.050486

(64)

The variance decreases through the dense head, but it does not collapse to zero and changes little between epochs 50 and 100. The full selected-candidate diagnostic range was approximately $[-4.003, 4.035]$ before activation and $[-0.200, 4.035]$ after activation, which is consistent with retaining a scaled negative branch. The corresponding ReLU post-activation minimum was exactly zero. These observations support controlled negative attenuation

and a stable signal scale for the selected run, although they do not establish that this mechanism alone caused its small validation advantage.

The bounded candidates show the clearest saturation-related evidence. Across the Tanh diagnostics, post-activation values reached -0.999355 and 0.999393, so some values were close to the limits where its derivative is small; this candidate also had the large early gradient maximum described above. The only activation values outside the fixed histogram interval $[-10, 10]$ occurred for Sigmoid, at zero-based fold 4, layer 5, from epochs 78 to 100. At epoch 100 that layer had pre-activation minimum -10.9256, maximum 10.6715, and variance 7.78885, while the post-activation range over the complete Sigmoid grid was approximately $[1.8 \times 10^{-5}, 0.999977]$. This behaviour is consistent with saturation and with the slow early convergence of Sigmoid, but the diagnostics do not prove a causal explanation for the validation ranking. Histogram edge counts were treated only as evidence of values reaching the configured limits, because out-of-range values are clamped into the edge bins. Figure 26 combines the peak checks with the selected-candidate depth profiles; the gradient and update panels aggregate maxima over folds, layers, and epochs, while the lower panels identify their fold and population-moment aggregation.

The vanishing-gradient risk is most directly assessed from the distribution of the pre-activation value z, rather than from the post-activation value alone. If $a_j = f_j(z_j)$, the chain rule inserts one derivative factor at every nonlinear layer. Ignoring the separate linear weight transformations, the activation contribution along a path through k nonlinearities is

$$\frac{\partial \mathcal{L}}{\partial z_j} = \frac{\partial \mathcal{L}}{\partial a_j} f_j'(z_j), \qquad G_{\mathrm{act}}(k) = \prod_{j=1}^{k} \left| f_j'(z_j) \right|. \tag{65}$$

When the actual values z_j fall in a plateau, G_{act} becomes small even when the upstream loss gradient is non-zero. A broad or heavy-tailed pre-activation distribution therefore raises the fraction of samples and paths that receive weak local gradients; it does not imply that every path vanishes, because values near the central high-slope region can still carry a useful signal.

The scale of this effect is substantial for the bounded functions. For Sigmoid, $|f'(5)| \simeq 6.65 \times 10^{-3}$ and $|f'(10)| \simeq 4.54 \times 10^{-5}$. For Tanh, the corresponding values are approximately 1.82×10^{-4} and 8.24×10^{-9}. If five consecutive hidden nonlinearities were simultaneously evaluated at $|z| = 5$, the activation-only factors would already be approximately $(6.65 \times 10^{-3})^5 \simeq 1.3 \times 10^{-11}$ for Sigmoid and $(1.82 \times 10^{-4})^5 \simeq 2.0 \times 10^{-19}$ for Tanh. These products are mechanism illustrations, not measured end-to-end gradients: the actual network also contains weight matrices, and different layers need not lie in the same plateau. They nevertheless show why a widespread distribution can create a vanishing-gradient bottleneck through depth.

The rectifiers have a different but equally important failure mode. ReLU has an exact zero derivative for $z \leq 0$, so a unit whose inputs remain on the negative branch contributes no local gradient through that activation. LeakyReLU replaces this strict plateau with a constant negative-branch slope α. This avoids an exactly dead path, but it does not eliminate attenuation: k consecutive negative-branch traversals contribute α^k, so the tested $\alpha = 0.05$ can still multiply the signal by 0.05^k, while $\alpha = 0.01$ attenuates it more strongly and $\alpha = 0.1$ less strongly. Thus, "non-saturating" should not be interpreted as "immune to vanishing gradients"; the relevant question is how much of the observed pre-activation distribution lies in each branch and how those branches are arranged across depth.

Figure 25 makes this distributional argument explicit using the actual recorded values. It shows the empirical pre-activation histograms at layer 5, the first dense nonlinearity adjacent to the layer where the largest early gradient peaks were observed, pooled over all five folds at epochs 1 and 100. The black curve is the exact absolute derivative of the corresponding activation, plotted on the right axis, and the shaded area marks either $|f'(z)| \leq 0.01$ for Tanh and Sigmoid or the negative branch for the rectifiers. At epoch 100, approximately 4.0% of the Sigmoid layer-5 inputs lie in the displayed low-derivative tails, whereas approximately half of the inputs lie on the zero-slope ReLU branch or on the constant low-slope LeakyReLU branch; the selected $\alpha = 0.05$ run has approximately 49.8% negative inputs at this layer. The Tanh layer-5 distribution has no visible mass beyond the 0.01 derivative threshold at epoch 100, which does not contradict the rare near-limit values found elsewhere in the complete diagnostic grid. The percentages are approximate histogram masses, not per-unit gradient measurements, and the fixed edge bins include values outside $[-10, 10]$. The figure therefore demonstrates the available mechanism and its plausibility, while the recorded layer-aggregate gradient norms remain necessary to determine whether it produced harmful end-to-end gradient loss in this experiment.

The final refit of the selected activation used all 1713 training samples and retained the large search topology. Its training SIMM objective decreased from 0.286768 at epoch 1 to 0.00865917 at epoch 100; the interval was set to 101, so no intermediate test values were written to the final diagnostic history. The summary reports a physical MSE of 0.00273892 on the untouched 429-sample test set after selection. This is a post-selection estimate for the refitted activation-search model and must not be presented as the test performance of the ordinary production model.

The **cross-validation winner** is therefore `LeakyReLU` with $\alpha = 0.05$, with physical validation MSE $0.00442742 \pm 0.00086526$ on the fixed large topology. The **production configuration** in `CNN/main.cpp` uses the same

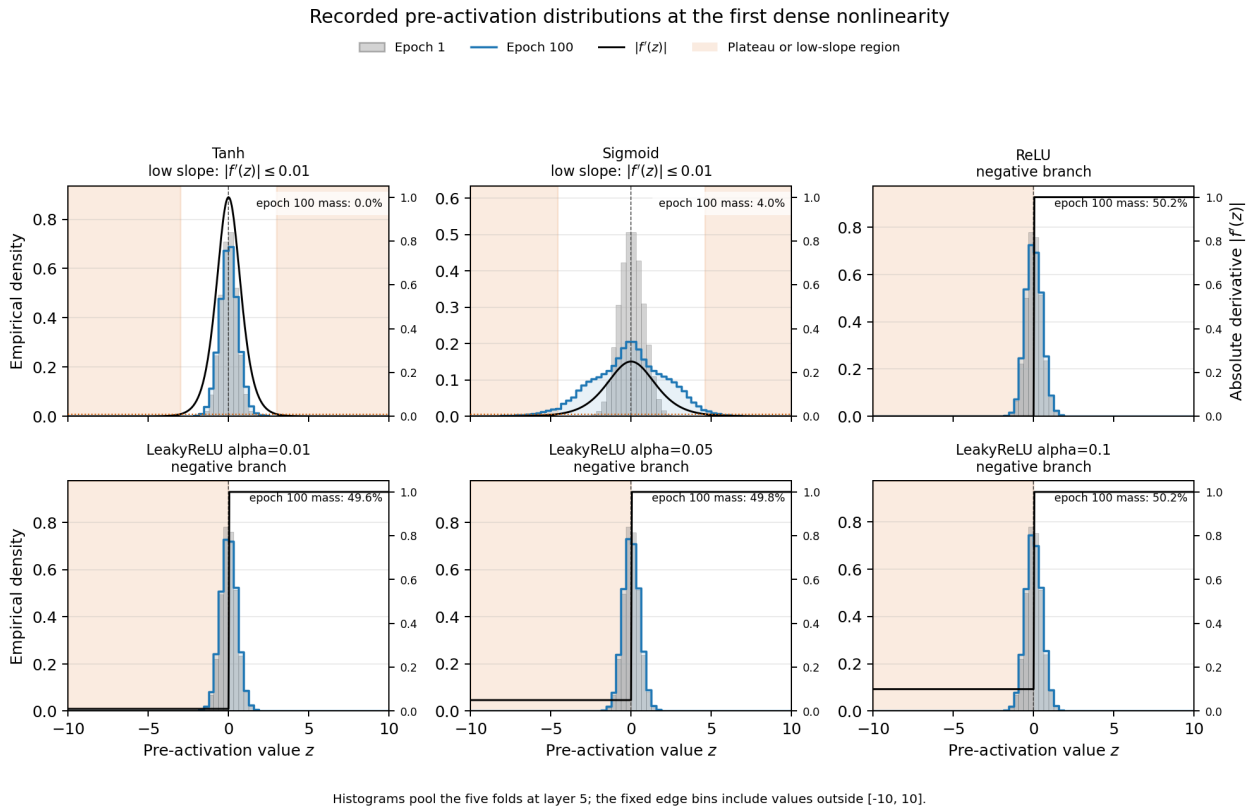


Figure 25: Empirical distributions of the actual pre-activation values z at the first dense nonlinearity (stable model layer 5) for the six activation candidates. Gray bars show epoch 1 and blue step curves show epoch 100, with histograms pooled over the five folds. The black curves show the exact absolute derivative $|f'(z)|$ on the right-hand scale. Orange shading identifies the Tanh and Sigmoid regions with $|f'(z)| \leq 0.01$ and the negative branch for ReLU and LeakyReLU. The epoch-100 mass annotations are approximate because the committed diagnostics store fixed-bin counts; the edge bins include values outside the displayed $[-10, 10]$ range.

activation choice at its source defaults, namely `LeakyReLU` with $\alpha = 0.05$, but it uses the smaller dense head $(128, 64, 1)$, Adam learning rate 10^{-3}, physics weight 0.10, global batch size 257, 200 nominal epochs, no dropout, no L1 or L2 penalty, gradient clipping at 1, and ordinary-training early stopping with a 10 percent holdout. Command-line overrides can change these defaults. Thus, the activation decision agrees between the search and production source, while the activation experiment does not validate the complete production configuration because its topology, learning rate, batch size, and stopping policy differ. To conclude, we retain $\alpha = 0.05$ as the selected production activation because it had the lowest observed sample-weighted validation MSE and diagnostics comparable to the neighbouring rectifiers. The small margin over $\alpha = 0.01$ and ReLU, the single seed, and the sample-level rather than airfoil-level folds limit the strength of this conclusion beyond the recorded experiment.

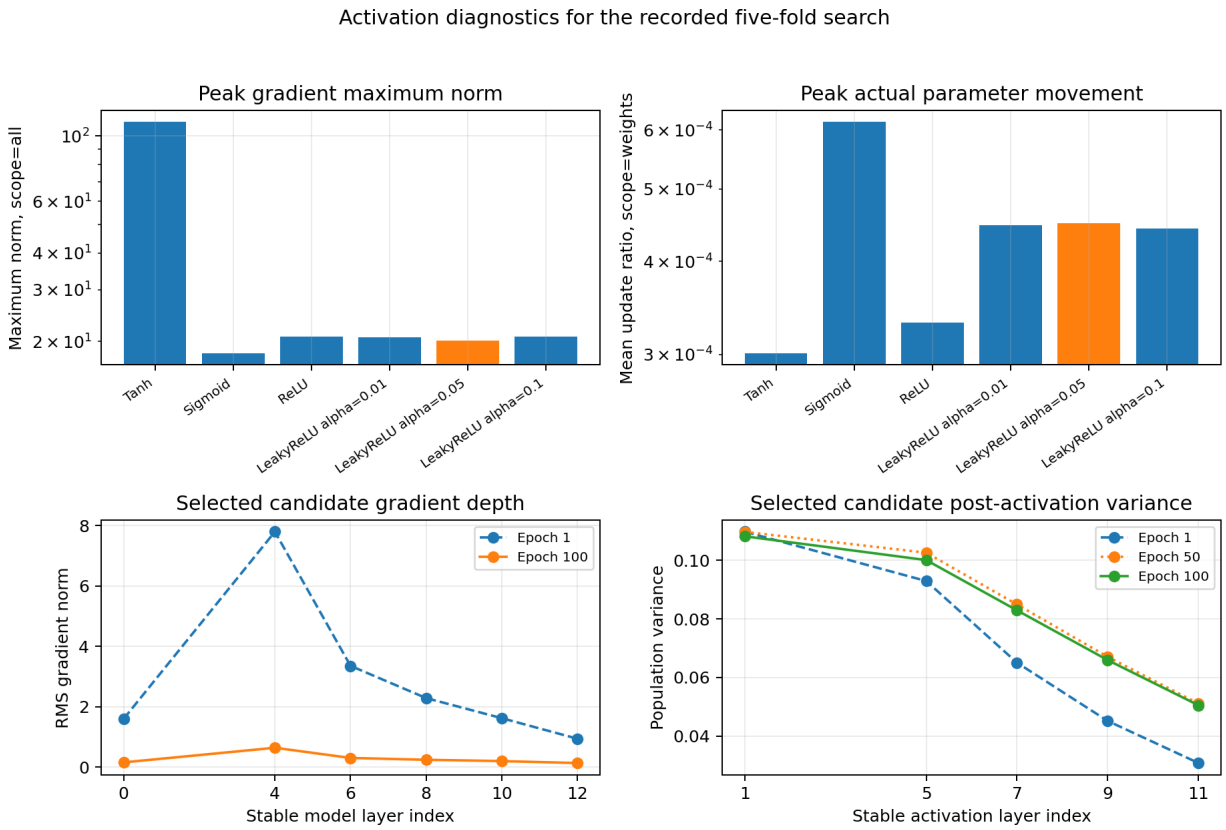


Figure 26: Diagnostics for the activation-function search. The upper panels show, for each candidate, the maximum gradient `maximum_norm` with `parameter_scope=all` and the maximum actual weight update ratio with `parameter_scope=weights`, taken over all five folds, layers, and epochs. The lower left panel shows fold-averaged RMS gradient norms for the selected candidate at epochs 1 and 100 over stable trainable layer indices. The lower right panel shows post-activation population variances for the selected candidate at epochs 1, 50, and 100, pooled over folds from the recorded population moments.

4.8. Learning-rate and schedule tuning

After choosing the topology, optimizer, loss settings, and activation, we studied the learning rate because it controls the scale and duration of the parameter updates. In Adam, the learning rate multiplies the bias-corrected adaptive direction in Equation (55); it is therefore not removed by the coordinate-wise denominator, even though that denominator adapts to the history of each parameter. A rate that is too small can leave a finite epoch budget underused, whereas a rate that is too large can amplify the interaction between the SIMM residual, the dense layers, and the optimizer moments. The schedule determines how long each scale is applied. Cosine decay has been studied as part of the SGDR family (Loshchilov and Hutter, 2017), while warm-up was introduced to address early optimization difficulties in large-minibatch stochastic-gradient training (Goyal et al., 2017). Those references motivate the schedule families tested here, but they do not predict the behaviour of this Adam run: the present experiment uses no warm restarts, a different optimizer, and a different data set. Let $e \in \{0, \dots, E-1\}$ denote the zero-based epoch index and let E be the total number of epochs. The constant candidates use

$$\eta_e = \eta_0, \tag{66}$$

for every epoch. For step decay, the implementation uses

$$\eta_e = \eta_0 \gamma^{\lfloor e/S \rfloor}, \tag{67}$$

where S is the step interval and γ is the multiplicative factor. The cosine candidate uses

$$\eta_e = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min}) \left[1 + \cos\left(\frac{e}{E-1} \pi \right) \right], \tag{68}$$

which reaches $\eta_{\max}$ at $e = 0$ and $\eta_{\min}$ at $e = E - 1$. The warm-up candidate is defined by

$$\eta_e = \begin{cases} \eta_{\text{start}} + \dfrac{e}{W}(\eta_{\text{peak}} - \eta_{\text{start}}), & 0 \le e < W, \\ \eta_{\min} + \dfrac{1}{2}(\eta_{\text{peak}} - \eta_{\min}) \left[1 + \cos\left(\dfrac{e - W}{E - W - 1}\pi\right)\right], & W \le e < E, \end{cases} \tag{69}$$

where W is the number of warm-up epochs. In the recorded grid, $E = 100$, $W = 5$, $\eta_{\text{start}} = \eta_{\min} = 10^{-5}$, and $\eta_{\text{peak}} = 10^{-1}$. Thus, the warm-up branch returns 10^{-5} at the first completed epoch and reaches 10^{-1} at completed epoch 6. The step schedules change at zero-based indices $e = 30, 60, 90$ for $S = 30$, and at $e = 15, 30, 45, 60, 75, 90$ for $S = 15$, which correspond to completed epochs 31, 61, 91 and 16, 31, 46, 61, 76, 91, respectively. The configured optimizer rate and the scheduler output are distinct for the warm-up candidate: `epoch_metrics.csv` records the former in `configured_learning_rate` and the latter in `effective_learning_rate`. The latter is the rate used in the plots below.

The C++ driver evaluated the eleven candidates listed in Table 12. The fixed model was the previously selected topology with dense widths $(1024, 512, 256, 128)$, a 5×5 convolution producing eight channels, LeakyReLU with $\alpha = 0.05$, Adam with $\beta_1 = 0.9$, $\beta_2 = 0.999$, and $\varepsilon = 10^{-8}$, SIMM physics weight 0.25, zero L1 and L2 weights, global batch size 64, 100 epochs, element-wise gradient clipping at 1, and seed 42. The five folds were generated once with shuffled `RandomKFold`; the validation sets contained 343, 343, 343, 342, and 342 samples. Normalization was fitted on each fold's training subset and then applied to that fold's validation subset. Every candidate and fold constructed a fresh model and optimizer. Each fold processed 22 global batches per epoch, or 2200 optimizer steps, giving 121000 steps over the 55 candidate-fold fits. The diagnostic run recorded the training objective and effective rate at every completed epoch, while the physical validation MSE was checkpointed every ten epochs. Candidate selection used the sample-weighted physical-unit validation MSE, not the standardized SIMM objective, and the test file was not loaded during cross-validation.

Candidate	Configuration	Mean validation MSE ± SD	Fold range
`const_1e-3`	Constant, $\eta_0 = 10^{-3}$	0.003609 ± 0.001335	0.002092 to 0.006094
`step_s15_1e-3`	Step, $S = 15$, $\gamma = 0.5$, $\eta_0 = 10^{-3}$	0.004005 ± 0.000704	0.002826 to 0.004779
`step_s30_1e-3`	Step, $S = 30$, $\gamma = 0.1$, $\eta_0 = 10^{-3}$	0.004072 ± 0.000785	0.002865 to 0.004955
`const_1e-5`	Constant, $\eta_0 = 10^{-5}$	0.004419 ± 0.000790	0.002880 to 0.005001
`step_s30_1e-5`	Step, $S = 30$, $\gamma = 0.1$, $\eta_0 = 10^{-5}$	0.005034 ± 0.000901	0.003434 to 0.005786
`step_s15_1e-5`	Step, $S = 15$, $\gamma = 0.5$, $\eta_0 = 10^{-5}$	0.005094 ± 0.000905	0.003484 to 0.005861
`cosine_1e-1`	Cosine, $\eta_{\max} = 10^{-1}$, $\eta_{\min} = 10^{-5}$	$5.577 \times 10^8 \pm 2.251 \times 10^8$	2.087×10^8 to 7.949×10^8
`warmup_cosine_1e-1`	Warm-up and cosine, $\eta_{\text{peak}} = 10^{-1}$	$9.047 \times 10^8 \pm 8.866 \times 10^8$	1.207×10^8 to 2.507×10^9
`step_s15_1e-1`	Step, $S = 15$, $\gamma = 0.5$, $\eta_0 = 10^{-1}$	$5.010 \times 10^9 \pm 2.956 \times 10^9$	1.530×10^9 to 9.316×10^9
`step_s30_1e-1`	Step, $S = 30$, $\gamma = 0.1$, $\eta_0 = 10^{-1}$	$9.449 \times 10^9 \pm 4.726 \times 10^9$	3.197×10^9 to 1.631×10^{10}
`const_1e-1`	Constant, $\eta_0 = 10^{-1}$	$2.131 \times 10^{17} \pm 3.116 \times 10^{17}$	1.843×10^{16} to 8.313×10^{17}

Table 12: Complete ranking of the eleven learning-rate candidates. Means and deviations are the sample-weighted physical validation MSE and its weighted population standard deviation over the five folds, as recorded in `cv_summary.csv`; the final column gives the minimum and maximum fold scores.

The complete ranking is shown in Figure 27. The constant 10^{-3} candidate is the cross-validation winner with a mean physical validation MSE of 0.003608841294 and a weighted population standard deviation of 0.001334569865. Its five fold values, in zero-based fold order, are 0.002091737, 0.003001337, 0.003480483, 0.006093833, and 0.003383404. It is the best candidate in four of the five folds; the fourth fold is the visible high-error exception. Relative to the constant 10^{-5} baseline, the selected candidate reduces the weighted mean by approximately 18.3 percent. The nearest alternative is `step_s15_1e-3`, whose mean is 0.004005474214; its lower dispersion does not compensate for the higher aggregate validation error under the stated selection rule. The step candidates beginning at 10^{-3} remain in the same low-error scale, but their final effective rates are 1.5625×10^{-5} for $S = 15$ and 10^{-6} for $S = 30$. The results are consistent with the interpretation that these reductions leave less update magnitude late in the 100-epoch budget, but the experiment does not establish that mechanism causally. Starting at 10^{-5} and reducing further gives still smaller final rates, and both schedules perform worse than their constant baseline. Importantly, 10^{-5} is the lowest tested rate, so the selected 10^{-3} is the lowest observed validation-error setting, not the lowest rate or a universal stability threshold.

The convergence histories in Figure 28 separate the six candidates that remain in the low-error scale from the five candidates that reach 10^{-1}. The weighted validation curves for the low-rate group remain close to one another and finish between approximately 0.0036 and 0.0051, whereas the high-rate group remains many orders of magnitude above them even when its schedule later decreases. This separation is not an artefact of comparing the training objective with the validation error: the plotted curves use the physical validation column from the diagnostic files, and the SIMM training objective is kept separate. The curves also show why the small difference between the three 10^{-3} candidates should be described as a selection within this fold plan and budget rather than as a universal ordering of all possible schedules.

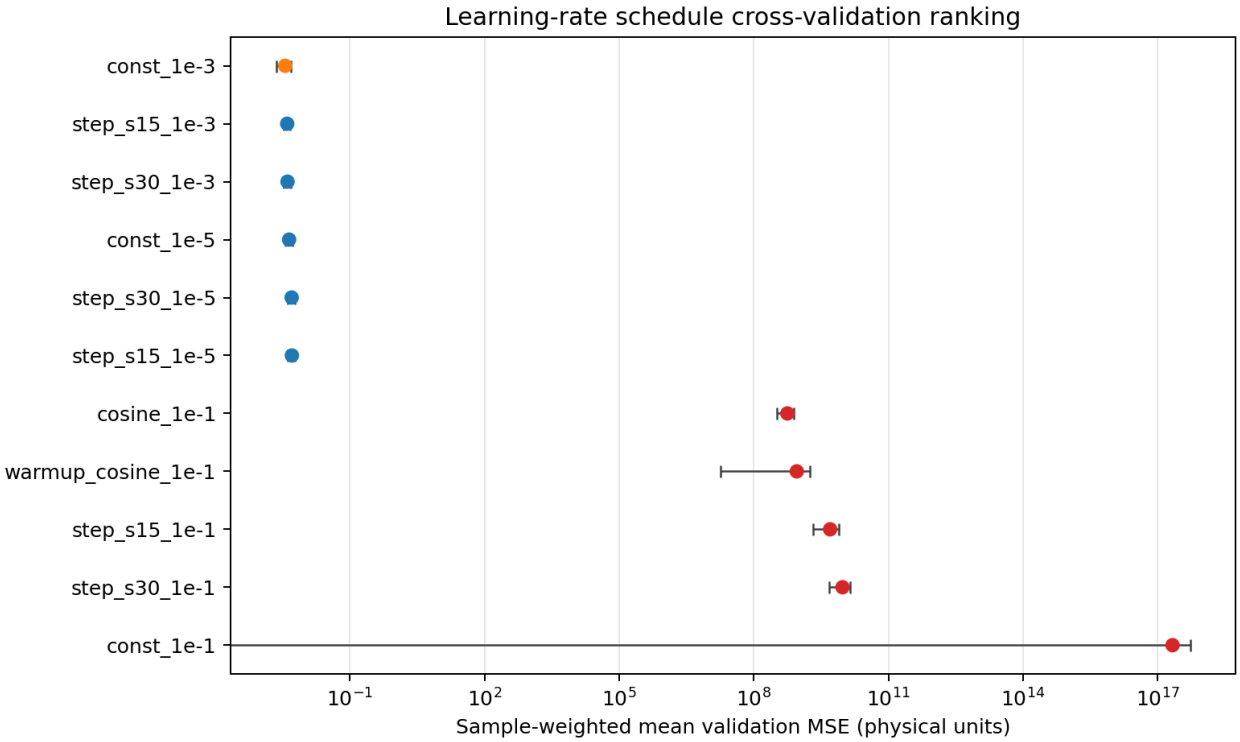


Figure 27: Complete learning-rate schedule ranking. Points show the sample-weighted mean physical validation MSE and horizontal bars show the weighted population standard deviation over five folds. The horizontal axis is logarithmic; orange identifies the selected constant 10^{-3} candidate and red identifies candidates whose rates reach 10^{-1}.

Figure 29 shows the effective rates returned by the scheduler, rather than only the base rate stored in the optimizer recipe. The constant candidate stays at 10^{-3}, while the $S = 15$ and $S = 30$ schedules reduce this value at the implementation-specific epoch boundaries given above. The cosine curve begins at 10^{-1} and reaches 10^{-5} only at the final epoch. The warm-up curve begins at 10^{-5}, reaches 0.020008 at completed epoch 2, reaches 0.1 at completed epoch 6, and then follows the cosine branch. This distinction matters for interpreting the warm-up result: the candidate spends most of the early optimization pass approaching the same high peak that is unstable in the other 10^{-1} candidates.

The high-rate candidates should be described as numerically unstable rather than as failed jobs. All 55 candidate-fold fits completed with finite diagnostic values and were included in the official ranking, but the five candidates that reach 10^{-1} have validation means from 5.577×10^{8} to 2.131×10^{17}. The diagnostic evidence explains this scale separation. For the selected constant 10^{-3} candidate, the maximum gradient norm over all folds, layers, and epochs, using `parameter_scope=all`, was 182.5645 at zero-based fold 2, epoch 1, and layer 0. The corresponding maximum over the final ten epochs was 3.1207, while the epoch-100 maximum was 1.9627. In the weight-only update rows, the largest actual mean update ratio was 0.0170216 at fold 2, epoch 1, and layer 4; the largest epoch-100 value was 0.0022612, and the largest value over the final ten epochs was 0.0024313. No near-zero denominator event occurred in the weight-scope rows used for this comparison. These are actual parameter movements, not the learning rate multiplied by a raw gradient. The gradient values are measured after synchronized aggregation and before the optimizer update, whereas the update ratios use the before and after parameter values.

The corresponding high-rate values are qualitatively different. The constant 10^{-1} candidate reaches a maximum gradient norm of 2.1467×10^{20} at zero-based fold 1, epoch 100, and layer 12. The cosine candidate reaches 2.8501×10^{19} at fold 3, epoch 7, and the warm-up candidate reaches 3.2478×10^{19} at fold 3, epoch 24. For the warm-up candidate in fold 0, the training objective increases from 0.3985 at completed epoch 1 to 3.7331×10^{8} at epoch 2, when the effective rate is 0.020008, and to 3.4329×10^{14} at epoch 6, when the peak rate is 0.1. The later reductions in the cosine and step schedules reduce the error scale relative to constant 10^{-1}, but do not return the runs to the low-error regime within 100 epochs. These observations support an instability interpretation at the tested high-rate scale; they do not provide a theoretical Lipschitz bound or a general threshold for this architecture.

The lower panels of Figure 30 provide a further check on the selected run. For `const_1e-3`, the fold-averaged RMS gradient at epoch 1 is between approximately 19.8 and 37.2 over the stable trainable layer indices, and

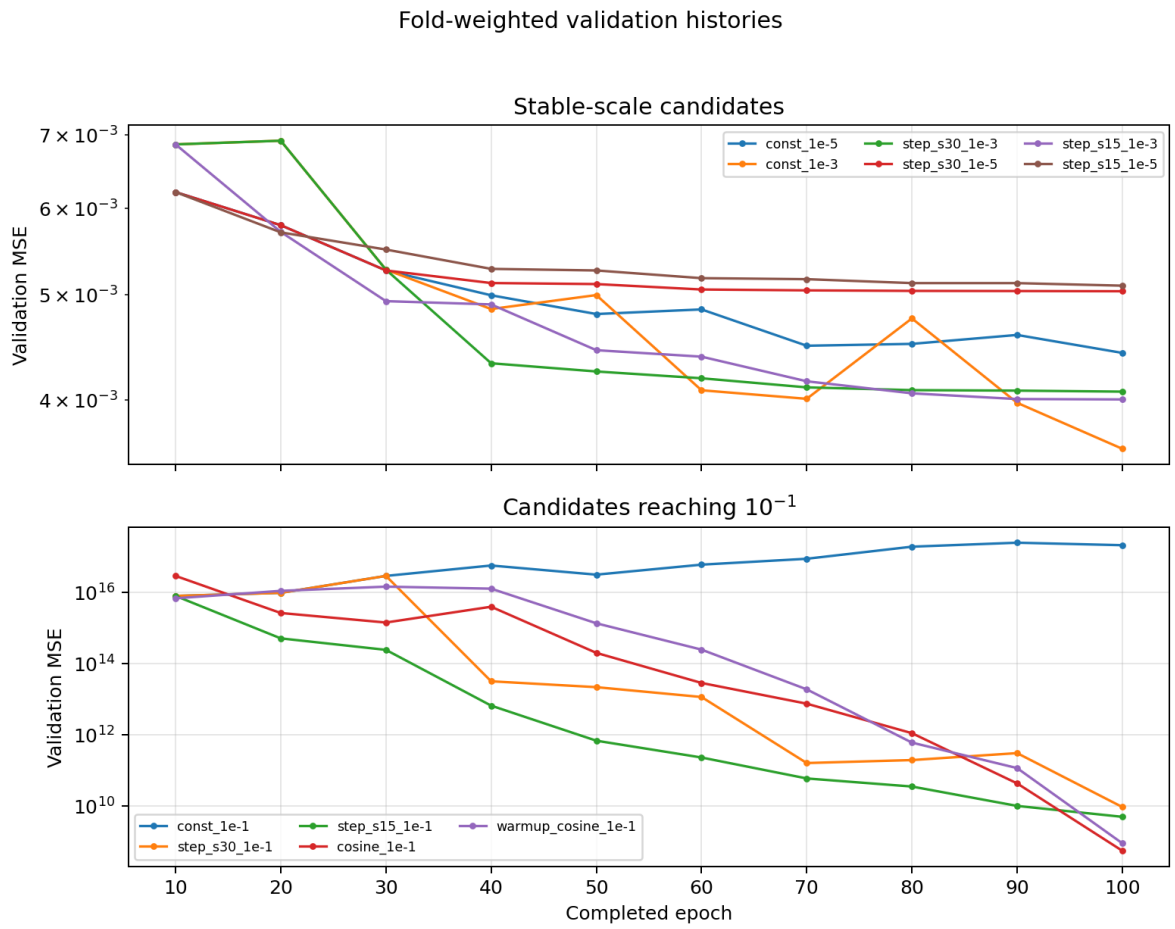


Figure 28: Sample-weighted validation histories at the ten-epoch checkpoints recorded for cross-validation. The upper panel contains the six candidates whose rates remain in the low-error scale, while the lower panel contains the five candidates that reach 10^{-1}. Both vertical axes are logarithmic and show physical-unit MSE.

at epoch 100 it is between approximately 0.0722 and 0.7793. The pooled post-activation variance at layer 5 decreases from 1.0967 at epoch 1 to 0.1053 at epoch 100, while the corresponding values at layers 7, 9, and 11 decrease from 1.7903, 1.7415, and 1.0981 to 0.1597, 0.1513, and 0.0969. The reductions occur together with decreasing validation error and gradient magnitude, so they are consistent with ordinary convergence and signal attenuation, but they do not by themselves establish harmful vanishing gradients. The diagnostic panels therefore support the performance ranking without replacing physical validation MSE as the selection criterion. After cross-validation, the tuning driver refitted `const_1e-3` on all 1713 training samples. The final metadata records the same large topology, batch size 64, physics weight 0.25, and 100-epoch budget, and reports a physical MSE of 0.003253844668 at completed epoch 100. This value agrees with the 0.00325384 value in `summary.txt`, but its provenance requires qualification. The final run passes `dataset/cnn_dataset_test.npz` as the validation data set and sets `validation_interval=1`, so the test MSE is written after every epoch rather than evaluated strictly once. Candidate selection was completed before this refit and no claim of early stopping from the test curve is made, but the recorded value should still be described as a post-selection refit estimate, not as a strictly untouched one-time test estimate.

The distinction from ordinary production training is also important. The **cross-validation winner** is the constant 10^{-3} candidate, with physical validation MSE $0.003608841294 \pm 0.001334569865$ on the fixed large-topology experiment. The **production configuration** constructed by the ordinary training path in `CNN/main.cpp` uses the same source-default Adam rate 10^{-3} and no attached scheduler, but it uses the smaller dense head $(128, 64, 1)$, physics weight 0.10, global batch size 257, nominal epoch count 200, gradient clipping at 1, seed 42, and early stopping on a 10 percent holdout. Thus, the selected learning-rate magnitude agrees with the ordinary source default, while the topology, loss weight, batch size, and stopping policy differ. The 0.00325384 refit value is consequently not a test score for the ordinary production model, and the learning-rate experiment does not validate the complete production recipe at once.

To conclude, within the fixed large topology, loss, fold plan, seed, and 100-epoch budget, the data support the

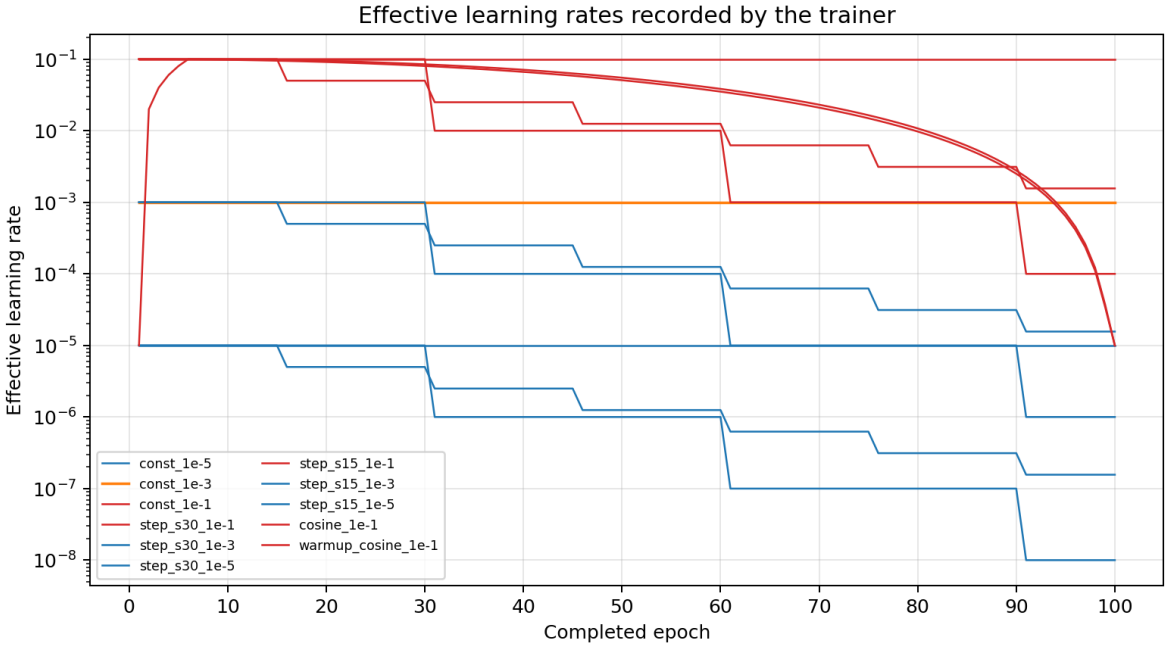


Figure 29: Effective learning rates from `epoch_metrics.csv` for zero-based fold 0, plotted against completed epoch. The scheduler settings are deterministic and identical across folds. The logarithmic vertical axis makes the constant, step, cosine, and warm-up branches comparable.

constant 10^{-3} schedule as the lowest observed validation-error configuration. The constant 10^{-5} baseline and its decayed variants are consistent with smaller rates underusing this budget, while the 10^{-1} group reaches a numerically unstable error scale even when cosine decay or a five-epoch warm-up is added. The result is a configuration choice for this experiment, not a universal stability limit. A repeated-seed study and an evaluation of the complete ordinary production recipe would be required to determine whether the small separation among the low-rate candidates persists beyond the recorded sample-level folds.

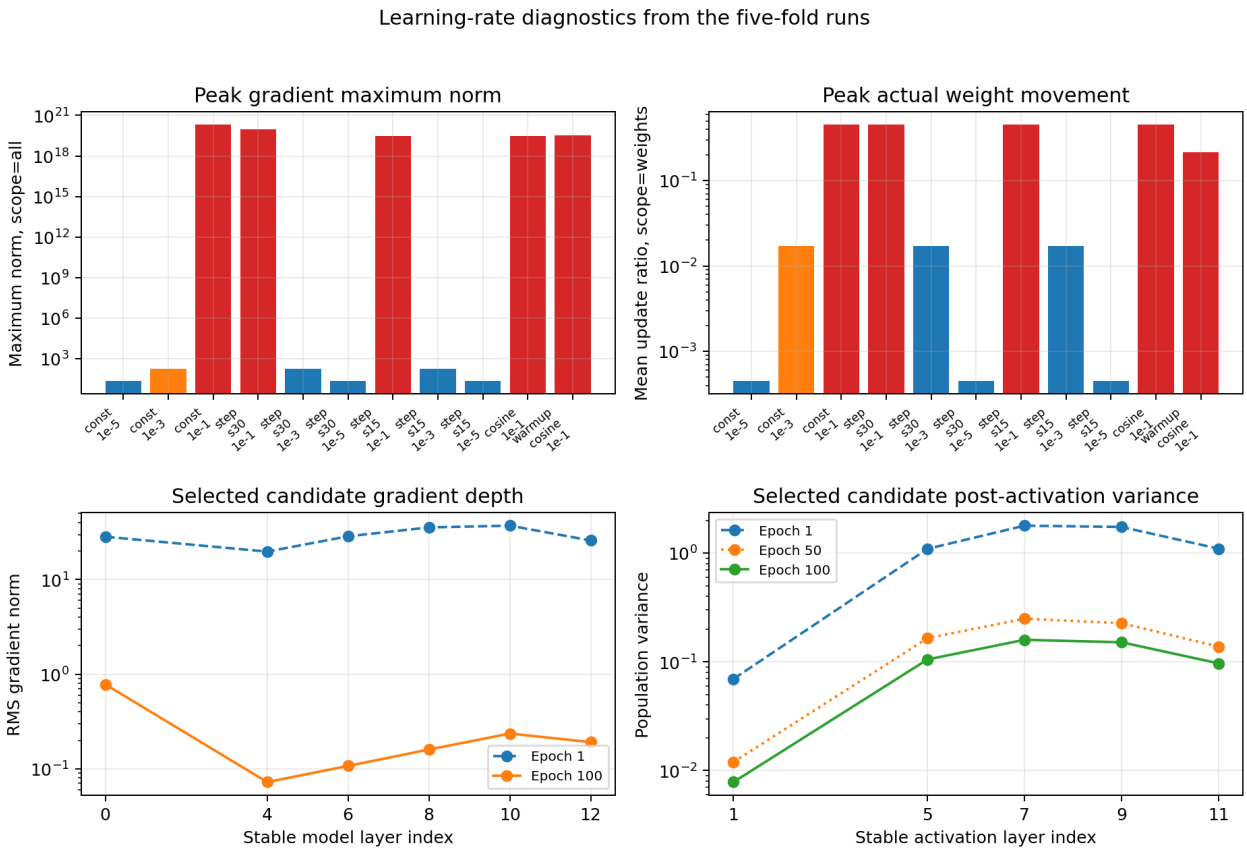


Figure 30: Learning-rate diagnostics from the complete five-fold run. The upper panels show the maximum over folds, layers, and epochs of `maximum_norm` with `parameter_scope=all` and of the actual `mean_ratio` with `parameter_scope=weights`. The lower-left panel shows the selected candidate's fold-averaged RMS gradient by stable model layer at epochs 1 and 100. The lower-right panel shows the selected candidate's pooled post-activation population variance at epochs 1, 50, and 100. All vertical axes are logarithmic.

4.9. Final production training

The final ordinary training was developed through a sequence of training trials rather than launched as a single fixed experiment. The first trial used a global batch size of 64 with the original range-based batching. Since the ordinary training split contains 1542 samples, this produced 24 batches of 64 samples and a final batch of only 6 samples. With 64 MPI ranks, that last update activated only six ranks, even though the gradients were still combined using the actual sample counts. The behaviour was changed first by balancing the batches, while keeping the requested size equal to 64, and then by increasing the global batch size to 257. The current implementation treats the configured size as an upper bound. If n is the number of training samples and B is this bound, it creates

$$K = \left\lceil \frac{n}{B} \right\rceil, \qquad q = \left\lfloor \frac{n}{K} \right\rfloor, \qquad r = n - Kq, \qquad n_j = \begin{cases} q+1, & j < r, \\ q, & j \geq r, \end{cases} \quad j = 0, \ldots, K-1, \tag{70}$$

where n_j is the size of global batch j. Equation (70) gives 17 batches of 62 and 8 batches of 61 for the balanced 64-sample experiment, while $B = 257$ produces six exact batches of 257. The former change removed the small tail without changing the 25 optimizer steps per epoch; the latter also reduced the number of updates per epoch to six. This distinction is important because the global batch size is not multiplied by the MPI rank count, and because the gradient synchronization in `CNN/src/model/CNNModel.cpp` weights each local gradient by its local sample count before dividing by the global count.

The main recorded outcomes of this sequence are collected in Table 13. All errors in the table are physical-unit MSE values. "Completed" is the number of epochs present in the diagnostic history, while "selected" is the epoch whose weights were retained for the reported metrics. The first four rows belong to the seed-42 development sequence. The three 20/20 rows are the comparable-step reruns with different seeds, and the last two rows show the longer seed-42 runs that led to the final production result.

Stage	B	Completed	Selected	Train MSE	Val. MSE	Test MSE
Initial tail, `trial 1`	64	200	195	0.002202	0.001889	0.001824
Balanced, `trial 2`	64	200	190	0.002408	0.001953	0.002030
Exact batches, `trial 3`	257	200	198	0.003572	0.003105	0.002636
Old stopping rule, `trial 4`	257	527	470	0.002064	0.002011	0.001924
20/20 policy, seed 0	257	834	743	0.001673	0.001499	0.001542
20/20 policy, seed 1	257	210	190	0.003172	0.005345	0.002700
20/20 policy, seed 42	257	834	831	0.001321	0.001325	0.001377
`trial 5`, seed 42	257	1200	1191	0.000719	0.000850	0.000754
Final production, seed 42	257	1525	1505	0.000493	0.000599	0.000673

Table 13: Chronology of the ordinary-training trials. Values are read from the corresponding `final_metrics.csv` files under `results/ordinary-training`; the batch sizes and completed epochs are checked against each run's metadata and diagnostic history. The old stopping-rule row predates the 20/20 policy.

The initial run showed recurrent physical-MSE excursions that affected training and validation together. In `slurm-55367279_trial_1`, the history contains the reported spike at epoch 93, where training and validation MSE reached 0.0159286 and 0.0164889, respectively. This pattern is consistent with a noisy update affecting the model as a whole rather than with validation-only overfitting. After the batches were balanced, the large three-to-four-fold excursions disappeared and the largest non-initial training MSE fell to approximately 0.00819, while the largest final gradient and update ratio also decreased. The final test MSE nevertheless changed from 0.0018235 to 0.0020304. Consequently, balancing improved the numerical behaviour without improving the final accuracy for this pair of runs; stability and accuracy must be considered separately.

The move to $B = 257$ removed the tail completely because $1542 = 6 \times 257$. However, a 200-epoch run then contained only 1200 Adam updates, compared with 5000 updates for the 64-sample runs. Its final test MSE of 0.0026358 is therefore not a clean measurement of batch size alone. The larger-batch trial was subsequently extended to 834 epochs, giving $834 \times 6 = 5004$ updates, which is comparable with the original 5000-update budget. The seed-42 run with the earlier stopping rule reached epoch 527 and retained epoch 470, with a test MSE of 0.0019236. This comparison motivated changing the stopping policy rather than treating the first longer run as the final answer.

The overfitting check is based on the physical training and validation MSE evaluated after the optimizer updates at every epoch. An epoch satisfies the gap condition in Equation (71) when

$$\mathrm{MSE}_{\mathrm{val},e} > (1+\rho)\,\mathrm{MSE}_{\mathrm{train},e}, \qquad \rho = 0.15, \tag{71}$$

but this condition is not sufficient by itself in the current implementation. The earlier rule stopped at the first qualifying epoch. With the 834-epoch configuration, that rule stopped the seed-0 and seed-1 reruns at epochs 2 and 4, after the initial optimizer transient, and their test MSE values were 0.0170249 and 0.0132663. The seed-1 run illustrates the role of checkpoint restoration: training reached epoch 4, but its best validation checkpoint was epoch 1. These results indicate that a single noisy excursion can be mistaken for sustained overfitting.

The revised policy retains the same 15% ratio but requires at least 20 completed epochs and 20 consecutive qualifying epochs. A new validation minimum resets the streak, as does a return below the ratio, and the model stores a parameter snapshot whenever the validation MSE improves. Once the streak reaches 20, `Trainer` stops and broadcasts the best stored parameters before recomputing the final training and validation metrics. This policy was tested with the same six-batch, 834-epoch configuration for seeds 0, 1, and 42. Seed 0 completed the budget and selected epoch 743, seed 1 stopped at epoch 210 after the epochs 191–210 formed the qualifying streak and restored epoch 190, and seed 42 completed the budget and selected epoch 831. The corresponding test MSE values were 0.0015422, 0.0026997, and 0.0013773. Their mean decreased from 0.0107383 with the old rule to 0.0018730 with the 20/20 policy, an 82.56% reduction. The remaining seed dependence shows that the policy addresses premature termination and checkpoint choice, but does not remove all variation caused by the holdout split and initialization.

The final production run used the ordinary path in `CNN/main.cpp`, not the large topology selected in the separate topology search. It used a 5×5 convolution with eight channels and stride five, LeakyReLU with $\alpha = 0.05$, scalar concatenation, and the dense head $(128, 64, 1)$. The command recorded in `training.sh` used Adam with learning rate 10^{-3}, physics weight 0.10, no dropout or L1 or L2 penalty, element-wise gradient clipping at 1, global batch size 257, seed 42, 64 MPI ranks, and a maximum of 2500 epochs. The ordinary split contains 1542 training samples and 171 validation samples; the 429-sample test set is loaded only after training and checkpoint restoration.

The final history begins with the same finite optimizer transient seen in the larger-batch trials: training and validation MSE were 0.0549925 and 0.0571277 at epoch 1, and 0.0593714 and 0.0644453 at epoch 2. They fell to 0.0187334 and 0.0188926 at epoch 3 and then continued downward on a much longer update budget. At

epoch 1505, the retained checkpoint had training MSE 0.000493208 and validation MSE 0.000599464. Epochs 1506–1525 all satisfied the gap condition without improving the validation minimum, so the run stopped at epoch 1525 and restored epoch 1505. Figure 31 shows the complete physical-MSE history in both linear and logarithmic y-axis views and distinguishes these two epochs. The SIMM training objective is not plotted because it is a different standardized quantity from the physical MSE.

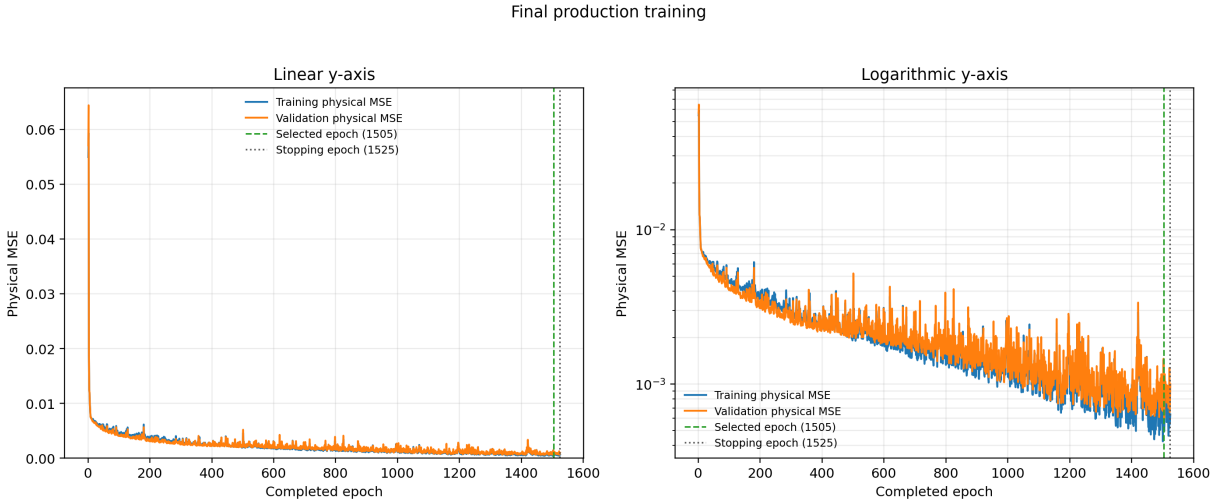


Figure 31: Physical training and validation MSE for the final production run. The left panel uses a linear y-axis and the right panel uses a logarithmic y-axis; both contain all 1525 completed epochs. The dashed line marks the selected epoch 1505 and the dotted line marks the stopping epoch 1525. Both panels use physical-unit MSE, while the standardized SIMM objective is not shown.

The restored production model was then evaluated on the test set, with the result separated according to the same 10° angle threshold used by the SIMM mask. The implementation assigns a sample to the high-angle group only when $|\alpha| > 10°$, so samples exactly at $\pm 10°$ remain in the low-angle group. Table 14 reports the complete split. The high-angle MSE was 0.000818557 larger than the low-angle MSE and was 2.497 times as large. In RMSE terms, taking the square root of the two stored MSE values gives 0.02338 and 0.03695 for the low- and high-angle groups. The high-angle samples are only $66/429 = 15.38\%$ of the test set, but they account for approximately 31.22% of the total squared error. Their larger error is consistent with the SIMM relation being inactive outside the low-angle regime, although this split alone does not establish that the mask is the cause; the high-angle group also contains fewer samples and a broader target range.

Test subset	Samples	Physical MSE	Difference vs. low	High / low MSE
Overall	429	0.000672714	–	–
$\lvert\alpha\rvert \leq 10°$	363	0.000546783	–	–
$\lvert\alpha\rvert > 10°$	66	0.001365339	+0.000818557	2.497

Table 14: Angle-of-attack split of the final production test metrics read from `test_metrics.csv`. The high-angle difference and ratio are computed from the two stored subset MSE values; no per-sample residual spread was recorded.

Figure 32 gives the same comparison graphically and includes the overall test MSE as a reference. It is a comparison of two physically defined test subsets, not a comparison of two separately trained models. The test set was not used to select the stopping epoch, and the figure therefore describes the aggregate subset error of the one restored production model rather than a new tuning criterion.
The final values should not be compared with every cross-validation value without qualification. The tuning experiments use fold-specific normalization and approximately 80 percent of the 1713 training samples in each fold, whereas ordinary training uses a 90/10 split with 1542 training samples and 171 validation samples. The production source also retains the smaller $(128, 64)$ head, while the topology search selected the larger head. The final numbers are therefore the performance of the recorded production configuration, not a test score for the large topology winner. They are nevertheless the appropriate values for describing the executable that was finally used.

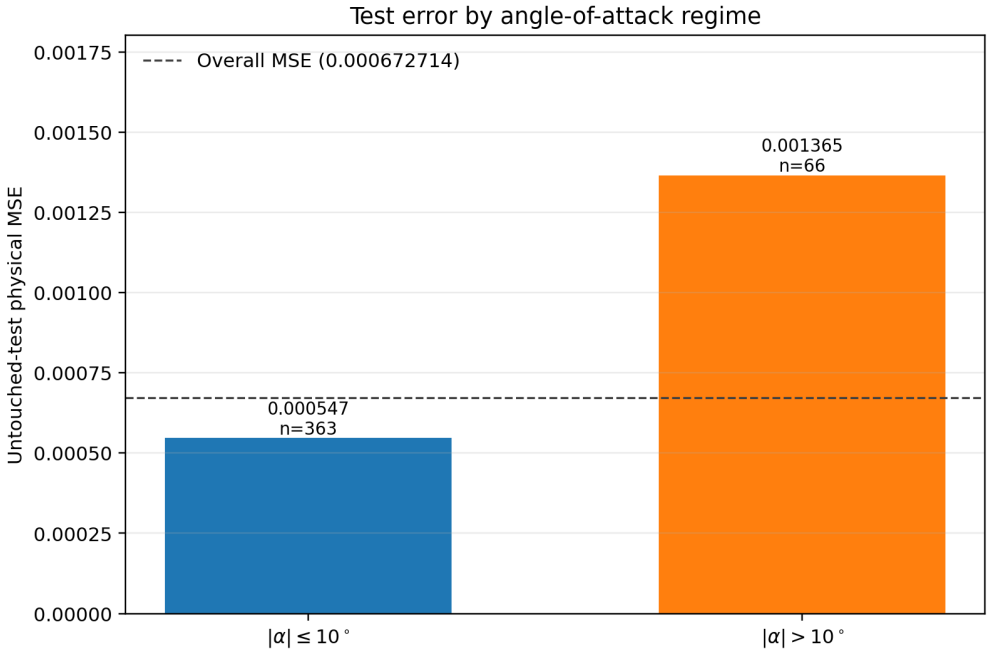


Figure 32: Untouched-test physical MSE for the restored seed-42 production model, split into the low-angle ($|\alpha| \leq 10°$) and high-angle ($|\alpha| > 10°$) regimes. The bars are based on 363 and 66 samples, respectively, and the dashed line marks the overall MSE over all 429 test samples.

4.10. Conclusions

The tuning campaign shows that the largest improvement among the structural choices came from replacing the initial $(128, 64)$ dense head with the selected topology `conv5x5-dense-1024-512-256-128`. Its cross-validated physical MSE was 0.004560, compared with 0.006010 for the topology baseline, and the improvement was observed in all five folds. Adam was the best optimizer at the tested rate 10^{-5}, with a mean validation MSE of 0.004427. The activation comparison selected LeakyReLU with $\alpha = 0.05$, and the learning-rate search selected a constant Adam rate of 10^{-3}, which reduced the cross-validated error to 0.003609 for that experiment. These choices are supported not only by the mean scores, but also by finite diagnostics, small actual update ratios, and the absence of late-epoch divergence in the selected candidates.

The regularization results were different from what might be expected for a large network. Both dropout and weight penalties increased the recorded validation error, and the physics-weight sweep also selected the zero-weight reference. The dropout and physics-weight values must be read as historical results from the earlier $(128, 64)$ head, because those searches were completed before their C++ drivers were corrected to construct the selected large topology. The mismatch is explicitly documented beside the result trees, and no corrected numerical claim is made without a new run. For the large topology, the L1 and L2 study likewise selected zero penalty, indicating that at the tested learning rate and 100-epoch budget the reduction in the training and weight norms did not compensate for the loss in fitting accuracy.

The final ordinary training produced a substantially lower physical test MSE of 0.0006727 after training the production model and restoring its best checkpoint. The high-angle test subset remained more difficult, with an MSE 2.497 times the low-angle value, although it represented only 66 of the 429 test samples. The final result reflects the full training procedure, the longer 2500-epoch budget, the 90/10 ordinary split, and the production defaults in `CNN/main.cpp`; it is not an unbiased test estimate for every tuning candidate. Overall, the experiments demonstrate that the accuracy of the surrogate depends on a coordinated choice of topology, optimizer, activation, learning rate, batch construction, and stopping policy rather than on adding regularization indiscriminately. They also show why numerical diagnostics and explicit provenance are necessary: a low validation value is meaningful only when the topology, split, metric, and training configuration that produced it are stated alongside the result.

5. Conclusions

This work successfully demonstrates an integrated approach to aerodynamic analysis, combining high-fidelity numerical simulation with a data-driven surrogate model. The primary achievement was the extension of the `LifeX` library to handle turbulent external aerodynamic flows through the integration of the **Spalart-Allmaras RANS model**. This new capability was validated on the **Leonardo HPC cluster**, establishing a robust pipeline for generating reference-quality simulation data. Additionally, a **Convolutional Neural Network** was developed and trained on this data, creating a surrogate model capable of near-instantaneous aerodynamic prediction.

The CFD simulations of the NACA airfoil produced lift coefficients that were physically consistent, though moderately underestimated compared to some experimental data. This behavior is physically consistent with the modeling choices made: the VMS-LES formulation employed in the Navier-Stokes step acts as an implicit filter that dissipates small-scale turbulent structures, suppressing the contribution of fine-grained vortical activity to the overall lift generation. Nonetheless, the results remain well within the range of published numerical and experimental values, confirming the physical validity of the simulation framework by Ladson (1988) and McCroskey (1988).

A further modeling assumption concerns the compressibility of the flow. The simulations were performed at a Mach number significantly below 0.3, a regime in which the relative density variations induced by pressure fluctuations are negligible, making the incompressible Navier-Stokes equations an appropriate model.

Regarding the deep learning implementation, the results demonstrate that a relatively simple Convolutional Neural Network architecture can guarantee robust performance, achieving a Mean Squared Error of 0.0104 on the validation set. The primary advantage of this approach lies in its computational efficiency: while a high-fidelity CFD simulation requires approximately 3 hours on an HPC node, the trained CNN inference time is under one second. Even when accounting for the 5 minutes and 11 seconds required for training, this deep learning framework achieves a remarkable $35.11\times$ speedup compared to conventional methods. This outcome strongly aligns with the premises established by Cuozzo et al. (2026), extending their insights to regular profiles, whereas their approach specifically accounted for deformed airfoils due to ice accretion.

This contrast between computational cost and prediction speed places our work within the context of modern aerodynamic design. From a broader perspective, the high-fidelity approach developed here occupies the opposite end of the cost spectrum from purely surrogate-based methods. For instance, the framework proposed by Cuozzo et al. (2026) achieves a $10^3 - 10^4\times$ reduction in wall-clock time at the cost of a $\sim 14\%$ underestimation of the iced lift curve slope. The present work, instead, prioritizes physical fidelity to provide the reference-quality data that such surrogate models ultimately depend on for training and validation. The two approaches are therefore complementary rather than competing, and the framework developed here constitutes a natural candidate as a high-fidelity data source for future surrogate-based pipelines targeting iced airfoil configurations.

Nevertheless, certain limitations inherent to each method must be acknowledged. These include the computational demand of the RANS simulations and, for the data-driven model, the absence of an *a priori* error estimate, the strict dependency on a training dataset that accurately resembles the target inference domain[22], and the general lack of interpretability characteristic of black-box models.

Looking forward, the framework developed here opens up promising avenues for engineering design and analysis. The speed of the trained CNN makes it ideal for real-time applications and rapid design iterations on consumer-grade hardware, completely bypassing the need for constant HPC access. This capability is particularly relevant for the agile aerodynamic design and field-testing of components for Unmanned Aerial Vehicles and quadcopters, where on-the-fly performance evaluation can accelerate development cycles. The simulation pipeline itself is now poised to serve as a data engine for training future surrogate models on more complex geometries, such as the iced airfoils that motivated this study.

[22] As noted, since the training data from the CFD was slightly biased, we expect this bias persists in the surrogate model.

6. Contributors

Name	CNN API	Topology	Dropout	Physics Weights	Report	Data Struct.	Optimizer	Regularization	Activation	Learning Rate	Training
Giulio Enzo Donninelli	✓	✓			✓	✓		✓	✓		✓
Alessia Rigoni	✓		✓	✓	✓		✓			✓	
Vittorio Sironi			✓	✓							
Alessandro Verrengia		✓						✓			

Table 15: Contributions to the work presented in this report. A checkmark indicates participation in the corresponding task.

References

P. C. Africa, I. Fumagalli, M. Bucelli, A. Zingaro, M. Fedele, L. Dedè, and A. Quarteroni. lifex-cfd: An open-source computational fluid dynamics solver for cardiovascular applications. *Computer Physics Communications*, 296:109039, 2024.

Ciro Cuozzo, Vincenzo Cusati, Agostino De Marco, Salvatore Corcione, James H. Page, and Alessandro Zanon. Ai-driven surrogate modeling for iced tailplane aerodynamic prediction. *Aerospace Science and Technology*, 168, 2026.

John Duchi, Elad Hazan, and Yoram Singer. Adaptive subgradient methods for online learning and stochastic optimization. *Journal of Machine Learning Research*, 12(61):2121–2159, 2011. URL `https://jmlr.org/papers/v12/duchi11a.html`.

Xavier Glorot and Yoshua Bengio. Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics*, volume 9 of *Proceedings of Machine Learning Research*, pages 249–256. PMLR, 2010. URL `https://proceedings.mlr.press/v9/glorot10a.html`.

Priya Goyal, Piotr Dollár, Ross Girshick, Pieter Noordhuis, Lukasz Wesolowski, Aapo Kyrola, Andrew Tulloch, Yangqing Jia, and Kaiming He. Accurate, large minibatch SGD: Training ImageNet in 1 hour. Technical Report arXiv:1706.02677, arXiv, 2017. URL `https://arxiv.org/abs/1706.02677`.

Arthur E. Hoerl and Robert W. Kennard. Ridge regression: Biased estimation for nonorthogonal problems. *Technometrics*, 12(1):55–67, 1970. doi: 10.1080/00401706.1970.10488634. URL `https://doi.org/10.1080/00401706.1970.10488634`.

Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations*, 2015. URL `https://arxiv.org/abs/1412.6980`.

Charles L. Ladson. Effects of independent variation of Mach and Reynolds numbers on the low-speed aerodynamic characteristics of the NACA 0012 airfoil section. Technical Report NASA TM-4074, NASA Langley Research Center, 1988.

Ilya Loshchilov and Frank Hutter. Sgdr: Stochastic gradient descent with warm restarts. In *International Conference on Learning Representations*, 2017. URL `https://arxiv.org/abs/1608.03983`.

W. J. McCroskey. A critical assessment of wind tunnel results for the NACA 0012 airfoil. Technical Report NASA TM-100019, NASA Ames Research Center, 1988.

Philippe R. Spalart and Steven R. Allmaras. A one-equation turbulence model for aerodynamic flows. *AIAA Paper*, 92-0439, 1992.

Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(56): 1929–1958, 2014. URL `https://jmlr.org/papers/v15/srivastava14a.html`.

Robert Tibshirani. Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society Series B: Statistical Methodology*, 58(1):267–288, 1996. doi: 10.1111/j.2517-6161.1996.tb02080.x. URL `https://doi.org/10.1111/j.2517-6161.1996.tb02080.x`.